

ТЕХНИЧЕСКИ УНИВЕРСИТЕТ – СОФИЯ

**ФАКУЛТЕТ „КОМПЮТЪРНИ СИСТЕМИ И
ТЕХНОЛОГИИ“**



**МНОГОПЛАТФОРМЕН ГРАФИЧЕН
ИНТЕРФЕЙС: ПРОЕКТИРАНЕ И
ЕКСПЕРИМЕНТАЛНА ОЦЕНКА НА
НЕЗАВИСИМОСТТА ОТ ПЛАТФОРМЕННИЯ
ИНСТРУМЕНТАРИУМ**

ДИПЛОМНА РАБОТА

за придобиване на ОКС „Магистър“

Изготвил:

Алекс Цветанов, фак. № **[TODO: фак. номер (магистратура)]**

Специалност: **[TODO: специалност]**

Научен ръководител:

[TODO: звание, име на ръководителя]

София, 2026

**[TODO: Тук се подвързва УТВЪРДЕНОТО ДИПЛОМНО ЗАДАНИЕ (подписано, с
печат)]**

[TODO: Тук се подвързва ДЕКЛАРАЦИЯТА ЗА АВТОРСТВО]

СЪДЪРЖАНИЕ

УВОД	1
I. АНАЛИЗ НА ПРЕДМЕТНАТА ОБЛАСТ И ЛИТЕРАТУРЕН ПРЕГЛЕД	4
1.1. Подходи за разработка на многоплатформени графични интерфейси	4
1.2. Съществуващи многоплатформени фреймуорци	6
1.3. Архитектурата на .NET MAUI	7
1.4. Изводи от литературния преглед	9
II. АНАЛИЗ НА ВЪЗМОЖНИТЕ РЕШЕНИЯ И ОБОСНОВКА НА ИЗБОРА	10
2.1. Критерии за оценка	10
2.2. Избор на целеви език и на езиковия стандарт	11
2.3. Стратегия за изграждане	15
2.4. Модел на собствеността върху обектите	17
2.5. Избор на платформени слоеве и на реда им	18
2.6. Подход към декларативното описание	19
2.7. Обобщение на направения избор	20
III. ПРОЕКТИРАНЕ НА СИСТЕМАТА	21
3.1. Синтез на общата структура	21
3.2. Проектиране на слоя за графични примитиви	23
3.3. Договори на виртуалните изгледи	24
3.4. Проектиране на слоя за преобразуване	24
3.5. Проектиране на алгоритмите за разположение	27
3.6. Проектиране на системата от свойства	29
3.7. Проектиране на потребителския програмен интерфейс	31
3.8. Формат на декларативното описание	32
3.9. Проектиране на платформените слоеве	32
IV. ПРОГРАМНА РЕАЛИЗАЦИЯ	33
4.1. Обща структура на изходния код	33
4.2. Организация на компилацията	33
4.3. Реализация на слоя за преобразуване	34

4.4. Реализация на алгоритмите за разположение	36
4.5. Реализация на системата от свойства	36
4.6. Реализация на платформените слоеве	37
4.7. Реализация на декларативния слой	40
4.8. Обработка на събития от интерфейса	41
4.9. Дисциплина на разработка и осигуряване на верността	42
V. МЕТОДИКА ЗА ЕКСПЕРИМЕНТАЛНА ПРОВЕРКА НА	
ЕКВИВАЛЕНТНОСТТА	45
5.1. Постановка на експеримента	45
5.2. Автоматизирано заснемане	46
5.3. Правила за оценяване	48
5.4. Количествено оценяване	50
5.5. Достоверност на измерването	53
5.6. Предварителна регистрация на хипотезите	55
VI. ЕКСПЕРИМЕНТАЛНИ РЕЗУЛТАТИ И АНАЛИЗ	56
6.1. Степен на съвпадение	56
6.2. Представителни съпоставки	59
6.3. Систематизация на нарушенията на независимостта	65
6.4. Цена на реализацията	71
6.5. Ограничения на изследването	76
ЗАКЛЮЧЕНИЕ	77
ЛИТЕРАТУРА	81
ПРИЛОЖЕНИЯ	85
Приложение А. Пълен изходен текст на срез Button	85
Приложение Б. Публичен програмен интерфейс на системата	88
Приложение В. Пълна таблица на съвпадението по интерфейси	114
Приложение Г. Екранни форми	125

УВОД

Разработката на приложение с графичен потребителски интерфейс за няколко операционни системи едновременно поставя задача, чието обичайно решение е приемано без проверка. Всеки от съществуващите многоплатформени инструменти твърди, че едно описание на интерфейса е достатъчно, за да се получи еднакво поведение върху различни платформи. Твърдението се приема на доверие, защото проверката му е трудна: за разлика от изчислителния код, чиято коректност се доказва с тест, „правилният“ вид на един потребителски интерфейс не е записан никъде във вид, който машина може да сравни. Това е частен случай на *проблема за оракула* — невъзможността да се отличи желаното от нежеланото поведение при даден вход — разгледан систематично в [1].

Настоящата работа изхожда от наблюдението, че тази проверка не е принципно невъзможна, а само лишена от еталон. Ако съществува зряла реализация, за която е прието, че рендира коректно, тя може да бъде използвана не като образец за подражание, а като *измервателен инструмент* — източник на еталонни изображения, спрямо които независима реализация на същото описание се сравнява количествено. Смяната на ролята е принципна: съпоставяната реализация престава да бъде „копие“ и става второ, независимо решение на една и съща задача, а разликите между двете стават измеримо свойство на абстракцията, а не въпрос на мнение. Този похват — независими реализации на една спецификация, сравнени една с друга — е познат в тестването като *диференциално тестване* [2] и е роднина на *N-версийното програмиране* [3]; ролята на зрялата реализация като меродавен еталон съответства на *golden-master* тестването [4].

Целта на дипломната работа е да установи до каква степен наблюдаваното поведение на графичен потребителски интерфейс може да бъде описано независимо от платформения инструментариум, който го изобразява, и къде точно тази независимост се нарушава — като изгради работеща реализация на такова описание върху няколко разнородни native инструментариума, методика за количествена проверка на еквивалентността на изобразяването и емпирична оценка на цената на избраната стратегия.

За постигането на целта са формулирани следните **задачи**:

1. литературен преглед на подходите за многоплатформено описание на потребителски интерфейс и анализ на архитектурата, върху която стъпва изследването;

2. анализ на възможните проектни решения — целеви език, стратегия за изграждане, модел на собствеността върху обектите, набор от платформени слоеве — и обосновка на направения избор;
3. проектиране на слоеста архитектура, в която описанието на интерфейса, изчисляването на разположението и системата от свойства са преносими, а платформено обусловената част е сведена до таблично зададено съответствие;
4. програмна реализация на архитектурата върху четири native инструментариума и един слой без екран;
5. изграждане на методика и инструментариум за автоматизирано измерване на еквивалентността на изобразяването между две независими реализации на едно и също описание;
6. експериментална проверка върху пълния набор от интерфейси, извеждане на систематизация на установените разминавания и оценка на цената на реализацията.

Обхватът на работата е логическият слой, който описва интерфейса, изчислява разположението и управлява свойствата, заедно с платформените слоеве, необходими за неговото изобразяване. Извън обхвата остават съвместимостните слоеве към предходни поколения на еталонната реализация, вътрешните ѝ помощни пространства от имена и допълненията, които не са част от ядрото ѝ.

Методологична бележка. Хипотезите за цената на реализацията — размер на артефакта, потребена памет, време за реакция — са *регистрирани предварително*, преди да е съществувало каквото и да е измерване, заедно с изискванията за валидност, на които всяко измерване трябва да отговаря. Целта е една погрешна прогноза да остане видима и да бъде отчетена като погрешна, вместо да бъде реконструирана след факта. Регистрацията е част от документацията на експеримента и е приложена към работата.

Структура на изложението. Раздел I разглежда предметната област и архитектурния шаблон, върху който стъпва изследването. Раздел II излага разгледаните проектни алтернативи и обосновава избора между тях. Раздел III представя проектирането на системата, а Раздел IV — нейната програмна реализация. Раздел V описва методиката за измерване на еквивалентността, която е самостоятелен принос на работата и е предпоставка за валидността на всичко, което следва. Раздел VI представя получените резултати — систематизацията на установените нарушения на независимостта и

измерената цена на реализацията. Заключението обобщава приносите, ограниченията и насоките за по-нататъшна работа.

I. АНАЛИЗ НА ПРЕДМЕТНАТА ОБЛАСТ И ЛИТЕРАТУРЕН ПРЕГЛЕД

Разделът е подреден от общото към частното: първо класификация на подходите според това какво в крайна сметка рисува пиксел на екрана, после конкретните представители на всяко семейство, и накрая — по-подробно — архитектурата на реализацията, която в тази работа изпълнява ролята на еталонен измервателен инструмент, а не на образец за копиране. Последното е разгледано подробно, защото границата, която тя въвежда, е и предметът на измерването в Раздел VI.

1.1. Подходи за разработка на многоплатформени графични интерфейси

Съществуващите решения се разделят на три семейства според това *какво* в крайна сметка рисува пиксел на екрана. Разделението не е самоцелна класификация — то определя всичко останало: вида на крайния контрол, размера на артефакта, достъпа до платформените услуги и — което е предметът на настоящата работа — *дали изобщо съществува понятие „правилен вид“*, спрямо което решението да бъде проверено.

1.1.1. Абстракция над native контроли

Едно описание се преобразува към контролите на *самата* платформа: списъкът е native списък, бутонът е native бутон. Предимствата идват от само себе си — външният вид, поведението при въвеждане, достъпността и езиковите настройки идват от платформата и се променят заедно с нея.

Цената е, че абстракцията трябва да *съгласува* инструментариуми, които се различават не само по имена, а по модел: различни правила за разположение, различни жизнени цикли, различни конвенции за координати. Именно тук възниква въпросът, който настоящата работа измерва — доколко такова съгласуване е постижимо и къде се проваля.

1.1.2. Собствен растеризатор

Фреймуоркът не използва native контроли, а рисува всеки елемент сам, върху графично платно. Резултатът е еднакъв на всички платформи *по замисъл*, което премахва целия клас проблеми на съгласуването.

Замяната обаче е реална: интерфейсьт престава да бъде native. Той не наследява автоматично промените във външния вид на платформата, а достъпността, въвеждането с клавиатура и текстовите услуги трябва да бъдат възпроизведени във фреймуорка. Артефактът носи и самия растеризатор.

1.1.3. Вграден уеб изглед

Интерфейсьт е уеб документ в native обвивка. Този подход има най-ниска цена на входа и най-голяма отдалеченост от платформата; той е и подходът с най-обилно изследвана цена в литературата [5].

1.1.4. Критерии за съпоставяне

Таблица 1. Съпоставяне на трите подхода за многоплатформен графичен интерфейс

Критерий	Над native контроли	Собствен растеризатор	Вграден уеб изглед
краен контрол	native	собствен	документен елемент
външен вид след промяна в платформата	следва я сам	изисква работа	изисква работа
достъпност, въвеждане	от платформата	във фреймуорка	от уеб слоя
размер на артефакта	малък	+ растеризатор	+ уеб механизъм
среда за изпълнение	по избор	обичайно да	да
наличие на еталон	да	не (сам си е еталон)	частично

Последният ред е решаващият за настоящата работа. При собствен растеризатор въпросът „съответства ли изобразеното“ не стои — изобразеното *по определение* е правилното, защото няма втора инстанция, спрямо която да се сравнява. Само при абстракция над native контроли твърдението за съответствие е смислено, проверимо — и, както показва §1.4, непроверено.

1.2. Съществуващи многоплатформени фреймуорци

1.2.1. Xamarin.Forms и .NET MAUI

Историческата линия е поучителна, защото съдържа точно един архитектурен завои. Xamarin.Forms [6] свързва всяка контрола с платформената чрез *renderer* — клас, който наследява по йерархия и включва както създаването на *native* изгледа, така и пренасянето на всяко свойство. Наследяването означава, че разширяването на една контрола изисква наследяване на нейния *renderer*, а той е обвързан с платформата.

.NET MAUI [7, 8] заменя този механизъм с *handler*: обектът вече не се наследява, а носи *таблица* от съответствия „свойство → действие“. Разликата е между разширяване чрез наследяване и разширяване чрез вписване в таблица — второто е съставимо, не изисква познаване на йерархията и позволява една контрола да бъде доразвита, без нищо да се наследява. Именно този шаблон е предметът на §1.3 и основата, върху която стъпва настоящата работа.

1.2.2. Flutter

Представител на семейството със собствен растеризатор [9]. Изгражда интерфейса от собствени примитиви върху графично платно, което дава пълна еднаквост между платформите и пълна свобода в анимациите, но и означава, че всяка контрола, всяко поведение при въвеждане и всяка услуга за достъпност съществуват във фреймуорка, а не в платформата.

По критериите от 1.1 Flutter заема крайното положение на второто семейство: пълна еднаквост между платформите, пълна независимост от промени в *native* инструментариума, и артефакт, който носи целия слой за изобразяване. За настоящото изследване той е показателен по обратен начин — при него описаният тук метод за измерване е *неприложим*, защото липсва втора независима реализация на същото описание, спрямо която да се сравнява.

1.2.3. Qt

Най-старият зрял C++ фреймуорк в областта [10] и единственият пряк съсед на настоящата работа по език. Съдържа и двата подхода: по-старият слой със *widgets* рисува сам, като подражава на външния вид на платформата, а по-новият декларативен слой е самостоятелен растеризатор със собствен език за описание.

Разглеждането му е особено уместно, защото Qt решава — на същия език — и задачата за собствеността без събирач на боклук: дървото от обекти е с притежаване от родител към дете, а наблюдаващите препратки са отделен тип. Настоящата работа стига до сходно разпределение на ролите по независим път (§2.4), което е довод, че решението е принудено от задачата, а не въпрос на предпочитание.

1.2.4. Avalonia, Slint и React Native

Avalonia [11] е собствен растеризатор върху управлявана среда, с декларативен език, силно повлиян от предходно поколение настолни технологии; позиционира се като преносим наследник на тези технологии, което го поставя във второто семейство въпреки близостта на езика му за описание до първото.

Slint [12] също рисува сам, но с вграден собствен език за описание и с изричен прицел към устройства с ограничени ресурси — което го прави най-близкия по цели представител на второто семейство за вградени приложения.

React Native [13] принадлежи на първото семейство: изгражда native контроли, но описанието се изпълнява в скриптов език, а границата между двете страни е асинхронна. Точно тази граница е предметът на измерването в [5] — изследване, което мери *цената* на връзката, но не и съответствието на изобразеното.

1.3. Архитектурата на .NET MAUI

Централната идея, върху която стъпва цялата работа, е че разглежданата реализация **не е растеризатор**. Тя не рисува нито един пиксел сама. Тя е абстракция, която преобразува едно многоплатформено описание на интерфейса към няколко native инструментариума, и е организирана около един-единствен повтарящ се шаблон, приложен еднакво за всяка контрола.

1.3.1. Шаблонът Handler: виртуален изглед, mapper, native контрола

Връзката е тристранна и е показана на (Фиг. 1). Най-простият ѝ срез — бутон — се състои от: *договор* (интерфейс само със свойства, без изобразяване); *обработчик*, който носи две таблици — за свойства и за императивни операции; и *native контрола* на всяка платформа. Таблицата за свойства съпоставя име на свойство с функция, която прилага текущата му стойност; таблицата за операции прави същото за действия, които нямат

стойност (например „получи фокус“).

Границата минава точно тук: договорът и таблиците са преносими, а само реализациите на отделните действия са платформени. Затова добавянето на инструментариум е *добавяне* на реализации, а не изменение на съществуващи — свойство, което Раздел III запазва съзнателно.



Фигура 1. Шаблонът на преобразуването. Виртуалният изглед носи само свойства; обработчикът съпоставя всяко свойство на действие, което го прилага върху native контролата. Смяната на инструментариума заменя само дясната колона.

1.3.2. Слоеста организация и ред на зависимостите

Зависимостите между слоевете вървят в една посока — графични примитиви, договори, обработчици, платформени слоеве, разположение, контроли, декларативен слой, услуги на устройството, инициализация — и този ред е задължителен и при независимо изграждане на такъв слой. Причината не е организационна: слой, който може да достигне до платформения, престава да бъде преносим, дори ако в момента не го достига. Пълното разглеждане на структурата, вече в реализирания вид, е в §3.1.

1.3.3. Системата от свойства и свързването на данни

Системата от свойства е вторият механизъм, който се повтаря навсякъде. Всяко свойство е описано от споделен описател (име, стойност по подразбиране, обратни извиквания), а всяка инстанция държи само стойността. Върху това стъпват три неща: известяване при промяна — което задвижва и таблиците от §1.3; *привеждане* (coercion) на стойността към допустим обхват; и *предимство* между едновременно валидни източници на една и съща стойност — подразбираща се, от стил, от свързване, от ръчно присвояване и т.н.

Предимството е частта, която най-лесно се подценява. То не е вътрешна подробност, а наблюдаемо поведение: то определя дали изрично зададен от приложението цвят надделява над цвят от стил, и дали „незададено“ се различава от „зададено на стойността

по подразбиране“. Раздел VI показва, че точно това разграничение е източник на цял клас разминавания.

1.4. Изводи от литературния преглед

Прегледът води до четири констатации, всяка от които се връзва със задача от Увода.

Първо. Твърдението за еднакво поведение върху различни платформи е *общо за цялото семейство* решения, които стъпват на native контроли, и никъде не е придружено от количествена проверка. Съпоставителните изследвания измерват друго — режимни разходи на границата към платформата [5], потребление на ресурси, удобство за разработчика — но не и степента, в която двете реализации на едно описание изобразяват едно и също.

Второ. Причината за тази празнина не е незаинтересованост, а липса на *оракул*: за разлика от изчислителния код, „правилният“ вид на интерфейс не е записан във вид, годен за машинно сравнение [1]. Изследванията върху визуално тестване на интерфейс [14] заобикалят това, като сравняват система със *самата нея* в предишен момент — което открива регресии, но не може да провери твърдение за съответствие между две различни реализации.

Трето. Съществува обаче положение, при което оракул е наличен: когато една зряла реализация се приеме за меродавна и втора, независима реализация изпълнява *същото* описание. Тогава разликата между двете изображения е измерима величина. Това е наблюдението, върху което стъпва настоящата работа.

Четвърто. Архитектурата с таблично преобразуване (§1.3) е особено подходяща за такава проверка, защото границата между преносимото и платформеното е *един* и е явен. Реализация, която го запазва, позволява разминаванията да бъдат отнасяни към определен слой; реализация, която го събаря в един растеризатор, не позволява това.

Тези четири констатации определят и подредбата на останалото изложение: избор на средства и стратегия (Раздел II), запазване на тази граница при проектиране и реализация (Раздели III и IV), изграждане на оракула в работеща методика (Раздел V) и получените от нея резултати (Раздел VI).

II. АНАЛИЗ НА ВЪЗМОЖНИТЕ РЕШЕНИЯ И ОБОСНОВКА НА ИЗБОРА

Всяко от разгледаните по-долу решения е било действителна възможност в момента на вземането му, а не реторична алтернатива, въведена за да бъде отхвърлена. Разделът излага първо критериите, после вариантите по всяко от петте определящи решения, и накрая — обобщена оценка. Където решението почива на измерване, а не на преценка, измерването е посочено.

2.1. Критерии за оценка

За да не бъде изборът въпрос на вкус, вариантите се съпоставят по пет предварително изброени критерия:

- К1. **Проверимост** — може ли резултатът от решението да бъде проверен автоматизирано, срещу външен източник на истина, без човешка преценка.
- К2. **Архитектурна вярност** — запазва ли решението разделението, което прави описанието преносимо, или го размива.
- К3. **Независимост от среда за изпълнение** — изисква ли решението допълнителна среда, вградена в крайния артефакт.
- К4. **Безопасност по конструкция** — превръща ли решението цял клас грешки в невъзможни, вместо да разчита на дисциплина.
- К5. **Цена при добавяне на платформа** — какво трябва да се напише отново при всеки нов платформен слой.

Критерий К1 е поставен пръв съзнателно. Работата изследва твърдение за еквивалентност; решение, което подобрява който и да е друг критерий за сметка на проверимостта, обезсмисля изследването.

2.2. Избор на целеви език и на езиковия стандарт

Таблица 2. Разгледани целеви езици по критериите K1–K5

Критерий	C++23	Rust	Zig	C + обвивки
K1 проверимост	пълна (пренасяне на тестовите е пряко)	пълна	пълна	пълна
K2 архитектурна вярност	висока	висока	висока	ниска (без обобщени типове)
K3 независимост от среда	да	да	да	да
K4 безопасност по конструкция	чрез типовата система	най-висока	средна	ниска
K5 цена за платформа	най-ниска (пряк достъп)	висока (обвързване)	висока	средна

Rust печели по K4 — проверката на заемането прави безопасността свойство на езика, а не на дисциплината, и то с доказани основания [15]. Загубва обаче по K5, който тук тежи повече: три от петте целеви инструментариума са с C или C++ интерфейс, а за Apple платформите смесеният език дава достъп без нито един ред обвързващ код. При Rust всеки от петте слоя би започнал с изграждане на обвързване. Тъй като K5 определя дали изобщо са постижими пет платформени слоя в рамките на работата, а K4 се оказва постижим и в C++ чрез само преместваеми типове (§2.4), изборът пада върху C++23.

Избран е **C++23**. Решаващият довод по K5 е достъпът до native инструментариумите: три от петте целеви инструментариума се обръщат от C++ без междинен слой за маршалиране, а за Apple платформите смесеният език позволява една единица за превод да вижда едновременно C++ и native обектния модел — native обектите се държат пряко в C++ клас, вместо да се пренасят през граница.

Изборът на *конкретния стандарт* обаче не е формалност и заслужава отделно разглеждане. Задачата има три свойства, които насочват към средства, появили се в последните ревизии на езика: изисква се (а) много работа, която в оригинала се извършва по време на изпълнение, да се премести в превода; (б) заместител на рефлексията, който да остане проверим от компилатора; и (в) модел на собственост, изразим в типовата система, а не в дисциплина. Следващите подпараграфи изброяват използваните средства заедно с

това, което всяко от тях замества, и — което е по-важно — заедно с *честната* преценка къде то се отразява на бързодействието и къде не.

2.2.1. Изчисления по време на компилация

Оригиналът изгражда голяма част от структурите си при стартиране: таблици за преобразуване, идентификатори на типове, описатели на свойства. Всяко от тези изграждания е работа, платена при всяко пускане на приложението.

В C++23 значителна част от тях се изместват в превода. `constexpr` и `constexpr` позволяват изчислението да бъде задължително по време на компилация — не „по възможност“, а проверимо: функция, обявена за `constexpr`, не се компилира, ако бъде извикана в контекст на изпълнение. `if constexpr` допълва картината, като позволява един и същ алгоритъм да има различна реализация в двата режима, без да се поддържат два отделни текста.

Пряката последица е, че наименуваните константи и таблиците, изградени по този начин, са *константно инициализирани* — те съществуват в изходния артефакт вече готови и не се изграждат при пускане. Второстепенната, но не по-малко ценна последица е, че така се избягва неопределеният ред на инициализация между единиците за превод, който при динамично изградени глобални таблици е класически източник на дефекти, зависещи от реда на свързване.

Мярка за дела на началното време, който отпада по този начин, *не* се привежда тук: тя изисква измерването по H2b-1, което към момента на предаване не е проведено (§6.4). Механизмът е описан, ефектът е предсказан и регистриран предварително, но числото не се твърди — това е разликата между обосновка и реклама.

2.2.2. Вграждане на описанието в превода

Декларативното описание на интерфейса обичайно се чете от файл и се разбира при стартиране. Средството `#embed` (C++23) позволява съдържанието на файл да стане част от единицата за превод, при което разборът може да се извърши от `constexpr` код — тоест по време на компилация.

Ползата не е само в спестения дисков достъп. Тя е в *момента, в който грешката се открива*: неправилно описание престава да бъде отказ при стартиране и става грешка при компилация. Това е същата смяна на момента, която прави статичната типизация полезна — приложена към формата на интерфейса.

Границата на средството е реална: то не е налично във всички компилатори, които проектът трябва да поддържа. За една от платформите това наложи резервен път — описанието се превръща в масив от байтове чрез отделна стъпка в изграждането, вместо от самия компилатор (§4.7). Резултатът е същият, цената е една допълнителна зависимост в изграждането за тази платформа.

2.2.3. Концепции вместо проверки по време на изпълнение

Концепциите (*concept*, *requires*) заместват два различни механизма на оригинала. Първо — ограниченията върху обобщените типове, които в управлявания език са описани декларативно, но се проверяват отчасти при изпълнение. Второ, и по-важно за тази работа — *незадължителните точки за разширение*. Базовият клас на обработчиците обекти открива по време на компилация дали производният предоставя дадена точка за разширение, чрез изискване за наличие на израз. Следствието е двойно: производният клас изброява само това, което наистина предоставя (няма подразбиращи се реализации, които да бъдат засенчени по невнимание), и извикването на несъществуваща точка е грешка при компилация, а не безшумно нищо.

2.2.4. Типизиран достъп вместо изтриване на типа

Системата от свойства на оригинала е изградена върху общия базов тип на средата за изпълнение: всяка стойност се *опакова*, а извличането ѝ изисква проверка на типа при изпълнение. Тук описателят на свойство е шаблон по типа на стойността, така че стойността по подразбиране, сравнението за равенство и обратните извиквания са типизирани. Не се извършва опаковане на стойностни типове, не се разчита на информация за типа при изпълнение и не се заделя памет за стойност, която се побира в самия обект.

Тук е и мястото на решението предимството между източниците на стойност да бъде опаковано в едно 64-битово число, чийто целочислен ред *съвпада* с реда на предимството (§3.6). Разрешаването на предимство — операция, която се извършва при всяка промяна на всяко свойство — се свежда до едно целочислено сравнение.

2.2.5. Наблюдения без копиране

`std::span` и `std::string_view` позволяват предаването на последователности и низове без заделяне и копиране. В слой, в който имената на свойства се сравняват при всяко преминаване през таблицата, разликата между сравнение на изглед и сравнение на

собствен низ не е козметична — второто заделя памет на място, изпълнявано при всяка промяна.

2.2.6. Детерминистично освобождаване

Липсата на автоматично събиране на боклука е ограничение, но носи и свойство, което управляваната среда няма: *моментът* на освобождаване е известен и се определя от обхвата, а не от събирач — идиомът RAII, при който придобиването и освобождаването на ресурс са обвързани с живота на обект [16]. Разрушителят на контрола освобождава native ресурса ѝ веднага; няма фаза, в която освободената логически памет още се държи.

Тук е необходима сдържаност. Изкушаващо е това да се представи като предимство в бързодействието; предварително регистрираната хипотеза **H2b-3** обаче твърди обратното за установен режим на изобразяване (§5.6), защото при малка купчина събирачът рядко надхвърля бюджета за един кадър, а по-голямата част от времето за кадър се изразходва в native инструментариума — еднакъв и в двата случая. Очакваният измерим ефект е в *следата на паметта* (H2a) и в *началното време* (H2b-1), не в честотата на кадрите.

2.2.7. Само преместваеми типове

Само преместваемите типове са това, което превръща модела на собствеността от съглашение в проверимо от компилатора свойство (§2.4). Езиковото средство — изтрит копиращ конструктор — е старо; новото е, че стандартната библиотека вече го поддържа последователно, включително за съхранение на извиквания обекти, което позволява един обработчик да улавя само преместваемо състояние.

2.2.8. Обобщение: къде езикът действително дава ефект

Средство	Какво замества	Очакван измерим ефект
<code>constexpr/constexpr</code>	изграждане при стартиране	начално време, размер
<code>#embed</code>	четене и разбор при стартиране	начално време; момент на грешката
концепции	рефлексия, проверки при изпълнение	момент на грешката (не бързодействие)
типизиран описател	опаковане на стойности	следа на паметта, заделяния
опаковано предимство	сравнение на обекти	време при промяна на свойство
<code>span/string_view</code>	копия на низове	заделяния по горещия път
детерминистично разрушаване	събиране на боклука	следа на паметта
само преместваеми типове	дисциплина по съглашение	коректност, не бързодействие

Таблицата е нарочно разделена така, че третата колона да е *проверимо твърдение*, а не реклама. Три от осемте средства не се очаква да повлияят на бързодействието изобщо — те плащат в коректност или в момента на откриване на грешката. Смесването на двата вида полза е обичайният начин един технически избор да бъде обоснован с числа, които не го подкрепят.

Установено ограничение, което не е предвидено предварително. Стандартната библиотека на един от използваните компилатори не предоставя типа за само-преместваема функция, макар той да е част от стандарта. Реализацията носи собствена заместваща реализация, отбелязана като временна. Отбелязва се, защото е показателно: изборът на стандарт не гарантира наличие на средствата му във всички компилатори, които проектът трябва да поддържа — това е реална цена на решението, а не подробност.

2.3. Стратегия за изграждане

Разгледани са три стратегии:

1. **Автоматизирано преобразуване** на изходния текст на еталонната реализация. Отхвърлена: преобразуваният текст носи модела на паметта на изходния език, тоест противоречи на К3 и К4, а полученият резултат не е проверим по К1 — няма как да се

твърди, че преобразуването е коректно, освен като се провери, което връща задачата в началото.

2. **Ново прочитане** — архитектура, вдъхновена от еталонната, но изградена наново. Отхвърлена по K1: без съответствие едно към едно между двете реализации не съществува общо описание, което и двете да изпълняват, а следователно и контролно условие за сравнение.

3. **Послойно изграждане, водено от тестове.** Избрана.

Избраната стратегия работи слой по слой отдолу нагоре и за всяка компонента изисква поведението да бъде *прочетено*, а не *предположено*. Изходната информация се взема от три различни източника, всеки от които отговаря на различен въпрос:

Въпрос	Източник	Какво дава
Каква е формата?	описание на публичната повърхност	имена, сигнатури, наличност
Как трябва да се държи?	тестовите на еталона	стойности по подразбиране, гранични случаи, ред
Как е реализирано?	изходният текст на еталона	алгоритми, свързване с платформата

Разделянето не е педантизъм. Формата не съдържа поведение; тестовите не съдържат алгоритъм; изходният текст не казва кое от това, което прави, е договор и кое е подробност. Замяната на един източник с друг е обичайният начин да се получи правдоподобна, но невярна реализация.

Редът на разрешаване при противоречие между трите източника е фиксиран предварително. Между *форма* и *поведение* противоречие практически не възниква — те описват различни неща. Реалният случай е противоречие между *изходния текст* на еталона и *действителното му поведение*, което възниква, защото изходният текст е снимка от един момент, а измерването се извършва срещу разпространената версия. Правилото, записано преди първото му прилагане, е: **предимство има изобразеното**. Изходният текст остава източник за механизма, но не и за стойност, която оттогава се е променила.

Първото му прилагане: изходният текст поставя долна граница от 44 единици за размера на превключвател, а разпространената версия изобразява 18 без никаква долна граница (измерено на две платформи). Системата премахна границата и отбеляза това като

документирано отклонение, цитирайки и двата източника. Резултатът: два интерфейса преминаха от разминаване към точно съвпадение.

2.4. Модел на собствеността върху обектите

Това е решението с най-големи последици, защото засяга всеки ред от програмния интерфейс. Дървото от изгледи по природа съдържа обратни връзки родител $\leftrightarrow$ дете и абонаменти за събития, а средата на еталона ги разрешава чрез автоматично събиране на боклука, каквото тук няма.

2.4.1. Разгледани варианти

(а) **Споделено притежаване навсякъде.** Най-близко до поведението на еталона и най-лесно за писане. Отхвърлено, защото цикълът „контрола притежава обработчик, който улавя контролата“ остава напълно безшумен: паметта никога не се освобождава, а инструментите за откриване на течове не съобщават нищо, защото обектите са *достижими* — просто взаимно. Че простото броене на препратки не събира цикли, е класически резултат [17].

(б) **Арена с индекси вместо указатели.** Решава цикъла по начина на изграждане и дава компактно разположение в паметта — регионният модел на управление на паметта [18]. Отхвърлено по K2: заменя типовете с идентификатори, при което програмният интерфейс губи типова информация и грешка в идентификатор става невъзможна за откриване от компилатора.

(в) **Разпределение на ролите между четири средства.** Приетият вариант.

2.4.2. Приетият модел

Притежаващият манипулатор е **само преместваем**, наблюдаващият е копируем, а съвместното притежаване се получава само чрез изричен метод. Следствието, заради което решението е взето, е конкретно: най-често срещаният начин да се създаде цикъл — улавяне на притежаващ манипулатор в обработчик, който самата контрола притежава — става *грешка при компилация*, а не безшумен и постоянно задържане на паметта, което при това не се засича от инструментите за откриване на течове.

2.4.3. Емпирична обосновка

Възражението срещу само преместваем манипулатор е ергономично: изисква допълнително действие при достъп до задържана контрола. Възражението е проверено чрез преброяване в кодовата база, а не преценено:

Наблюдение	Брой
места за свързване към събитие, улавящи обхващащия обект	322 от 392
места, улавящи контрола чрез копие на притежаващ манипулатор	0
места с действителен достъп до задържана контрола, върху които пада цената	111–130

Тоест предимството на копируемия манипулатор е предимство над случай, който на практика не се среща, докато цената му — безшумният цикъл — се среща лесно. Преброяването е извършено върху кода на примерните приложения и тестовите в хранилището към момента на вземане на решението и обхваща всички места, на които се свързва обработчик на събитие. То измерва как *се пише* с този интерфейс, не как *би могло* да се пише — което е и точният въпрос при преценка на ергономична цена.

2.4.4. Съпоставка с наличната практика

Съпоставката с наличната практика подкрепя избора от друга посока. Нито един зрял native инструментариум без автоматично събиране на боклука не предоставя на разработчика *копируем притежаващ* манипулатор към контрола — те неизменно въвеждат притежаване от родител към дете и отделен тип за наблюдаваща препратка. Системите, които предлагат копируем манипулатор, са подпрени или от събирач, или от проверка на заемането — и дори те препоръчват наблюдаваща препратка в обработчиците.

Приносът тук не е в откриването на разпределението, а в това, че то е направено *проверимо от компилатора*: същата препоръка, но като грешка при компилация вместо като бележка в документацията.

2.5. Избор на платформени слоеве и на реда им

Реализирани са пет слоя. Редът, в който са изградени, е част от решението:

1. **Слоят без екран** — първи. Той няма native инструментариум и служи като определение за това какво изобщо трябва да реализира един платформен слой. Определящ по K1:

докато той не съществува, нищо от преносимата част не е автоматизирано проверимо.

2. **Настолният слой на разработващата машина** — втори, защото доказва границата между преносимото и платформеното от край до край върху истински контроли.
3. **Мобилният и настолният вариант на същия инструментариум** — трети, преизползват по-голямата част от втория.
4. **Настолният инструментариум на Windows** — четвърти.
5. **Android** — последен, като най-скъпият по K5: границата минава през чужд обектен модел с отделно управление на живота на препратките.

Изборът кой слой да е втори следва от K5: смесеният език дава най-прекия достъп до native обектите — една единица за превод вижда двата свята, без междинно представяне.

2.6. Подход към декларативното описание

Езикът не предоставя рефлексия, а еталонната реализация използва точно нея — за откриване на обработчици, за внедряване на зависимости и за създаване на обекти от описанието. Затова изборът тук не е стил, а принудителен по същество:

1. **Тълкуване по време на изпълнение** — изисква регистър на типовете, който така или иначе трябва да бъде попълнен явно; грешките в описанието се откриват при стартиране.
2. **Пораждане на код при компилация** — грешките се откриват при компилация, няма разбор по време на изпълнение.
3. **Отказ от описание** — само програмен интерфейс.

Избран е вариант (2) — преобразуване по време на компилация — с вариант (1) като резервен път там, където езиковото средство не е налично.

Компромисът е конкретен: печели се откриване на грешките в описанието при компилация и липса на разбор при стартиране; губи се възможността описанието да се изменя без повторно компилиране, което при инструмент за бърза разработка би било съществено, а тук не е.

Явната регистрация е неизбежната цена на липсата на рефлексия и се проявява на три места: регистър на обработчиците (кой договор с кой обект се обслужва), регистрация

на свойствата (кое име към кой описател) и регистър на преобразувателите (как текст става стойност от даден тип). Всяко от тях в оригинала се получава чрез обхождане на типовете при изпълнение.

Тук се отбелязва и капан, установен в хода на работата: при самостоятелна регистрация (статичен обект, който се вписва в регистъра при зареждане) свързващият редактор изхвърля единица за превод, към която няма нито една препратка — и регистраторът никога не се изпълнява. Решението е тези единици да се събират в обектна библиотека, която не подлежи на такова изхвърляне.

2.7. Обобщение на направения избор

Таблица 3. Обобщение на петте решения по критериите K1–K5

Решение	Избран вариант	Решаващ критерий
целеви език	C++23	K5 — пряк достъп до три от петте инструментариума
стратегия	послойно, водено от тестове	K1 — само тя оставя общо описание, тоест контролно условие
собственост	четири роли, само преместваем притежател	K4 — цикълът става грешка при компилация
платформени слоеве	пет, започвайки без екран	K1 — без слоя без екран нищо не е проверимо автоматизирано
декларативен слой	преобразуване при компилация	K1 + K3 — грешките се откриват по-рано, няма разбор при пускане

Две наблюдения върху таблицата. **Най-скъпото решение** е моделът на собствеността: то е единственото, което засяга всеки ред от публичния интерфейс, и единственото, чиято промяна по-късно би означавала пренаписване, а не преработка. **Най-определящото за останалите** е слойът без екран — докато той не съществува, никое от другите решения не може да бъде проверено, а решение, което не може да бъде проверено, на практика не е взето.

Забележимо е и че K1 е решаващият критерий в три от петте реда. Това не е съвпадение: работата изследва твърдение за еквивалентност, и всяко решение, което прави проверката по-трудна, отнема от нея повече, отколкото дава.

III. ПРОЕКТИРАНЕ НА СИСТЕМАТА

Разделът излага проектните решения, върху които стъпва системата — от общата слоева структура, през договорите между слоевете и таблично управляваното преобразуване към платформените контроли, до алгоритмите за разположение и системата от свойства. Всяко решение е представено заедно с алтернативата, спрямо която е взето, и с изискването, на което служи: описанието на интерфейса да остане преносимо, а платформено обусловената част — сведена до най-малкото, което наистина е платформено обусловено.

3.1. Синтез на общата структура

Системата е разделена на девет слоя, подредени по посока на зависимостите. Слой с по-голям номер може да зависи от по-малък; обратното е забранено. Правилото не е стилово — то е това, което прави преносимата част преносима: ако слой за разположение можеше да достигне до платформения, нищо не би попречило платформена частност да проникне в алгоритъм, който трябва да дава един и същ резултат навсякъде.

0. **Графични примитиви** — цвят, точка, размер, правоъгълник, геометрия, платно за рисуване. Няма зависимости.
1. **Договори** — интерфейсите на виртуалните изгледи и на техните обработчици. Зависи само от примитивите.
2. **Обслужващи обекти (handler-и)** — таблиците за преобразуване.
3. **Платформени слоеве** — свързването с native инструментариума.
4. **Разположение** — алгоритмите за измерване и подреждане.
5. **Контроли** — програмният интерфейс за приложението, системата от свойства и свързването на данни.
6. **Декларативен слой** — преобразуване на описание в обектен граф.
7. **Услуги на устройството** — независими от графичния стек.
8. **Инициализация** — изграждане на приложението и регистрация.

Структурата е показана на (Фиг. 2).

Таблица 4. Съответствие директория → слой → обем

Директория	ЕП	Слой
graphics/	27	0 — примитиви
core/	64	1 + 2 — договори и обработчици
layouts/	8	4 — алгоритми за разположение
controls/	159	5 — програмен интерфейс, свойства, свързване
xaml/	35	6 — декларативен слой
essentials/	40	7 — услуги на устройството
hosting/	9	8 — инициализация
animations/	6	5 — анимации
platform/	318	3 — пет платформени слоя

ЕП = единици за превод (.cpp/.mm); публичните заглавни файлове са 716. Данните са към момента на предаване.

Две наблюдения се четат направо от таблицата. Първо, слойт с обработчиците *не* е в отделна директория, а живее заедно с договорите — защото двете се изменят заедно: добавянето на свойство към договор изисква ред в съответната таблица, и разделянето би създадо две места, които трябва да се редактират синхронно. Второ, платформените слоеве са най-обемната част — което е очакваното и е количествен израз на твърдението, че платформената обвързаност е неотстранима, а не подлежаща на абстрахиране.



Фигура 2. Слоеста структура на системата и посока на зависимостите. Само слой 3 е обвързан с конкретен native инструментариум; границата между него и слой 2 е това, което прави останалите слоеве преносими и проверими без екран.

3.2. Проектиране на слоя за графични примитиви

Слоят съдържа стойностните типове, с които всички останали слоеве си говорят: цвят, точка и размер в дробни координати, правоъгълник, радиус на закръгление, матрица на преобразуване, път и четки (плътна, линейна и радиална градиентна, шарка). Той е напълно преносим — няма нито една платформена зависимост — и по тази причина изграждането започва от него: всеки следващ слой се нуждае от неговите типове, а той не се нуждае от нищо.

Абстракцията за рисуване (canvas) е интерфейс за поредица от команди — линия, път, запълване, текст — а не за конкретна повърхност. Наред с платформените й реализации съществува и *записваща*: тя не рисува, а запазва получените команди. Ролята ѝ е за проверимост — позволява да се провери *какво* е поискано да бъде нарисувано, без екран и без сравняване на изображения.

Пример за изчисление, което тук се извършва по време на компилация, а в управлението оригинал задължително при изпълнение: превръщането на цвят от текстов запис в стойност. Записът е известен при компилация, разборът е `constexpr`, и в изходния артефакт попада готовата стойност — без разбор при пускане и без възможност за

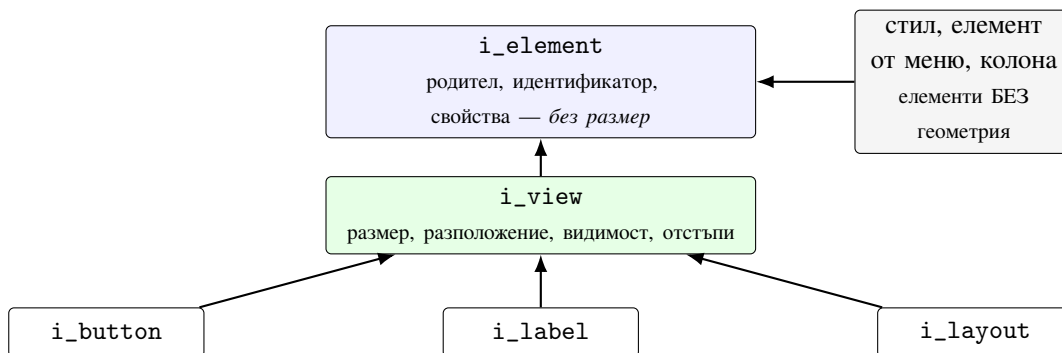
невалиден запис да оцелее до изпълнение.

3.3. Договори на виртуалните изгледи

Виртуален изглед е обект със свойства и без рисуване. Йерархията върви от `i_element` (най-общият елемент от дървото) през `i_view` (елемент с размер, разположение и видимост) към специализираните договори на отделните контроли. Договорът описва *какво* има контролата, но никъде не описва *как* изглежда — това е разделението, което позволява един и същ договор да бъде удовлетворен от разнородни `native` контроли.

Йерархията има три нива. `i_element` е най-общият: всичко в дървото е елемент — носи родител, идентификатор и участие в системата от свойства, но *не* носи размер. `i_view` добавя това, което има геометрия: желан размер, разположение, видимост, прозрачност, отстъпи. Специализираните договори (`i_button`, `i_label`, `i_layout`, ...) добавят само своето.

Разделянето на „елемент“ от „изглед“ не е излишно: не всеки елемент от дървото има размер. Дефиниция на стил, елемент от менюто, колона в решетка — всички те участват в дървото и в системата от свойства, но не се измерват и не се подреждат. Ако размерът беше на най-общото ниво, всеки от тях щеше да носи шест неизползвани свойства и — което е по-лошо — щеше да бъде възможно да му се зададе размер.



Фигура 3. Йерархия на договорите. Геометрията живее на `i_view`, не на `i_element` — елементите вдясно участват в дървото и в системата от свойства, но нямат размер и не бива да могат да получат такъв.

3.4. Проектиране на слоя за преобразуване

Това е повтарящата се единица на цялата система. За всяка контрола съществува обработчик, който свързва виртуалния изглед с `native` контролата и пренася стойностите между тях.

3.4.1. Таблица „свойство → действие“

Преобразуването е *таблично*, а не процедурно: обработчикът притежава карта, съпоставяща името на всяко свойство на функция, която прилага текущата му стойност върху native контролата, и втора карта за императивните операции. Смяна на стойност предизвиква извикване на съответното действие; свързване на нов изглед предизвиква пълно преминаване през таблицата.

Картите се *верижат*: картата на контролата пада назад към картата на изгледа, а тя — към картата на елемента. Проектното решение, което заслужава внимание, е как се разрешава припокриване: ключ, дефиниран в по-близка карта, надделява по *съдържание*, но се изпълнява на *позицията*, която ключът заема в най-далечната карта, където се среща. Така редът на прилагане остава стабилен — действие, което трябва да се изпълни първо (например установяване на контейнера), не може да се премести назад само защото по-близък слой е предефинирал друго свойство.

Числов пример. Обслужващият обект на бутон има собствена таблица с четири ключа (`text`, `text_color`, `font`, `character_spacing`) плюс втора за изображението (`source`, `content_layout`); те се верижат към общата таблица на изгледа, която съдържа `background`, `visibility`, `opacity` и останалите общи свойства. Ако бутонът предефинира `background`, при пълно преминаване се изпълнява *неговата* реализация, но на позицията, която `background` заема в таблицата на изгледа — тоест преди `text`, а не след него. Обратното би означавало, че установяването на фона се измества след изписването на текста, което на някои инструментариуми води до презаписване.

Съзнателни опростявания спрямо оригинала: няма кеш на слетите таблици (в оригинала той е оптимизация за бързодействие — списъкът с ключове се преизчислява при всяко преминаване) и няма ключ за пакетно прилагане (в оригинала той е точка за платформено групиране на промени, каквато тук няма). Нито едно от двете не променя *кои* действия се изпълняват и в *какъв ред* — само колко работа се повтаря.

3.4.2. Обобщен базов клас без виртуални шаблони

Базата на обработчиците е шаблон, параметризиран с производния тип, типа на виртуалния изглед и типа на native контролата. Изборът на този похват — вместо виртуален базов клас с шаблонни методи — следва от езиковото ограничение, че шаблонен метод не може да бъде виртуален: обобщената част се разрешава по време на компилация,

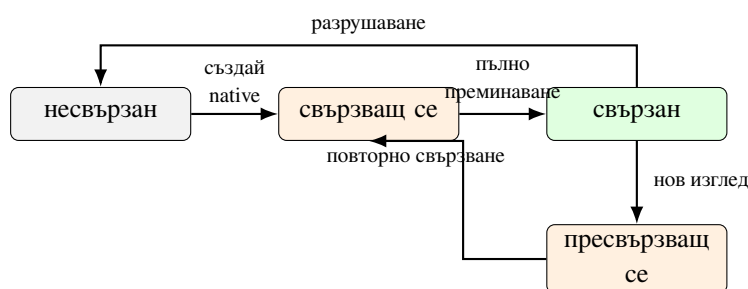
а полиморфизмът се получава през нешаблонен интерфейс, който базата реализира. Незадължителните точки за разширение се откриват по време на компилация чрез изискване за наличие на израз, така че производният клас изброява само онова, което наистина предоставя, и няма подразбиращи се реализации, които да бъдат засенчени по невнимание.

Интерфейсът на обработчика се наследява *виртуално* [16]. Причината е конкретна: обработчик на контейнер трябва да удовлетворява едновременно общия интерфейс и интерфейса за контейнери; без виртуално наследяване той би получил два отделни подобекта на общия интерфейс.

3.4.3. Жизнен цикъл и собственост

Жизненият цикъл има четири състояния: *несвързан*, *свързващ се*, *свързан* и *пресвързващ се*. Последователността при първо свързване е: създаване на native контролата → преминаване в „свързващ се“ → еднократно свързване (абонаменти, вписване в native дървото) → пълно преминаване през таблицата → „свързан“. Повторно подаване на същия изглед е операция без действие.

Състоянието „свързващ се“ е нарочно *видимо* за платформените действия. То им позволява да пропуснат работа по свойства, чиято стойност и без това е подразбиращата се — при първоначалното преминаване това е повечето от тях. Без този признак всяко свързване би изпълнило пълния набор действия, включително онези, които не променят нищо.



Фигура 4. Състояния на обработчика. „Свързващ се“ е видимо за платформените действия, за да могат да пропуснат работа по свойства със стойност по подразбиране — при първото преминаване това е повечето от тях.

Собствеността е насочена и еднозначна: изгледът притежава обработчика, а обработчикът притежава native контролата. Обратните връзки — от обработчика към виртуалния изглед и от детето към родителя — са непритежаващи. Това е прякото

следствие от липсата на автоматично събиране на боклука и е разгледано като самостоятелно решение в §3.7.

Тук се отбелязва капан, установен в хода на работата и струвал значително време: свързването на контрола с обработчик изисква вписване в *два* отделни списъка — регистъра, който съпоставя типовете, и списъка, който се изгражда при инициализация. Контрола, вписана само в единия, се компилира без предупреждение и се стартира без грешка, но не се изобразява — страницата излиза празна. Признакът е показателен за цялата система: липсващата регистрация не може да бъде уловена от компилатора, защото е данна, а не тип.

3.5. Проектиране на алгоритмите за разположение

Разположението се изчислява в преносимия слой, а не се възлага на native инструментариума. Това е едно от определящите решения на цялата работа: платформената контрола се използва като правоъгълник с даден размер и дадено оформление, а не като участник в собствената си подредба. Цената е, че алгоритмите трябва да бъдат възпроизведени точно; ползата е, че разположението е едно и също навсякъде и може да бъде проверено без екран.

3.5.1. Двухазният модел

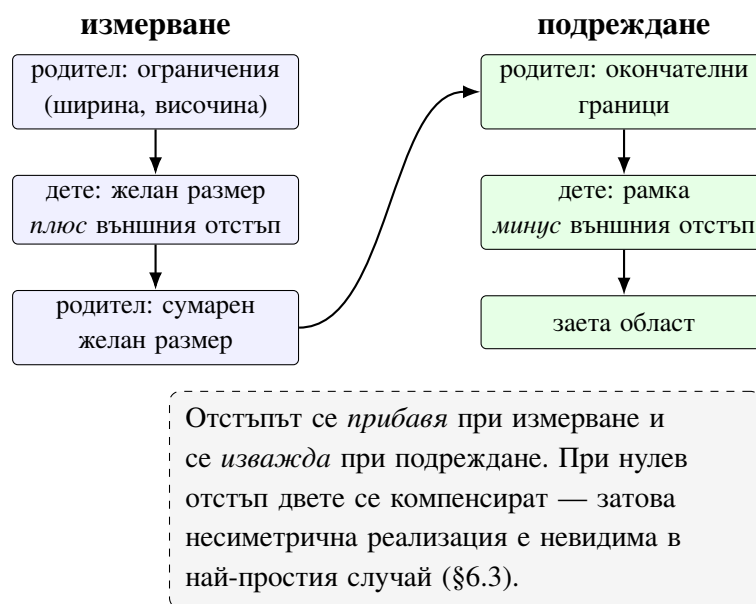
Интерфейсът на един алгоритъм се състои от две операции: *измерване*, което по дадени ограничения за ширина и височина връща желаня размер, и *подреждане*, което по дадени граници разполага децата и връща действително заетия размер. Разделянето на две фази е необходимо, защото размерът на родителя може да зависи от децата, а положението на децата — от окончателния размер на родителя.

Двете фази и симетрията между тях са показани на (Фиг. 5).

Ограничението „неограничено“ се предава като безкрайност и означава „вземи толкова, колкото ти трябва“ — използва се от подвижните области, които не ограничават децата си по посоката на превъртане. Разликата между него и голяма крайна стойност е съществена: дете, което разпределя оставащото място пропорционално, при безкрайност няма какво да разпределя и трябва да се сведе до собствения си желан размер.

През вложени контейнери двете фази се разпространяват рекурсивно: измерването слиза надолу с ограничения и се връща с размери, подреждането слиза надолу с

граници. Дълбочината на дървото умножава цената, което прави тази двойка проходи най-натоварената част от преносимия слой — и мястото, където се очаква измерима разлика по хипотеза H2b-2.



Фигура 5. Двufазният модел на разположението и симетрията на външния отстъп между двете фази.

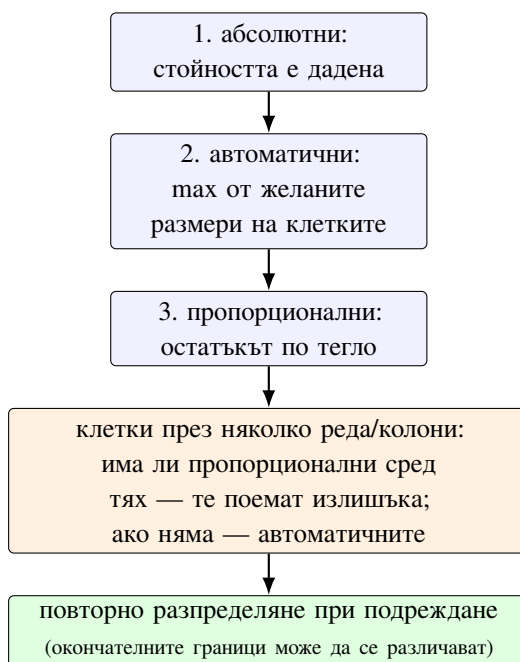
3.5.2. Разпределяне на пропорционалните измерения

Разпределянето на измеренията в решетка е най-сложният алгоритъм в системата — с порядък по-обемин от всеки друг мениджър на разположение. Трите вида измерение се разрешават в строг ред:

1. **Абсолютно** — стойността е дадена; разрешава се веднага.
2. **Автоматично** — размерът е максимумът от желаните размери на клетките в реда/колоната; изисква измерване на децата и затова следва абсолютните.
3. **Пропорционално** — оставащото място след първите две се разпределя между тях по тегло.

Усложнението идва от клетките, разпростиращи се върху няколко реда или колони. Такава клетка не може да бъде отнесена към едно измерение; желаният ѝ размер трябва да бъде разпределен между обхванатите, при това по различно правило според това дали сред тях има пропорционални — ако има, те поемат излишъка и автоматичните остават непроменени; ако няма, излишъкът се разпределя между автоматичните.

Второ усложнение: разпределянето се извършва *отново* във фазата на подреждане, защото окончателните граници може да се различават от онези, при които е извършено измерването.



Фигура 6. Разрешаване на измеренията в решетка. Редът е задължителен: пропорционалните могат да бъдат разрешени едва след като е известно какво остава от абсолютните и автоматичните.

3.5.3. Координатна система при подреждане

Проектното решение — в какви координати се подават границите при подреждане, абсолютни или относителни спрямо контейнера — изглежда като подробност и не е такава. При вложен контейнер с ненулево начало погрешният избор води до двукратно прибавяне на отместването и до дефект, който се проявява само при влагане, тоест не при най-простите проверки. Случаят е записан като клас в систематизацията в §6.3.

3.6. Проектиране на системата от свойства

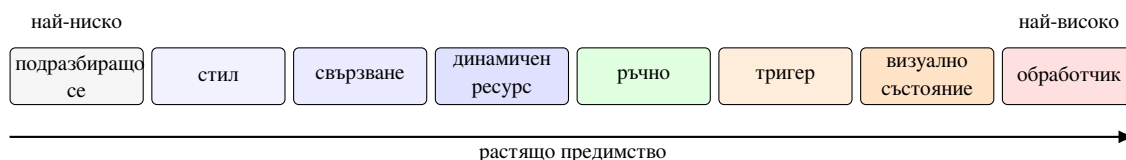
Системата от свойства съхранява стойностите, известява за промяна — прилагайки шаблона *наблюдател* [19] — и разрешава предимството между едновременно валидни източници на една и съща стойност.

3.6.1. Типизиран описател вместо изтрит тип

Описателят на едно свойство — име, стойност по подразбиране, обратни извиквания — е *шаблон по типа на стойността*, за разлика от оригинала, където той е изграден върху общия базов тип на средата за изпълнение. Следствията са преки: стойността по подразбиране, сравнението за равенство и обратните извиквания са типизирани, не се извършва опаковане на стойностни типове и не се разчита на информация за типа по време на изпълнение.

3.6.2. Предимство на стойностите като едно цяло число

Предимството е представено като опакована в едно 64-битово число ключова стойност, чийто *целочислен ред съвпада с реда на предимството*. Възходящо:



Фигура 7. Нива на предимство при разрешаване на стойност. Подредбата им е целочислена: сравнението на две нива е едно сравнение на числа. Стойността от обработчика стои най-високо, но се измества от всяко изрично присвояване — тя отразява какво е направила native контролата, а не какво е поискало приложението.

Решението има три следствия. Сравнението на две предимства е едно целочислено сравнение. Стойността е изчислима по време на компилация, така че наименуваните константи се инициализират статично и не са изложени на неопределения ред на инициализация между единиците за превод [20]. И трето — стойността, зададена от обработчика, се съхранява на най-високото ниво, но се измества от всяка друга, което възпроизвежда поведението на оригинала.

3.6.3. „Незададено“ срещу „зададено на стойността по подразбиране“

Разграничението изглежда академично и има пряк визуален ефект: контрола, чийто цвят на текста не е задаван, трябва да получи цвета по подразбиране на платформата, докато контрола, на която е зададен изрично цвят, съвпадащ с подразбиращия се, трябва да получи точно него. Реализация, която проверява стойността вместо наличието на стойност, обърква двата случая — и това е точно грешката, която прави текст невидим при тъмна

тема. Класът е описан в §6.3.

3.7. Проектиране на потребителския програмен интерфейс

Липсата на автоматично събиране на боклука е най-същественото разминаване спрямо средата на оригинала, защото дървото от изгледи по природа съдържа обратни връзки родител ↔ дете и абонаменти за събития. Приетият модел разпределя работата между четири сътрудничащи роли:

Роля	Средство	Семантика
конструиране	изграждащи функции	връщат притежаващ манипулатор върху поддървото
притежаване	<code>view_ref<T></code>	само преместваем; точно един притежател
съхраняване	<code>weak_ref<T></code>	копируем, непритежаващ; безопасен достъп
изменение	<code>observable<T></code>	реактивно; изменя се <i>данните</i> , не изгледът

Последната роля — изменя се *данните*, а изгледът следва — е модела на представянето (presentation model), основата на шаблона MVVM [21].

Решението притежаващият манипулатор да *не* е копируем прави невъзможен точно един клас грешки — и това е най-често допусканата: улавяне на притежаващ манипулатор в обработчик, който самата контрола притежава. Такъв цикъл не се съобщава от нито един инструмент, защото обектите са достижими; те просто са достижими само един от друг. При копируем манипулатор това е валиден код; тук е грешка при компилация.

Цената в ергономичност е реална, но ограничена: достъпът до задържана контрола изисква преминаване през наблюдаваща препратка и проверка за живост. Емпиричната ѝ величина е измерена в §2.4 — тя пада върху малцинството места, които изобщо достигат до задържана контрола.

За случаите, в които съвместното притежаване е наистина необходимо (два елемента, които взаимно се изменят и имат независим живот), съществува изричен метод, който създава втори притежател. Съществено е, че той е *изричен и откриваем* — членът от тип наблюдаваща препратка обявява „наблюдател“, а членът от тип притежаващ манипулатор обявява „притежател“, така че намерението се чете от типа.

3.8. Формат на декларативното описание

Входът е дървовидно описание на елементите с атрибути. Преобразуването му минава през три отделни стъпки, съзнателно разделени: *разбор* (текст → дърво от възли), *изграждане* (възли → обекти, чрез регистър на типовете) и *свързване* (изразите за свързване се превръщат във връзки към данни).

Разделянето позволява първите две да се извършат по време на компилация, а само третата — при изпълнение, защото тя зависи от данните. Регистърът на типовете и регистърът на преобразувателите (текст → стойност) са явни и заместват рефлексията: всеки тип, който може да се появи в описанието, е вписан изрично.

3.9. Проектиране на платформените слоеве

Всеки платформен слой реализира едни и същи обработчици върху различен native инструментариум. Реализирани са пет: два върху UIKit (мобилният и настолният му вариант), един върху native настолния инструментариум на същата операционна система, един върху Android View системата, един върху съвременния настолен инструментариум на Windows, и един *без екран*.

Слоят без екран не е спомагателен детайл. Той е условието, при което поведението на преносимата част — разположение, предимство на стойностите, известяване — може да бъде проверено автоматизирано, без устройство и без изобразяване. Без него проверимостта на цялата система би зависела от заснемане на изображения.

Двата слоя върху една и съща операционна система споделят всичко, което е обвързано с езика и с общите графични услуги: преобразуването на цветове и шрифтове, рисуването върху платно, работата с дати. Не може да бъде общо това, което зависи от *йерархията на контролите*: двете системи имат различни базови класове, различни правила за влагане и различни жизнени цикли на прозореца.

Слоят върху native настолния инструментариум е принципно несравним попикселно с останалите, защото изгражда интерфейса от друго семейство контроли, отколкото еталонът използва на същата операционна система. Затова изискването към него е формулирано различно — пълнота на изобразеното съдържание и съгласуваност между двата входни пътя — и той не участва в количествените таблици (§5.1). Това не е отстъпка, а признание, че число без смисъл е по-лошо, отколкото изобщо да липсва число.

IV. ПРОГРАМНА РЕАЛИЗАЦИЯ

Разделът описва как проектните решения от Раздел III са изразени в код: организацията на изходния текст, конфигурациите за компилация, реализацията на всеки слой и — най-същественото за верността на резултата — дисциплината на разработка, която не позволява поведение да бъде измислено.

4.1. Обща структура на изходния код

Изходният текст е разделен на публични заглавни файлове и реализация. Директориите следват слоевете от §3.1: графични примитиви, ядро (договори и обработчици), разположение, контроли, декларативен слой, услуги на устройството, инициализация, анимации и платформени слоеве.

Едно отклонение от очакваното заслужава обяснение: обработчиците живеят заедно с договорите в общата директория на ядрото, а не в собствена. Причината е, че двете се изменят заедно — добавянето на свойство към договор изисква ред в таблицата на съответния обработчик — и разделянето им би създавало две места, които трябва да се редактират синхронно, без нищо да го налага.

Съответствието директория–слой–обем е дадено в (Табл. 4) (§3.1). Имената на пространствата съответстват на тези в оригинала с преобразуване на стила, а всеки публичен заглавен файл носи в коментар пълното име на типа, от който произхожда — така изходният текст остава съпоставим при проверка, без да се налага търсене по подобие.

4.2. Организация на компилацията

Компилацията е организирана като набор от независими конфигурации, всяка в собствено дърво: слой без екран, конфигурация със статичен анализ, три платформи със санитаризери за поведение, за нишки и за неинициализирана памет, и по една платформа за всеки платформен слой.

Разделянето поражда практическия проблем, че пълната проверка е всъщност N независими компилации. Организацията му е част от реализацията, а не околност: конфигурацията без екран се изпълнява първа, за да напълни кеша на компилатора, която платформата за статичен анализ след това преизползва — при еднакви флагове тя плаща

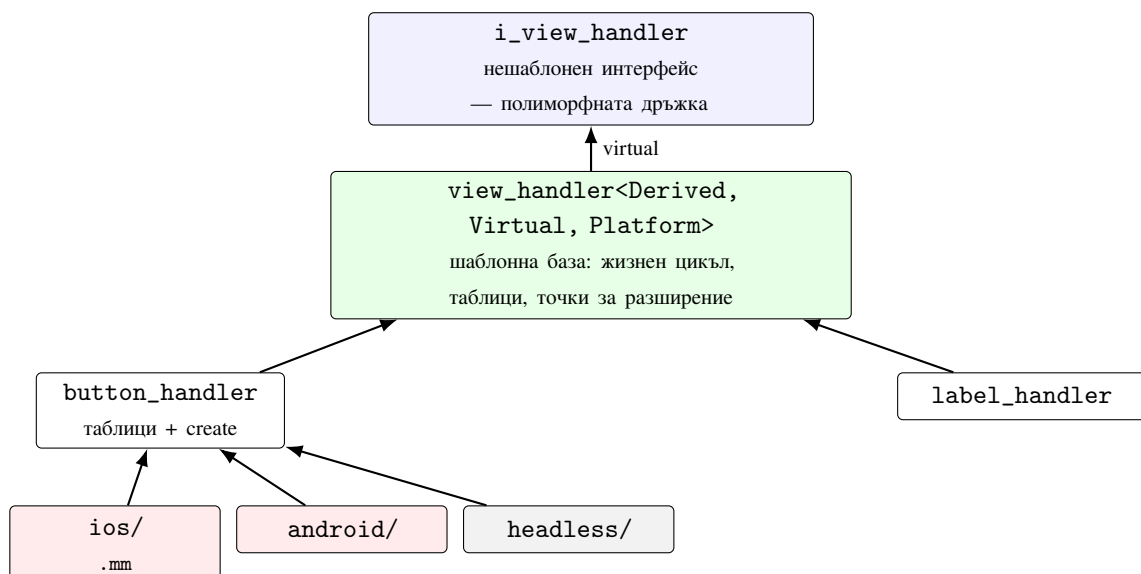
само за самия анализ. Компилациите са инкрементални по подразбиране; изчистването е изричен избор, а не предпазна мярка.

Режимите на работа са два и смесването им е обичайната загуба на време. **Бързият вътрешен цикъл** компилира една конфигурация инкрементално и изпълнява само тестовете, чиито имена съвпадат с даден израз — секунди. **Пълната проверка** минава през всички конфигурации, включително санитайзерите и статичния анализ — десетки минути.

Инструментите са нарочно два отделни, а не един с ключ, защото изборът влияе върху навика: инструмент, който по подразбиране прави пълната проверка, се използва рядко, а инструмент, който по подразбиране прави бързата, се използва след всяка промяна. Пълната се пази за проверка преди предаване.

4.3. Реализация на слоя за преобразуване

4.3.1. Йерархия на класовете



Фигура 8. Йерархия на обработчиците. Наследяването на интерфейса е виртуално, за да може обработчик на контейнер да удовлетвори едновременно него и интерфейса за контейнери без два подобекта. Платформените реализации са отделни единици за превод, а не класове — добавянето на платформа е добавяне на файлове.

Разделянето на частичен клас с платформени файлове, което оригиналът използва, е изразено чрез отделни единици за превод: обща реализация плюс по една реализация на платформен слой, избирана при конфигуриране. Така добавянето на платформа е *добавяне*

на файлове, а не изменение на съществуващи — което е и проверката дали границата между преносимото и платформеното е на правилното място.

4.3.2. Коментар на изходния текст: един представителен срез

Срезът за бутон е представителен — всяка друга контрола е със същата форма — затова се разглежда подробно веднъж:

```
1 // Текст и изглед на текста, ключирани по i_text_button.
2 property_mapper<i_text_button, button_handler>& button_handler::text_mapper()
3 {
4     static property_mapper<i_text_button, button_handler> table{
5         {"text",          &button_handler::map_text},
6         {"text_color",    &button_handler::map_text_color},
7         {"font",          &button_handler::map_font},
8         {"character_spacing", &button_handler::map_character_spacing},
9     };
10    return table;
11 }
```

Листинг 1: Таблицата за свойства на обработчика за бутон

Четири неща в тези редове носят проектните решения от Раздел III. Таблицата е `static` — изгражда се веднъж за целия процес, не за всеки бутон. Ключовете са низове, защото са същите имена, които системата от свойства известява при промяна. Стойностите са указатели към функции-членове, а не към виртуални методи — разрешаването е по време на компилация. И типът е шаблон по *договора* (`i_text_button`), а не по контролата, така че една и съща таблица обслужва всеки тип, който удовлетворява договора.

Платформената страна допълва картината:

```
1 std::unique_ptr<button_platform> button_handler::create_platform_view()
2 {
3     auto platform = std::make_unique<button_platform>();
4     UIButton* const button = [UIButton buttonWithType:UIButtonTypeSystem];
5     set_control_properties_from_proxy(button);
6     platform->native = (__bridge_retained void*)button; // слотът поема една
    ↪ препратка
7     return platform;
8 }
```

Листинг 2: Създаване на native контролата (iOS)

Тук се вижда границата между двата модела на живот: `__bridge_retained` прехвърля една препратка от управлението на средата към C++ слота, а разрушителят

я връща обратно. Смесеният език позволява това да се случи в *една* функция — без междинно представяне и без слой за маршалиране.

4.4. Реализация на алгоритмите за разположение

Разпределянето на пропорционалните измерения е с порядък по-обемно от всеки друг мениджър на разположение — само по себе си довод за решението алгоритмите да се пренасят водени от тестове, а не да се пишат по описание: алгоритъм с толкова разклонения има твърде много гранични случаи, за да бъдат отгатнати.

```
1 double star_count = 0.0;
2 for (const definition& current : definitions)
3 {
4     if (current.is_star())
5     {
6         star_count += current.length().value();
7     }
8 }
9 return star_count;
```

Листинг 3: Сумиране на теглата на пропорционалните измерения

Функцията изглежда тривиална и точно затова е показателна: сумата се смята върху *всички* определения, а не само върху незаетите, защото клетка, разпростряна върху няколко колони, вече е внесла своя дял. Разделянето на остатъка после става на остатък / `star_count` на единица тегло. Грешка тук — например изключване на колона, чиято ширина е вече разрешена — не се проявява при една пропорционална колона и се проявява при две с различни тегла, което е и причината този алгоритъм да се пренася воден от тестове.

4.5. Реализация на системата от свойства

Стойностите се съхраняват не като една стойност, а като *подредена по предимство* редица: всеки източник (подразбиращ се, стил, свързване, ръчно присвояване, тригер, визуално състояние, обработчик) заема свое ниво, а действащата стойност е тази с най-високото заето ниво. Изчистването на един източник не изтрива стойността, а разкрива следващата отдолу — което е точното поведение, което оригиналът има.

Стойността, зададена от обработчика, се съхранява на най-високото ниво, но се измества от всяко друго присвояване. Причината е, че тя не е избор на приложението,

а отражение на това, което native контролата е направила сама — и всяко изрично присвояване трябва да я надделее.

Разрешаването на пътя при свързване се извършва *без рефлексия*: всяко свързваемо свойство се вписва в регистър при своята регистрация, а изразът за свързване се разрешава срещу този регистър.

4.6. Реализация на платформените слоеве

Петте платформени слоя се различават не само по инструментариум, но и по езиковата граница, през която минават.

4.6.1. Слоевете върху Apple (macOS и iOS)

Слоевете върху Apple се пишат на смесен език (.mm), при което една единица за превод вижда едновременно C++ и native обектния модел — native обектът се държи пряко в C++ клас, вместо да се пренася през граница (листинг 11). Управлението на живота се съгласува изрично при преминаване на слота и обратно при разрушаване.

Защо това не превръща целия проект в смесен език. Естественото притеснение е, че щом част от файловете се компилират като Objective-C++, това ще плъзне нагоре: заглавните файлове ще започнат да носят native типове и всеки консуматор ще трябва да се компилира по същия начин. Не се случва, и причината е една — *native типът никъде не се появява в заглавен файл*. Платформената структура държи native обекта зад непрозрачен указател — идиомът `rimpl`, компилационна преграда, отделяща интерфейса от реализацията [22]:

```
1 struct button_platform : view_platform_base
2 {
3     void* native = nullptr;    // UIButton* / NSButton* --- типът е известен само на .mm
4 };
```

Листинг 4: Native обектът зад непрозрачен слот

Заглавният файл вижда `void*` и нищо повече. Само реализацията — която *вече* е .mm — знае какъв е действителният тип и извършва преобразуването. Следствията са три: кросплатформените заглавни файлове се компилират от обикновен C++ компилатор; тестовите без екран не се нуждаят от Apple инструментариум; и смяна на native типа не предизвиква повторна компилация извън собствената му единица за превод.

Как изграждането отделя платформената част. Разделянето не е чрез условна компилация вътре във файловете, а чрез *подбор на файлове*. Изграждането има една променлива — избраният платформен слой — и според нея включва точно една директория:

Стойност	Директория	Език на единиците
headless	platform/headless/	обикновен C++
apple	platform/apple/	Objective-C++ (AppKit)
ios	platform/ios/	Objective-C++ (UIKit)
maccatalyst	platform/ios/	същите файлове, друга целева платформа
windows	platform/windows/	C++ с проекция на компонентния модел
android	platform/android/	C++ през интерфейса за извикване

Два реда заслужават внимание. `maccatalyst` не е отделен слой — изграждането пренасочва към *същите* файлове на UIKit и само сменя целевата платформа; това е и причината настолната Apple платформа да получи два различни слоя без двойно писане (§2.5). А `headless` е обикновен C++ и не изисква нито един платформен инструментариум — затова проверката на преносимата логика се изпълнява навсякъде, включително там, където Apple компилатор няма.

Онова, което наистина пази разделението, е правилото от §3.1: зависимостите сочат само надолу. Ако кросплатформен заглавен файл можеше да достигне до платформен тип, нито подборът на файлове, нито непрозрачният указател биха помогнали — разделението щеше да бъде обходено още при включването.

4.6.2. Слойт върху Android

Слойт върху Android минава през интерфейса за извикване към управляваната платформа. Той е най-скъпият по K5 от §2.1 по три причини: всяко обръщение изисква търсене на метод по подпис; препратките към обекти имат два вида живот — локален (валиден до връщане от функцията) и глобален (изрично освобождаван), и смесването им е източник на дефекти, които се проявяват късно; и всяка нишка, която обръща към платформата, трябва да бъде изрично присъединена и отделена.

Жизненият цикъл на дейността се съгласува с този на приложението чрез изрично препредаване на събитията — платформата може да разруши и пресъздаде дейността, без приложението да е приключило.

4.6.3. Слойт върху Windows

Слойт върху Windows минава през проекция на компонентния модел в заглавни файлове. Основното ограничение, което инструментариумът налага, е върху *свободното разположение*: контейнерът, който позволява произволни координати, се държи различно от този, който подрежда сам, и системата трябва да избере правилния според това какво изисква описанието.

Отбелязват се и три езикови капана, установени в хода на работата, защото всеки от тях струва часове и нито един не е документиран като такъв: имена от системните заглавни файлове, които съвпадат с имена от системата (разрешава се с преименуване или с премахване на макрос); съкратено име на пространство, което влиза в конфликт с име на тип; и невъзможността да се преопредели заделянето на памет за проектираните типове.

Обхватът на реализираните контроли на тази платформа е *частичен по замисъл* към момента на предаване: реализирани са прозорецът, страницата, контейнерите за разположение, надписът и бутонът, а останалите контроли все още използват огледалото без екран и не изобразяват нищо. Това се отбелязва изрично тук, защото иначе би се подразбрало от таблиците — интерфейс, изграден само от нереализирани контроли, е празен и в двете колони, което е незавършена реализация, а не съвпадение.

4.6.4. Слойт без екран

Слойт без екран реализира същите обработчици, но „native контролата“ е обикновена структура, която само записва получените стойности. Това го прави пълноценен обект на автоматизирана проверка: всичко, което не е рисуване — разположение, предимство на стойностите, известяване, жизнен цикъл — е проверимо върху него, без устройство и без сравняване на изображения.

Той е и *определението* за това какво трябва да реализира един платформен слой: новият слой се сверява срещу него, а не срещу някой от вече съществуващите, които носят собствените си особености.

4.7. Реализация на декларативния слой

4.7.1. Входът: едно описание, два потребителя

Описанието е обикновен XML документ. Съществено е, че *същите байтове* се четат и от еталонното приложение, и от системата — това е условието, което прави сравнението в Раздел V валидно.

```
1 <ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
2           xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
3           x:Class="MauiReference.Pages.BoxViewPage" Title="BoxView">
4     <ScrollView>
5       <VerticalStackLayout Spacing="6" Padding="12">
6         <Label Text="Default" />
7         <BoxView BackgroundColor="CornflowerBlue"
8           WidthRequest="160" HeightRequest="160" />
9       </VerticalStackLayout>
10    </ScrollView>
11 </ContentPage>
```

Листинг 5: Откъс от описание на интерфейс

4.7.2. Конвейерът

Описанието влиза в единицата за превод чрез `#embed` — директива на C++23, която вгражда съдържанието на файл като масив от байтове по време на компилация. Оттам нататък всичко е обикновена `constexpr` обработка:

```
1 namespace
2 {
3     // Суровият markup, вграден от компилатора (пътят е спрямо този файл).
4     constexpr unsigned char data_binding_xaml_bytes[] = {
5         #embed "data_binding.xaml"
6     };
7     constexpr maui::fixed_string data_binding_xaml{data_binding_xaml_bytes};
8 }
9
10 std::unique_ptr<maui::controls::content_page> data_binding_page(
11     std::unique_ptr<ViewModels::greeting_view_model> vm)
12 {
13     auto page = maui::build_page<ViewModels::greeting_view_model, data_binding_xaml>();
14     page->bind_to(std::move(vm)); // BindingContext -> {Binding} изразите оживяват
15     return page;
16 }
```

Листинг 6: Вграждане и изграждане на страница по време на компилация

Ключът е сигнатурата `build_page<ViewModel, fixed_string>()`: описанието е *шаблонен параметър*, не аргумент. Това го прави достъпно за компилатора, който разбира документа и изгражда съответните извиквания на конструктори — без разбор

при стартиране и без нито един ред кодогенерация извън езика: няма отделна стъпка в изграждането и няма породени файлове в дървото.

Трите стъпки от §3.8 се разпределят така:

Стъпка	Кога	Какво изисква
разбор	при компилация	<code>constexpr</code> обработка на вградените байтове
изграждане на графа	при компилация	регистър на типовете (замества рефлексията)
свързване на данни	при изпълнение	зависи от данните, не от описанието

Печалбата е в момента, в който грешката се открива: непознат елемент или сгрешено име на свойство спира *компилацията*, вместо да предизвика отказ при стартиране.

4.7.3. Границата на средството

`#embed` не е налична във всички компилатори, които проектът поддържа. Там, където липсва, същият вход минава през отделна стъпка в изграждането, която го превръща в масив от байтове — еднакъв резултат, една допълнителна зависимост за тази платформа. Цената на целия декларативен път е измерена в §6.4 и се оказва *код*, а не данни.

4.8. Обработка на събития от интерфейса

Дотук стойностите текат в една посока — от приложението към *native* контролата. Събитията текат в обратната и изискват собствен механизъм, защото *native* страната е тази, която ги поражда.

4.8.1. Механизмът от страната на C++

Няма делегати и няма събиране на боклука, затова събитието е самостоятелен тип — пряка реализация на шаблона *наблюдател* [19]: `event<Args...>` с `connect` (връща *жетон*), `disconnect` и `raise`. Жетонът е съществен — той позволява абонаментът да бъде прекратен *детерминистично*, а не при следващо събиране. Има и обвивка, която прекратява абонамента в разрушителя си (RAII [16]), така че животът на връзката съвпада с обхвата, в който е създадена.

4.8.2. Пътят от native контролата до приложението

Между двата обектни модела стои малък посредник. На Apple платформите това е обект от native типовата система, който няма друга задача освен да препрати:

```
1 // Obj-C trampoline: препраща target-action на UIButton към виртуалния изглед.  
2 @interface MauiButtonEventProxy : NSObject  
3 @property(n nonatomic) maui::core::button_handler* handler;  
4 - (void)onTouchUpInside:(id)sender;  
5 @end
```

Листинг 7: Посредник, който препраща native действието към виртуалния изглед

Веригата е: native действие → посредник → обработчик → `send_clicked()` на виртуалния изглед → контролата издига собственото си събитие → обработчиците на приложението.

Два детайла не са произволни. Първо, *редът* на събитията се пренася точно: при отпускане на пръста върху бутона първо се издига „отпуснат“ и едва след това „натиснат“ — редът на еталона, който приложение може да наблюдава. Второ, посредникът държи *непритежаваща* препратка към обработчика; притежаваща би затворила цикъл (обработчикът притежава native контролата, която държи посредника) — точно класът цикли, който моделът на собственост от §2.4 прави невъзможен по-нагоре в стека.

4.8.3. Именувани събития за декларативния слой

Описанието може да свърже действие по *име* (например атрибут `Clicked`). Тъй като рефлексия няма, всяка контрола вписва събитията си в регистър при изграждането си — съответствие „име → функция, която абонира“. Декларативният слой търси там, намира функцията и я извиква. Същата размяна както при свойствата: явен запис вместо откриване по време на изпълнение.

4.9. Дисциплина на разработка и осигуряване на верността

Този параграф описва процеса, а не кода — и е поставен в раздела за реализация съзнателно, защото при работа с външен еталон именно процесът е това, което отличава пренасяне на поведение от повторното му измисляне.

4.9.1. Цикълът „по една компонента“

За всяка компонента редът е фиксиран: прочитане на договора и на *тестовете* на оригинала; пренасяне на тестовете *преди* реализацията, така че те да се компилират и да се провалят; реализация до преминаването им; и чак тогава преминаване към следващата — дисциплината на разработката, водена от тестове [23]. Правилото, което този ред налага, е формулирано отрицателно и е по-важно от самия ред: *когато поведението не е известно, то се прочита от оригинала — не се предполага*. Правдоподобно предположение, което се разминава с действителното поведение, е дефект, а не приблизително решение.

Трите нива на истината и въпросите, на които отговарят, са изложени в §2.3: описанието на публичната повърхност дава *формата*, тестовете дават *поведението*, изходният текст дава *реализацията*.

Разминаването между тях е реално и е било разрешавано. Пример: изходният текст на еталона поставя долна граница на размера на една контрола; разпространената версия изобразява по-малък размер без никаква граница. Формата (публичната повърхност) не казва нищо по въпроса, тестовете също. Приложеното правило — предимство има изобразеното — е записано предварително (§5.3) и промяната е отбелязана в кода като документирано отклонение с цитиране на двата източника. Без предварително записано правило това решение би било произволно.

4.9.2. Статичен анализ и санитайзери като част от цикъла

Статичният анализ се изпълнява като отделна конфигурация с изискване за *нула* находки, и потискането на находка вместо отстраняването ѝ не е допустимо. Санитайзерите — за неопределено поведение, за състезания между нишки [24] и за неинициализирана памет [25] (семејство динамични анализатори, чийто прародител е проверката на достъпа до памет [26]) — са три отделни конфигурации, защото не могат да се съчетаят в една.

Всичко това е част от цикъла, а не заключителна проверка, по проста сметка: находка, открита седмици след внасянето си, изисква възстановяване на контекста, който вече не е в главата на никого, докато същата находка в цикъла, в който е внесена, се отстранява веднага. Разликата е в порядъци, не в проценти.

4.9.3. Обхват на автоматизираната проверка

Таблица 5. Обем на автоматизираната проверка по слой

Слой	Файлове	Какво покриват
графични примитиви	13	стойностни типове, геометрия
ядро (договори, таблици)	17	свойства, предимство, жизнен цикъл
разположение	9	измерване и подреждане на всеки мениджър
контроли	190	поведението на всяка контрола
декларативен слой	9	разбор, изграждане, свързване
услуги на устройството	34	—
инициализация	4	регистрация, изграждане на приложението
анимации	5	—
платформени слоеве	19	връзките към native страната

По записа от дневника на разработката към последната пълна проверка конфигурацията без екран изпълнява **3765** случая, всички преминаващи. Числото расте с всяка добавена компонента и се чете наново непосредствено преди предаване.

Съществено е *какво този обем не доказва*. Той измерва преносимата част: разположение, свойства, известяване, жизнен цикъл. Изобразяването не се проверява от него — то се проверява от методиката в Раздел V. Двете мерки са допълващи се и нито една не замества другата: система с изцяло преминаващи тестове може да изобразява погрешно — тестването показва наличието, но не отсъствието на дефекти [27] — а система с точно съвпадение на изображенията може да има грешна семантика, която тези интерфейси просто не докосват.

V. МЕТОДИКА ЗА ЕКСПЕРИМЕНТАЛНА ПРОВЕРКА НА ЕКВИВАЛЕНТНОСТТА

Разделът описва как твърдението за независимост от платформения инструментариум се превръща в измерима величина. Изложението върви от постановката на експеримента, през инструментариума за автоматизирано заснемане и правилата за оценяване, до мерките срещу недостовърно измерване. Последните заемат непропорционално място съзнателно: основната заплаха пред такова изследване не е неточност, а правдоподобна неистина.

По вид подходът принадлежи към визуалното тестване на графичен интерфейс [14], но целта е различна: там сравнението е между две състояния на една и съща система във времето (откриване на регресия), тук — между две независими реализации на едно и също описание в един и същ момент.

5.1. Постановка на експеримента

Измерваната величина е степента на съвпадение между изображението, получено от еталонната реализация, и изображението, получено от разработената, при идентично входно описание, едно и също устройство, една и съща тема и един и същ момент.

Хипотезата е, че за интерфейс, описан изцяло в общото описание, съвпадението може да бъде доведено до праг, при който остатъчната разлика се обяснява само с изброими причини — всяка от които е или отстраним дефект, или изрично изключение от §5.3. Отхвърлянето ѝ би означавало, че съществува клас различия, неотстраними и необясними, тоест че независимостта има граница, която абстракцията не може да опише.

5.1.1. Едно описание, две независими реализации

Методиката е *диференциално тестване* [2]: две независими реализации на едно описание се изпълняват и изходите им се сравняват един с друг, вместо с предварително записан очакван резултат. Валидността на сравнението почива на това, че двете колони получават *един и същ файл*, а не два еквивалентни файла. Описанията на интерфейсите са споделени: едни и същи байтове се четат от еталонното приложение и се вграждат по време на компилация в приложението на разработената система. Така разлика между двете изображения не може да бъде обяснена с различно входно описание — единствената

променлива остава реализацията.

Измерването обхваща **172 интерфейса**, подбрани така, че да покрият всяка реализирана контрола и всеки алгоритъм за разположение, а не да представят типично приложение.

Третата колона е съществена и заслужава обяснение, защото не е очевидна. Разработената система има два входни пътя: изграждане чрез програмния интерфейс и изграждане от общото описание. Ако се сравняваше само единият, разминаване между двата би останало невидимо — а такова разминаване е дефект точно толкова, колкото разминаването с еталона. Затова третата колона изгражда същите интерфейси по другия път и участва в оценяването самостоятелно.

5.1.2. Обхват на измерването и различният статут на слоевете

Измерването обхваща четири платформи, всяка с 172 интерфейса и по две теми, и с по две колони на системата — тоест по четири оценки на интерфейс (§5.3). Петата платформа, тази с различния native инструментариум, участва качествено, но не количествено, по изложената по-долу причина.

Един от слоевете има различен статут и това е част от методиката, а не пропуск: слойът върху native настолния инструментариум изгражда интерфейса от друго семейство контроли, отколкото еталонът използва на същата операционна система. Попикселното съвпадение там е принципно недостижимо и число за него не би означавало нищо. Изискването към този слой е формулирано различно — пълнота на изобразеното съдържание и съгласуваност между двата входни пътя — и той не участва в количествените таблици. Да се оценява количествено там, където оценката няма смисъл, е по-лошо от това да не се оценява.

5.2. Автоматизирано заснемане

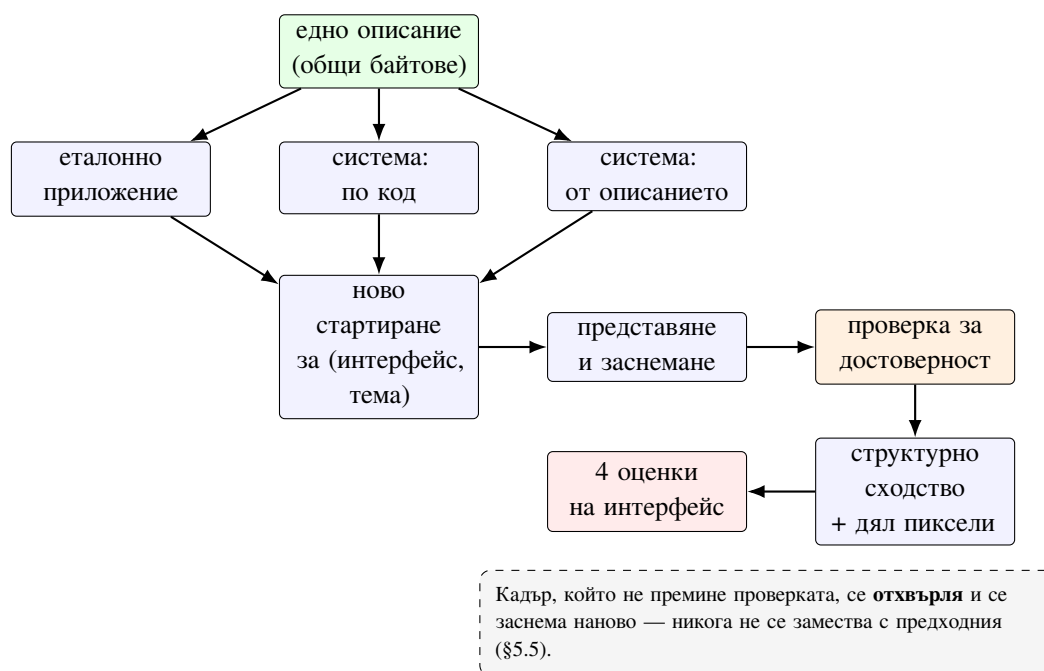
Конвейерът е показан на (Фиг. 9). Средата е виртуална машина, управлявана през отдалечена обвивка; от страната на госта работи агент, който изпълнява отделните действия — разгръщане, стартиране, задвижване, представяне на прозореца в точно определен размер, заснемане — а оценяването се извършва на управляващата машина върху изтеглените кадри.

Възпроизводимостта изисква и средата да е записана. Настолните платформи се

снемат във виртуална машина с изрично зададена разделителна способност 1512×950 при мащаб 1:1. Изричното задаване е необходимо, защото списъкът с режими на такава машина съдържа и режими с двоен мащаб, при които всеки кадър излиза с двойни размери и бива отхвърлен от проверката за размер — без това да личи от съобщенията, тъй като задаването на режима се отчита като успешно. Мобилните платформи се снемат на симулатор и емулатор с фиксиран модел устройство.

Средите към момента на измерването: домакин macOS 26.5.2 (управляваща машина и симулатор), виртуална машина с macOS за настолните платформи, виртуална машина с Windows 11 за платформата на WinUI, и Android емулатор с ниво на API 34. Еталонната реализация е закрепена на разпространена версия 10.0.71 — това е и версията, спрямо която важи правилото „предимство има изобразеното“ (§5.3).

Ограничение: версиите на операционните системи в гостите не са закрепени и се обновяват; при повторение на измерването те трябва да бъдат отчетени наново, тъй като част от разминаванията (§6.3, клас 5) зависят от поведението на конкретна версия.



Фигура 9. Гръбнакът на измерването. Едно описание захранва три приложения; всяка двойка „интерфейс – тема“ се сема от новостартиран процес; кадър, който не премине проверката за достоверност, се отхвърля и заснемането се повтаря, вместо да бъде заместен.

5.2.1. Изолация на всяко измерване

Всяка двойка „интерфейс – тема“ се сема от *новостартиран процес*: желаният интерфейс се задава чрез променлива на средата при стартиране, а не чрез навигация в

работещо приложение. Решението е взето заради заснемането — то премахва зависимостта от предисторията на сесията — но има важно следствие за измерванията в Раздел VI: всяка извадка идва от студен процес, така че нито едно измерване не зависи от реда, в който са минати интерфейсите.

Обратната страна на същото решение се записва тук като ограничение на инструмента, а не се премълчава: при положение че всеки интерфейс се наблюдава в отделен процес, конвейерът *не може* да наблюдава освобождаване на памет при напускане на интерфейс. Величината, която той измерва, е следата на един интерфейс, а не своевременността на освобождаването ѝ.

5.2.2. Задвижване на интерфейса

Задействието има два пътя. Първият е по *абсолютни координати* — агентът в госта натиска в дадена точка. Той работи винаги, но е чувствителен към всяко разместване на разположението: промяна, която измества контролата с няколко пиксела, превръща натискането в натискане по празно място, при това безшумно.

Вторият е по *идентификатор на елемент*: приложението носи вграден агент, достъпен по локален мрежов канал, който намира елемента по идентификатор и предизвиква действието. Този път е устойчив на разместване, но изисква идентификаторите да са зададени в описанието, а не всеки интерфейс ги има.

Изборът е автоматичен: когато стъпка от сценария носи идентификатор и агентът отговори, се използва вторият път; при липса на идентификатор или при отказ се преминава към координати. Отказът се отбелязва в дневника, за да не остане незабелязано, че устойчивият път не е бил използван — иначе една тиха замяна би превърнала устойчив сценарий в чувствителен, без нищо да се промени видимо.

5.3. Правила за оценяване

Сравнението на две изображения дава число; превръщането на числото в оценка „дефект“ или „допустимо разминаване“ е решение. За да не се взема това решение всеки път наново — което би позволило всяко неудобно разминаване да бъде обяснено *post factum* — методиката включва *предварително записан набор от правила за оценяване*. Всяко правило е формулирано веднъж, датирано, и след това се прилага механично. Наборът се допълва само в една посока: ново наблюдение се записва като въпрос и се

решава *преди* да бъде приложено, не след като резултатът е известен.

5.3.1. Основното правило

Еталонът е меродавен за цялото съдържание. Всяка разлика в цвят, размер, разстояние или текст е дефект на разработената система — никога „несъвършенство на еталона“. Без това правило всяко разминаване би подлежало на предоговаряне и мярката би загубила смисъл.

5.3.2. Изрично изброените изключения

Правилото има изключения и те са именно изброени, а не изведени по преценка. Три категории:

1. **Рамка на заснемане.** Външната обвивка, в която еталонното приложение поставя всеки интерфейс, е свойство на приложението-носител, а не на изобразяването. Тя поражда равномерно отместване, което не изменя размер, цвят или вътрешно разстояние на нито една контрола.
2. **Особеност на конкретен платформен инструментариум, която еталонът също търпи.** Когато еталонът върху една платформа изобразява по-малко или различно от това, което изобразява върху останалите, а разработената система изобразява пълното съдържание навсякъде, възпроизвеждането на липсата би било грешка — системата е по-вярна на описанието, отколкото еталонът на тази платформа.
3. **Артефакт на измерването,** а не на изобразяването (§5.5).

Пример за всяка категория:

(1) **Рамка на заснемане.** Еталонното приложение поставя всеки интерфейс във вътрешна рамка с голям външен отстъп и подрязва горния и долния му край. Това измества съдържанието равномерно, но не изменя размер, цвят или вътрешно разстояние на нито една контрола — затова величината на отместването не се брои за разминаване, докато всяка разлика в самите контроли се брои.

(2) **Особеност на инструментариум, която еталонът също търпи.** На една от настолните среди еталонът не изобразява точките на индикатор, не изобразява определени вградени изображения и не рисува сянка, докато на другите две среди ги изобразява. Системата ги изобразява навсякъде. Възпроизвеждането на липсата би било грешка — системата е по-вярна на описанието, отколкото еталонът на тази среда.

(3) Артефакт на измерването — §5.5.

Всяко изключение е *ограничено* до конкретна контрола и конкретна среда, а не е общо разрешение за разминаване. Точно това ограничаване е разликата между изключение и оправдание: изключение, което важи навсякъде, е отказ от мярката.

5.3.3. Противоречие между документацията на еталона и поведението му

Отделно правило урежда случая, в който изходният текст на еталона казва едно, а действително изобразеното от него е друго — положение, което възниква, защото изходният текст е снимка от един момент, а измерването се извършва срещу разпространената версия. Правилото е: **предимство има изобразеното**. Изходният текст остава източник за *механизма*, но не и за стойност, която оттогава се е променила. Всяко такова разминаване се отбелязва в кода като документирано отклонение, с посочване и на двата източника.

5.3.4. Четирите задължителни сравнения

Еталонът се съпоставя *независимо* с всяка от двете колони на разработената система, в двете теми — четири оценки на интерфейс:

№	Еталон	Съпоставена колона	Тема
1	светла	изграждане по програмен интерфейс	светла
2	светла	изграждане от описанието	светла
3	тъмна	изграждане по програмен интерфейс	тъмна
4	тъмна	изграждане от описанието	тъмна

Четирите оценки **не се осредняват**. Осредняването би скрило точно случая, който представлява интерес: единият входен път да е верен, а другият — не. Средната стойност на „вярно“ и „невярно“ изглежда като „почти вярно“, което е погрешно описание на положението.

5.4. Количествено оценяване

Оценяването е детерминистично и се състои от две допълващи се мерки, изчислени за всяка двойка изображения.

Структурно сходство [28] — мярка, чувствителна към промени в структурата на изображението и слабо чувствителна към равномерни отмествания на яркост. Тя

е подходяща, защото интересът е към разлики във *формата и разположението* на изобразеното.

За два съответни прозореца x и y от двете изображения мярката се пресмята като

$$\text{SSIM}(x, y) = \frac{(2\mu_x\mu_y + C_1)(2\sigma_{xy} + C_2)}{(\mu_x^2 + \mu_y^2 + C_1)(\sigma_x^2 + \sigma_y^2 + C_2)}, \quad (1)$$

където μ е средната яркост в прозореца, σ^2 — дисперсията, σ_{xy} — ковариацията между двата прозореца, а $C_1 = (K_1L)^2$ и $C_2 = (K_2L)^2$ са стабилизиращи константи при динамичен обхват $L = 255$ и обичайните $K_1 = 0,01$, $K_2 = 0,03$. Трите множителя в израза съответстват на сравнение по *яркост, контраст и структура*; оценката за цялото изображение е средното на $\text{SSIM}(x, y)$ по всички прозорци. Изчислението се извършва върху яркостния канал.

Свойството, заради което мярката е избрана, се чете направо от (1): равномерно отместване на яркостта изменя μ , но оставя σ_{xy} и σ непроменени, така че третият множител — структурният — не се влияе. Изместване на контрола обаче руши ковариацията и мярката пада рязко.

Дял на видимо различаващите се пиксели. Втората мярка е

$$D = \frac{1}{N} \sum_{i=1}^N \left[\max_{c \in \{R, G, B\}} |a_i^c - b_i^c| > \tau \right], \quad (2)$$

където N е броят пиксели, a_i и b_i са съответните пиксели на двете изображения, $[\cdot]$ е индикатор (1 при истина, 0 иначе), а τ е прагът, отделящ действителна разлика от шум при заглаждане на шрифта. Максимумът по канали, а не средното, е съзнателен: разлика само в един канал — например само в синьото — е разлика, а средното по трите я разрежда до незабележимост. По-сложна перцептуална мярка за цвetoва разлика, като CIEDE2000 [29], беше съзнателно отклонена: тя моделира *прага на човешкото възприятие* на цвят, докато тук целта е обратната — да се улови всяка обективна разлика между два рендера на едно описание, включително такава под прага на забележимост, защото именно тя издава разминаване в реализацията.

Дял на видимо различаващите се пиксели — дялът на пикселите, при които абсолютната разлика по някой от каналите надхвърля праг, отделящ действителна разлика от шум при компресия и заглаждане.

Двете се отчитат заедно съзнателно. Структурното сходство може да остане високо

при дребна, но систематична разлика в цвят — разпределена равномерно по цялото изображение — докато делът на различаващите се пиксели я улавя. Обратно, делът може да остане нисък при отместване на малък, но структурно значим елемент, което структурното сходство наказва. Всяка от двете мерки поотделно има сляпо петно; заедно нямат общо сляпо петно.

Праговете на оценката, приложени към *по-лошата* от двете теми, а не към средната:

Присъда	Структурно сходство	Различаващи се пиксели
съвпада	$\geq 0,98$	и $\leq 1,0 \%$
незначителна разлика	$\geq 0,90$	и $\leq 8,0 \%$
съществена разлика	всичко останало	

Двете условия се изискват *едновременно* — изпълнението само на едното понижава оценката. Пикселът се брои за видимо различаващ се, когато абсолютната разлика по някой от трите канала надхвърли **25** от 255; по-ниският праг брои и шума от заглаждането на шрифта, тоест наказва еднакво изобразен текст.

Праговете са калибрирани срещу два вида наблюдения. Долната граница за „съвпада“ е поставена така, че разлика, дължаща се единствено на заглаждането на шрифта при иначе идентично разположение, да остава под нея — иначе всеки интерфейс с текст би излизал незначително различен. Горната граница за „незначителна“ е поставена така, че изместване на цяла контрола или липсваща контрола да я надхвърля — те трябва да излизат като съществено разминаване.

Втората проверка — по колко интерфейса всяка от двете мерки поотделно би дала различна оценка от съвместната — се изчислява от `comparison.json` и показва дали двете не са излишни една спрямо друга. От **688** оценени двойки: при **8** делът различаващи се пиксели сам би дал оценка, различна от съвместната, а при **3** — структурното сходство само. Нито веднъж двете не се разминават едновременно от съвместната.

Числата са малки, но ненулеви *и в двете посоки* — което е и точният отговор на въпроса: всяка от мерките улавя случаи, които другата пропуска, и премахването на която и да е от тях би променило оценката на единадесет интерфейса. Ако разминаванията бяха само в едната посока, по-простата мярка би била достатъчна.

5.4.1. Отказ от оценяване чрез езиков модел

Първоначално оценките се формираха и от оценка на изображенията чрез езиков модел (практиката „езиков модел като съдия“ [30]). Този път е *изоставен*, след като две

независими такива оценки на едни и същи двойки се разминаха помежду си и се разминаха с измеримото. Причината за отказа е методологична, а не техническа: невъзпроизводима оценка не може да служи за мярка, колкото и полезно да е описанието, което тя дава. Отзивът на такъв модел остава полезен при *диагностика* — за формулиране на хипотеза защо две изображения се различават — но не при *оценяване*.

5.5. Достоверност на измерването

Този параграф разглежда основната заплаха пред изследването. Заснемащ конвейер, който докладва „заснети 1104, откази 0“, не твърди, че файловете показват онова, което претендират — твърди само, че са записани. Разликата не е теоретична: описаните по-долу режими на отказ са настъпвали в хода на работата и нито един не е бил сигнализиран от самия конвейер.

5.5.1. Заснемане, което произвежда правдоподобна истина

Кадър от чужда колона. Когато представянето на прозореца се провали, а предходният файл остане на мястото си, изтеглянето връща предишния файл под новото име. Полученото изображение е напълно валидно, с правилни размери и правдоподобно съдържание — и сравнява едната реализация със самата себе си или с чужда тема. Установеният случай доведе до оценка от около 80% съвпадение за един интерфейс, която не отговаряше на никакъв дефект: никаква промяна в реализацията не би могла да я подобри.

Съвпадащи байтове между теми. Ако темата не е приложена при стартиране, „тъмното“ заснемане е второ светло заснемане. Признакът е точен, защото фонът на интерфейса винаги се различава между двете теми — интерфейс, чиито два кадъра съвпадат байт по байт, е сигнал за дефект, а не рядкост.

Кадър по време на преход — заснемане преди установяване на изобразяването.

И трите преминават всяка проверка, основана на наличие и размер на файла.

5.5.2. Механични проверки вместо доверие

Контролът е механичен и се извършва *преди* оценяването:

- хеширане на съдържанието на всички кадри и откриване на съвпадащи байтове между колони (винаги дефект) и между теми в една колона (дефект, освен ако интерфейсът не

е изрично обявен за независим от темата);

- изискване неуспешното представяне да *отпадне* като кадър, вместо да бъде заместено с предходния;
- изискване двете колони на едно сравнение да произхождат от едно и също изпълнение.

Изведеното от това правило на методиката е формулирано отрицателно: *при съмнение кадърът се отхвърля, а не се приема*. Всеки резервен вариант при заснемане произвежда измислица — разликата е само в това колко правдоподобна.

5.5.3. Смущаващи фактори, установени в хода на работата

Трите фактора по-долу са изброени, защото всеки от тях първоначално е бил диагностициран като дефект на реализацията и е погълнал работа, преди да бъде разпознат.

1. **Смяна на денонощието между двете заснемания.** Интерфейс, показващ дата или час, изменя ширината на изображения низ, което имитира дефект в разположението. Следствие за методиката: интерфейсите с дата и час са съпоставими само когато двете колони идват от едно изпълнение.
2. **Различен мащаб на екрана.** Заснемане на еталона при увеличен системен мащаб поражда цял клас „еталонът е по-едър“, който не е дефект и при опит да бъде „поправен“ води до влошаване на реализацията.
3. **Източник на темата.** Темата произхожда от операционната система, а не от настройка на приложението; ако това не се управлява изрично, двете колони се снемат в различни теми.

Към същия списък спада и системната лента на една от платформите: тя съдържа час и показател за заряд, които се различават между две заснемания по причина, нямаща нищо общо с изобразяването. Тя се изключва от сравнението чрез отрязване на горната ивица от двете изображения преди оценяването; обхватът на отрязването е определен чрез измерване — системната лента заема първите около 135 реда и се различава на всеки интерфейс, докато съдържанието под нея се подравнява.

5.6. Предварителна регистрация на хипотезите

Хипотезите за цената на реализацията са записани *преди* първото измерване, заедно с изискванията за валидност, на които всяко измерване трябва да отговаря. Практиката е установена в емпиричната софтуерна инженерия като *регистриран доклад* [31]; тук е приложена в опростен вид — регистрацията е част от документацията на хранилището и е датирана.

Изискванията за валидност, договорени предварително:

- еднаква конфигурация на компилация от двете страни — сравнение, в което едната страна е компилирана за отстраняване на грешки, а другата за издаване, е негодно и в двете посоки;
- повторения и разпределения вместо единични числа [32];
- отчитане на времето за кадър по горни проценти, а не по средна стойност — забавянето е явление на опашката на разпределението и средната стойност го скрива [33];
- разграничаване на студено от топло стартиране;
- определение за „стартиране“, независимо от реализацията — до първия заснет кадър, който не е начален екран, а не до появата на прозорец, защото прозорец може да предхожда първото изобразяване значително в едната реализация и почти никак в другата;
- слой, който не може да бъде измерен, се отчита като *неизмерен*, а не се осреднява с останалите.

Регистрацията включва и хипотеза, формулирана **срещу** очакването на автора — за величина, при която се очаква да *не* се установи разлика. Ако тя се потвърди, това се отчита като резултат, а не като пропуск: изследване, в което всички прогнози удобно се сбъдват, се чете като застъпничество.

Пълният текст на регистрацията, с датите и с формулировките в оригиналния им вид, е приложен към работата (Приложение Д). Той се възпроизвежда без редакция — преформулирана след факта прогноза не е прогноза.

VI. ЕКСПЕРИМЕНТАЛНИ РЕЗУЛТАТИ И АНАЛИЗ

Разделът представя получените резултати в следния ред: степен на съвпадение и какво от нея е годно за цитиране, представителни съпоставки, систематизация на установените нарушения на независимостта, измерена цена на реализацията, и ограничения.

Основното съдържание е в §6.3. Броят съвпадащи интерфейси е състояние на една реализация към една дата и остарява със следващия отстранен дефект; систематизацията на това *къде* и *защо* абстракцията протича се обобщава извън конкретния артефакт.

6.1. Степен на съвпадение

Обобщението е в (Табл. 6): от 172 интерфейса на Mac Catalyst 161 съвпадат, 9 се различават незначително и 2 — съществено. Пълната картина по интерфейси и платформи е дадена като топлинна карта в Приложение В (Табл. 15), където всяка клетка носи измереното структурно сходство, а цветът на фона следва стойността (зелено 1,000 — жълто 0,75 — червено 0,50) — така изключенията се намират с поглед, вместо да се търсят из четири отделни списъка.

Таблица 6. Степен на съвпадение по платформена лента и входен път

Лента	Входен път	съвпада	незначителна	съществена
iOS	по код	159	12	1
	от описанието	160	11	1
Mac Catalyst	по код	161	9	2
	от описанието	160	10	2
Android	по код	159	10	3
	от описанието	160	10	2
Windows	по код	170	0	2
	от описанието	170	0	2

6.1.1. Интерфейси с побайтово съвпадение

Част от интерфейсите се изобразяват от системата *побайтово идентично* с еталона — не близко, а един и същ файл по съдържание (Табл. 7).

Таблица 7. Кадри с побайтово съвпадение между еталона и системата

Платформа	светла тема	тъмна тема
iOS	44	89
Mac Catalyst	0	0
Android	65	0
Windows	53	53

Резултатът заслужава проверка, преди да бъде приет: побайтово съвпадение е и признакът на известен режим на отказ — кадър, попаднал под две имена (§5.5). Разграничението е направено с три независими проверки:

1. **Времена на файловете.** Трите колони на един и същ интерфейс носят различни времена, подредени по реда на заснемащите проходи — дни и часове една от друга. Размножен кадър би носил един момент.
2. **Темите остават различни.** Във всичките 172 случая светлото и тъмното заснемане се различават. Известният режим на отказ има точно обратния признак — съвпадение и между темите.
3. **Съгласуваност с останалите платформи.** Платформата, при която заснемането обхваща прозорец с жива системна лента и сянка, има *нула* такива случая — което е очакваното, защото там побайтова устойчивост е невъзможна по начина на изграждане.

Побайтовото съвпадение е постижимо тук, защото двете колони минават през един и същ заснемащ инструмент, при една и съща разделителна способност, с фиксиран часовник в системната лента и детерминистичен кодер: еднакви пиксели дават еднакви байтове.

Защо побайтово идентичните са по-малко от получените точна оценка. Числата в (Табл. 7) изглеждат малки до момента, в който се сравнят с броя кадри, получили *точна* оценка — структурно сходство 1,0000 и 0,00 % различаващи се пиксели. Такива са **552**, а побайтово идентични от тях са **304**. Останалите **248** се оценяват като точно съвпадение, но файловете им се различават.

Причината е в самата мярка и е *нарочна*. Пикселът се брои за различаващ се едва когато разликата по някой канал надхвърли 25 от 255 (§5.4) — праг, поставен така, че заглаждането на шрифта да не се брои за разминаване. Измерено върху тези 248 кадъра, най-голямата разлика по канал е между 1 и 24, тоест изцяло под прага. Второ,

структурното сходство се записва с четири знака, така че „1,0000“ означава $\geq 0,99995$, а не точно единица.

Тоест побайтовото съвпадение не е по-слаб резултат, а *по-строг критерий*: 304 кадъра са буквално един и същ файл, а други 248 се различават само с количество, което мярката съзнателно пренебрегва. Двете числа отговарят на два различни въпроса и затова се докладват отделно.

Системната лента. Тя се изключва от сравнението *само* на едната мобилна платформа, където се отрязват първите 140 реда (§5.5). На останалите платформи горната лента участва в оценяването — на симулатора часовникът е фиксиран, така че тя е детерминистична, но на настолната платформа тя съдържа *заглавието на прозореца*, което е име на приложение и се различава по построение. Това е и причината един интерфейс да попадне в (Табл. 8) без нито един различен пиксел в съдържанието си.

Следствие за инструмента, а не за данните. Механичната проверка от §5.5 обявява всяко междуклонно съвпадение за дефект. Правилото е било вярно, докато системата още не е достигала точно съвпадение; след това то поражда лъжливи тревоги, чийто брой расте с качеството на реализацията. Признакът, който остава безусловно верен, е по-тесен — междуклонно съвпадение *в съчетание със* съвпадение между темите. Наблюдението е част от резултата: критерий за достоверност може да остарее заради успех на измерваната система, и това е още едно основание проверката да се преразглежда, а не да се приема за вечна.

6.1.2. Платформи с ограничена съпоставимост

Три платформи носят ограничения, които се отчитат отделно и не се осредняват с останалите:

1. **Тъмната тема на една от мобилните платформи не е съпоставима.** Еталонът там изобразява светъл интерфейс независимо от поисканата тема (измерена средна яркост на тялото 137,7 срещу 139,3 при двете настройки — тоест на практика еднакви), докато системата изобразява действително тъмен. Разминаването е системата да е права срещу неработещ еталон, а не дефект; но точно затова числото за тази тема не носи информация и се отчита отделно.
2. **Една платформа има частично реализиран платформен слой (§4.6).** Интерфейсите,

изградени изцяло от още нереализирани контроли, са празни и в двете колони — което не е съвпадение, а незавършена реализация.

3. **Платформата с различния native инструментариум** не се оценява количествено по замисъл (§5.1); за нея се отчитат пълнота на съдържанието и съгласуваност между двата входни пътя.

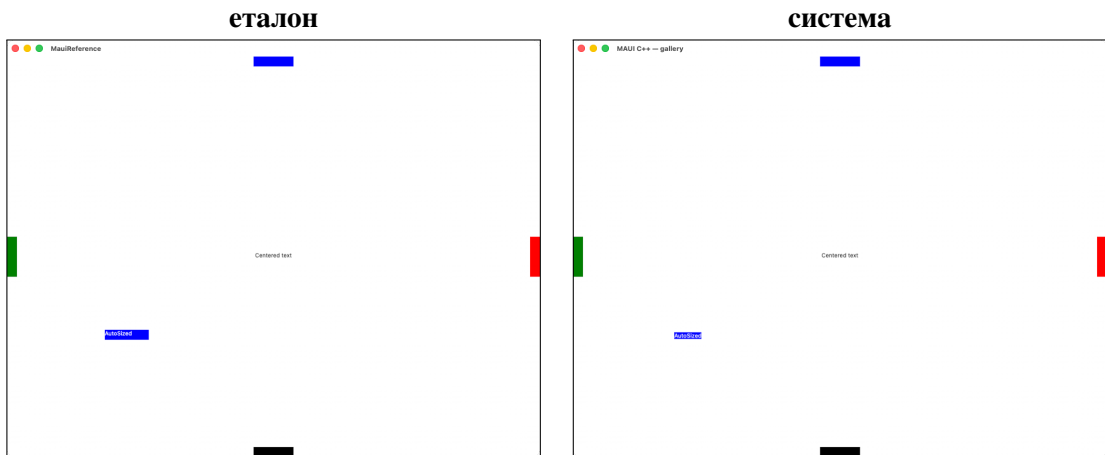
6.1.3. Динамика на резултата

Дневникът на разработката съдържа състоянието на борда след всяка кампания за отстраняване на разминавания (записи от вида „борд: 123→126 съвпадащи“, „147 съвпадащи / 19 незначителни / 6 съществени“), от които се чете, че резултатът е достигнат постепенно, през последователни кампании, а не наведнъж.

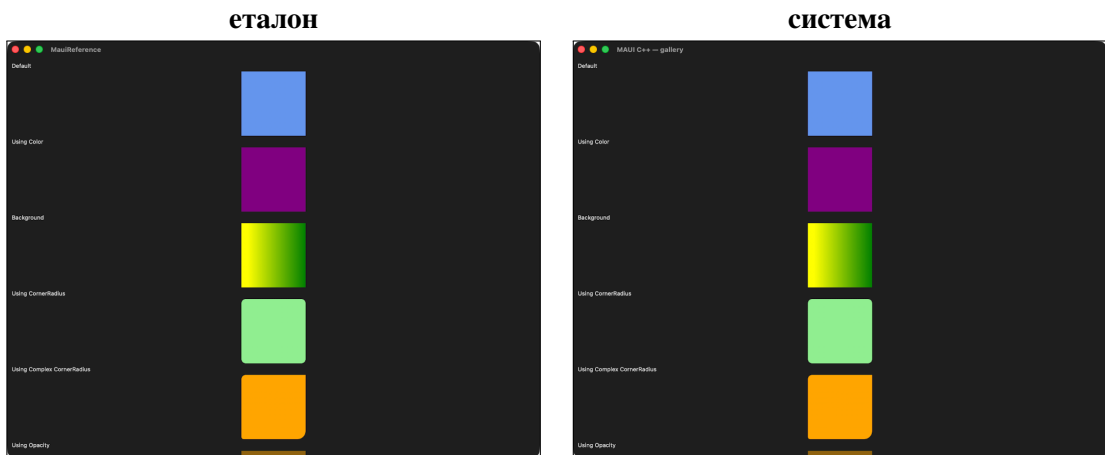
Ограничение, отчетено вместо да бъде заобиколено: тези записи са свободен текст, а не машинно четими данни, и не покриват всяка кампания с еднаква пълнота. Затова тук не се привежда графика — възстановената от непълни записи крива би внушила точност, каквато източникът няма. Систематизацията в §6.3 е изведена от същите кампании и е носителят на този опит; кривата би била илюстрация към него, не доказателство.

6.2. Представителни съпоставки

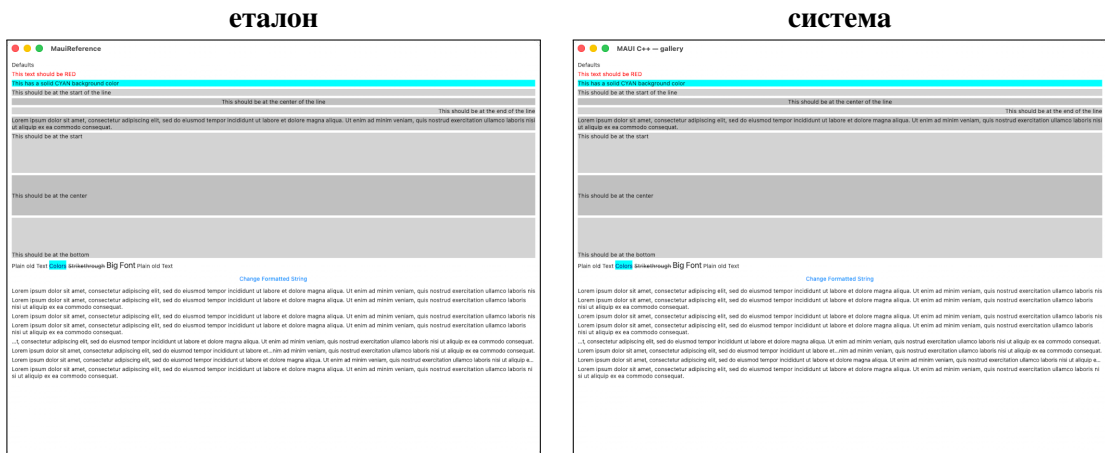
Фигурите по-долу поставят еталона и системата една до друга при идентично входно описание. Изображенията се вземат чрез символни връзки към борда (tools/gen_tex.py), тоест са *същите файлове*, които са били оценени — не подбрани отделно за изложението.



Фигура 10. Абсолютно разположение, светла тема — контрола, при която разминаването би било изместване на всеки елемент поотделно.



Фигура 11. Правоъгълници с цвят, градиент и закръгление, тъмна тема — проверява цвят, геометрия и отсичане едновременно.



Фигура 12. Текстова контрола — проверява шрифт, разстояние между знаците и водещата линия едновременно.

еталон

система

Фигура 13. Списък с групиране — най-натовареният клас интерфейс: заглавия на групи, вътрешни отстъпи и превъртане.

Двойка ПРЕДИ и СЛЕД отстраняване на конкретно разминаване не се привежда като фигура: предишните състояния не са запазени като заснемания, а само като измерени стойности в дневника (например „6 px / 10,5 % → 0 / 0,5 %“). Числата са дадени при съответния клас в §6.3; изображенията не могат да бъдат възстановени, без съответната версия да бъде компилирана наново.

За анимираните интерфейси (14 на брой) фигурата би показвала *първия кадър*; където такъв се използва, това се отбелязва в легендата.

6.2.1. Най-слабо съвпадащите интерфейси

Прозрачността изисква да бъдат показани не само добрите случаи. Таблицата се генерира от данните; само колоната „причина“ се поддържа на ръка, защото оценката е измерване, а причината е находка — интерфейс без установена причина се отпечатва като „открит“, а не получава правдоподобно обяснение.

Таблица 8. Интерфейси със структурно сходство под 0,90

Интерфейс	Платформа	по код	от опис.	Причина
context_flyout	Windows	0,475	0,475	живо външно уеб съдържание
image	Android	0,478	0,999	незарежен отдалечен образ
image	iOS	0,524	0,524	незарежен отдалечен образ
swipe_item_size	Mac Catalyst	0,641	0,640	равномерно отместване 32 px (изрязване от рамката)
context_flyout	Android	0,728	0,728	живо външно уеб съдържание
hybrid_web_view	Android	0,803	0,803	страница за грешка на браузъра
drag_drop	Windows	0,810	0,810	непокрита област: черна в еталона, боядисана в системата
clip	Mac Catalyst	0,885	0,884	заглавието на прозореца, не съдържанието

Общо 8 двойки под 0,90, от които 5 под 0,75.

Три от случаите не са дефекти и не могат да бъдат. Два интерфейса вграждат *живо външно уеб съдържание* — страница, изтеглена по мрежата в момента на заснемането. Двете колони я теглят в различни моменти и получават различно съдържание; при неуспешна мрежа едната показва страница за грешка. Никаква промяна в реализацията не може да ги сближи, защото различието не е в изобразяването, а във входа. Тези интерфейси са изрично изключени от оценяването по правилото за артефакт на измерването (§5.3) и се докладват тук именно защото изключването им иначе би останало невидимо.

Интерфейсът `image` показва защо двата входни пътя не се събират в едно число. На iOS двата пътя се разминават еднакво (0,524 и за двата), но на Android само пътят по код се разминава — 0,478 срещу 0,999 за пътя от описанието. Тоест на тази платформа единият вход изобразява вярно, а другият не; ако двете стойности се осредняваха или се вземаше по-добрата, случаят щеше да изчезне. Това е точно положението, заради което §5.3 изисква четирите сравнения да се отчитат поотделно — и е първото му измерено потвърждение.

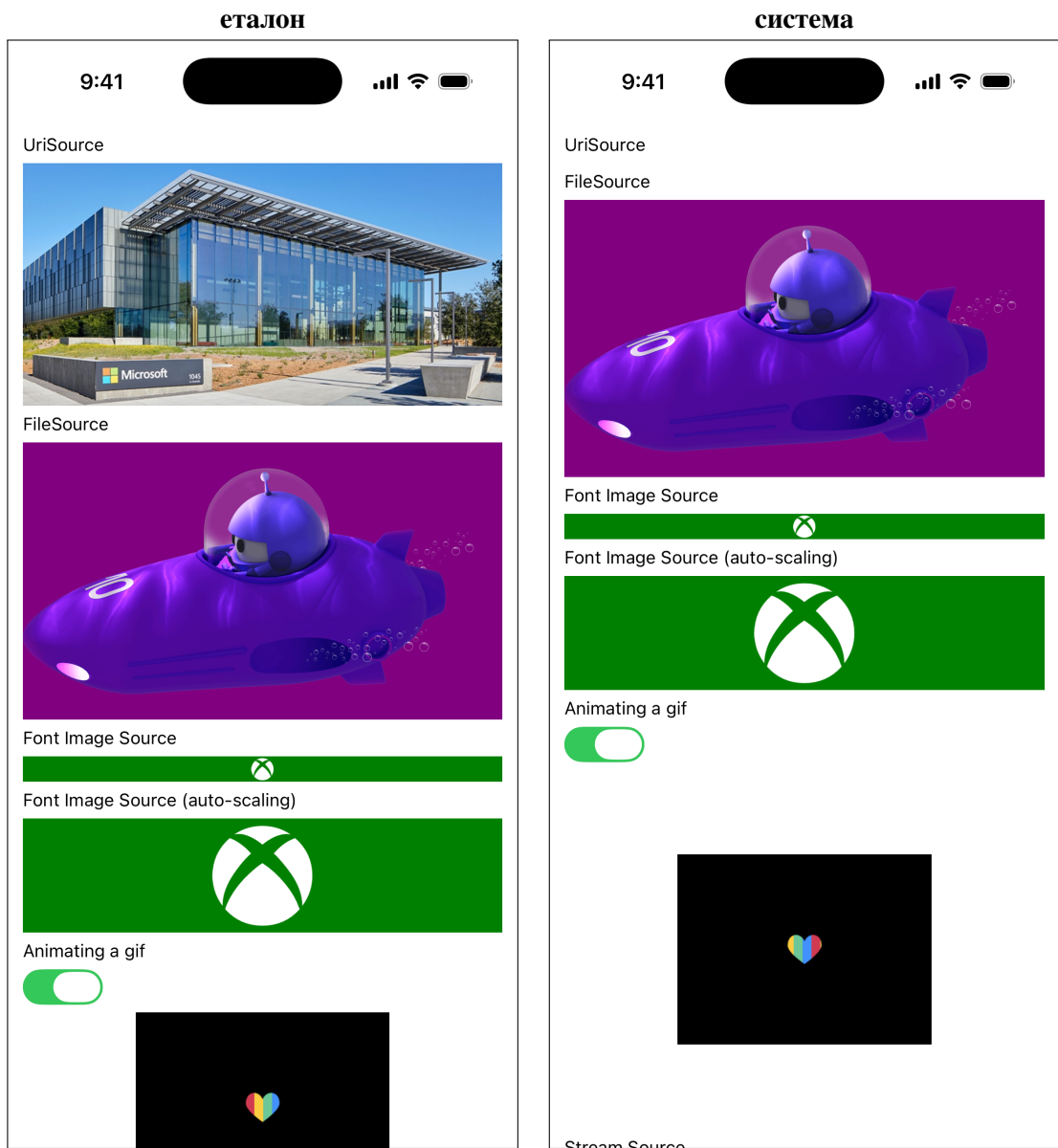
Останалите три бяха доведени до причина чрез оглед на самите изображения — и нито едно от трите не се оказа онова, което числото подсказва.

Равномерно отместване. При единия интерфейс сканирането по вертикално

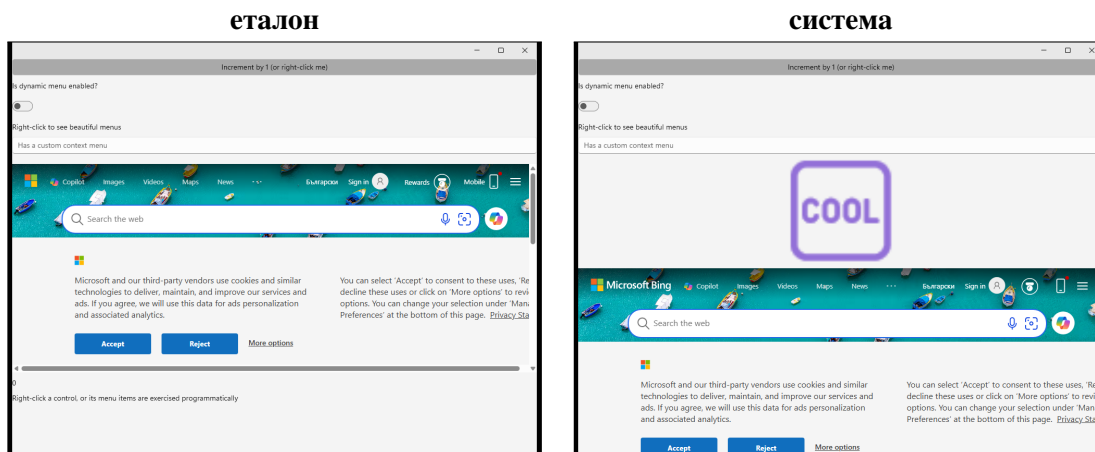
отместване показва, че при изместване с 32 пиксела средната разлика пада от 11,06 на 0,86 — почти 13 пъти. Двете изображения са *една и съща картина*, разминала се по вертикала: рамката на еталонното приложение отрязва горния край на страницата, докато системата я изобразява цялата. По правилото за рамката на заснемане (§5.3) това е изключение, а не дефект — но структурното сходство не различава отместване от различие и го наказва като пълно разминаване.

Заглавие на прозореца. При втория интерфейс разликата е в 716 пиксела от общо 800 000, съсредоточени в правоъгълник 13×115 в горния ляв ъгъл — заглавната лента на прозореца, където двете приложения носят различни имена. Съдържанието на страницата е идентично. Причината структурното сходство да падне до 0,884 от толкова малка област е свойство на мярката: върху почти празна страница локалните прозорци имат нищожна дисперсия, и всяко структурирано различие в тях натежава непропорционално.

Непокрита област. При третия — в тъмна тема — областта под съдържанието е *черна* в еталона и боядисана с фона на страницата в системата. Това е класът на допусканията при наслагването (§6.3, клас 1): там, където съдържанието не покрива прозореца, еталонът оставя основата да се вижда, а системата я боядисва.



Фигура 14. image на iOS. В колоната на системата липсва само изтегляният по мрежата образ; останалите източници се изобразяват вярно, а липсата измества съдържанието под тях.



Фигура 15. Най-ниската измерена стойност на борда — и не е дефект: интерфейсът вгражда жива уеб страница, която двете колонии изтеглят в различни моменти.

6.3. Систематизация на нарушенията на независимостта

Основният резултат на работата. Всяко разминаване, установено в хода на измерването и доведено до първопричина, попада в един от изброените по-долу класове. Систематизацията не е съставена по интуиция: тя обобщава *доведени до първопричина* случаи от кампаниите за отстраняване на разминавания, всеки от които е документиран с измерване преди и след.

Общата констатация, която следва от разпределението им, е нетривиална и противоречи на очакването: *абстракцията не протича на границата на програмния интерфейс на контролите*. Съответствието „свойство → native свойство“ се пренася почти без съпротива. Протичането е на местата, където всеки инструментариум кодира *неизказана уговорка* — допускане, което не е част от документирувания му интерфейс, но е част от поведението му.

Таблица 9. Класове нарушения на независимостта

№	Клас	Къде се проявява	Откриваем без екран?
1	допускания при наслагването	състав от слоеве	не
2	геометрични конвенции	измерване/подреждане	да
3	разрешаване на стойности и ред	таблици, стилове	да
4	входове от средата	всички платформи наведнъж	не
5	семантика на отстъпите	ръбове, вложени изгледи	частично
6	поведение само от реализацията	разклонения по тип	да
7	не са дефекти	—	—

Броят доведени до първопричина случаи във всеки клас не се привежда като число: той би отразявал реда, в който дефектите са били търсени, а не действителното им разпределение. Съотношението, което е информативно, е в последната колона — три от шестте действителни класа са откриваеми без екран, а три изискват сравнение на изображения.

6.3.1. Клас 1 — допускания при наслагването

Разминавания, при които системата рисува вярно, но резултатът се наслагва върху основа с други свойства от очакваните. Класът е особено труден за откриване, защото изолираното измерване на слоя показва правилен резултат — грешката се проявява едва в състава.

Разгледан пример — фон на прозореца, изграден от два слоя. Еталонът установява на прозореца системен ефект зад съдържанието (полупрозрачно замъгляване на работния плот), а върху него — *полупрозрачен* слой за областта на съдържанието, взет от ресурсите на инструментариума. Наблюдаваният цвят на тялото е *съставен* от двете.

Системата възпроизведе първо само първия слой. Измереното при това положение — стойност 232 в светла тема — вече доказваше, че ефектът работи: непрозрачно запълване със стойността по подразбиране би дало 243, не 232. Въпреки това цялото не съвпаднаше, защото липсваше вторият, полупрозрачен слой. Обратно — добавянето на *непрозрачно* запълване на основата се наслагва *пред* системния ефект и го обезсилва напълно. Правилното решение е основата да не се боядисва изобщо там, където ефектът е наличен, и да се боядисва само на система, която не го поддържа.

Класът е поучителен по два начина. Първо, изолираното измерване на всеки слой поотделно показва правилен резултат — грешката съществува само в състава. Второ, посоката на поправката е обратна на интуитивната: не „добави още боя“, а „махни боята, за да се вижда това, което е отдолу“.

Измерените стойности: при първата реализация тялото на интерфейса отчита яркост 232 в светла тема — точно стойността на системния ефект. Непрозрачно запълване със стойността по подразбиране би дало 243, което е и проверката, че ефектът действително е бил приложен. Тъмната тема отчита 32 при същото положение.

Заснемане отпреди поправката не е запазено — бордът съхранява текущото състояние, не историята на изображенията. Възстановяването му изисква компилиране на съответната предишна версия; величината на разминаването е дадена по-горе с измерените стойности, които са запазени.

6.3.2. Клас 2 — геометрични конвенции

Класът с най-много случаи. Общото между тях: всяка от двете страни има вътрешно съгласувана конвенция, конвенциите не съвпадат, и нито една не е записана в интерфейса.

Разгледан пример — несиметрично отчитане на външния отстъп. Договорът за измерване изисква измерващата фаза да *прибави* външния отстъп към съобщения размер, а подреждащата — да го *извади* от рамката; при нулев отстъп двете се компенсират, затова грешката е невидима в най-простия случай. В реализацията базовият изглед изпълняваше и двете, но преопределянето за контейнерите — само изваждането. Следствието е, че рамката на *всеки* контейнер губеше двукратния си отстъп: при поискана ширина 100 и отстъп 12 подреждането даваше 76 вместо 100.

Двете фази и посоката на прибавяне/изваждане са показани на (Фиг. 5). Измерените стойности: при поискана ширина 100 и външен отстъп 12 подреждането даваше 76 — отклонение точно от двукратния отстъп, което и насочи към несиметричността.

Случаят е показателен и в друго отношение. Преди установяването на действителната причина бяха проверени и *отхвърлени с измерване* две правдоподобни хипотези — че вътрешният отстъп свива изричния размер, и че конкретна контрола пренебрегва собствения си външен отстъп. И двете бяха опровергани чрез насочена проверка без екран, а не чрез разсъждение. Отхвърлените хипотези са записани, за да не бъдат изпробвани отново.

Втори пример — половинпикселно свиване на контура. Инструментариумът за

чертане свива контура на фигура с половин единица навътре, за да го центрира върху ръба. Същата фигура обаче се използва и като *път на отсичане* за съдържанието вътре в нея; там свиването няма основание и води до отрязване на половин пиксел от съдържанието по целия периметър. Разминаването е дребно на всеки отделен ръб и видимо като цвят по цялата рамка.

Трети пример — координатната система при подреждане (§3.5): абсолютни спрямо относителни на контейнера координати. Погрешният избор води до двойно отместване само при вложен контейнер с ненулево начало, тоест не се проявява при най-простите интерфейси — което го прави типичен представител на класа.

6.3.3. Клас 3 — разрешаване на стойности и ред на прилагане

Разгледан пример — зависимост от реда в таблицата за преобразуване. Размерът на бутон се изчислява от вътрешните отстъпи на native контролата, които трябва да включват и дебелината на контура. Двете свойства се прилагат от таблицата в определен ред — отстъп преди контур — така че при изчисляване на размера дебелината още е нула и контурът никога не достига до отстъпите. Еталонът оцелява при същия ред, защото на по-горното си ниво *замества* действието за отстъп с преизчисляване при измерване, тоест чете дебелината в момента на измерването.

Поуката е обобщима: *редът на прилагане в таблицата е част от наблюдаваното поведение*, а не вътрешна подробност. Решението в системата е изчислението да получава дебелината като аргумент, с което става независимо от реда по начина на изграждане — поправка на класа, а не на случая.

Втори пример — минимална ширина, наложена от стила на платформата. Еталонът съдържа две присвоявания, които зануляват минималната ширина и височина на превключвател без текст. На една от платформите те са *безрезултатни*: действието за минимална ширина в таблицата изтрива локално зададената стойност, след което в сила влиза минимумът от стила на инструментариума — 120 единици. Този минимум надделява дори над изрично зададена от приложението ширина, защото инструментариумът повдига ширината до минимума.

Системата първоначално пренесе двата реда буквално и получи 32 единици (или голата зададена ширина) вместо 120. Вярното пренасяне е *да не се пренасят*: незадаването на стойност е точно състоянието след изтриването, и тогава минимумът от стила се проявява от само себе си. Случаят показва, че буквалното пренасяне на ред от еталона

може да *разминава* поведението — значение има резултатът, а не текстът.

Трети пример — „незададено“ срещу „зададено на стойността по подразбиране“. Проверка, която пита за *стойността* вместо за *наличието* на стойност, не може да разграничи двата случая. Последницата е конкретна: контрола, на която е зададен изрично черен цвят на текста, се третира като контрола без зададен цвят и получава цвета по подразбиране на платформата — бял при тъмна тема. Текстът става невидим. Поправката е въвеждането на изрична проверка „това свойство задавано ли е“, различна от сравнение със стойността.

6.3.4. Клас 4 — входи от средата

Разминавания, чийто източник е стойност, която приложението *не* задава, а получава от средата. Отличителният им признак: проявяват се едновременно на всички платформи, а причината е на едно място.

Разгледан пример — източникът на темата. Приложението има слот за тема, но началната му стойност *не* е „светла“ — еталонът я установява в конструктора си от операционната система. Системата отначало пропускаше този един ред, при което слотът оставаше на подразбиращата се стойност. Последниците не са малки: на машина с тъмна тема всяка платформа едновременно показва интерфейс, изграден по светли правила, върху тъмна системна основа.

Отличителният признак на класа е именно този — разминаването се появява *наведнъж навсякъде*, докато причината е на едно място. Клас, при който всяка платформа се проявява поотделно, почти никога не е от този вид.

Интерфейсът, който свързва темата (`app_theme_binding`), е и този, на който разминаването се вижда най-пряко — той изобразява стойности, избрани според текущата тема. Измереното след поправката: от 4,6 % различаващи се пиксела до 0,08 % — тоест от съществено разминаване до точно съвпадение.

Заснемане отпреди поправката не е запазено по същата причина; запазено е измерването.

6.3.5. Клас 5 — семантика на отстъпите и безопасните зони

Класът, който най-ясно показва, че протичането не е между *платформи*, а между *уговорки* — две среди от едно и също семейство могат да се разминават помежду си.

Разгледан пример — двойно отчитане на безопасната зона. Отстъпът на

безопасната зона се подава на съдържанието на страницата, но съдържанието е отместено от ръба и от собствения си външен отстъп. На мобилната среда системната лента *покрива* съдържанието — двете величини се мерят от един и същ ръб — така че подаването на пълния отстъп върху вече отместено съдържание го брои двукратно. Поправката изважда собствения отстъп на съдържанието.

Същата поправка обаче *влоши* настолната среда от същото семейство: там платформата на прозореца стои *над* областта на съдържанието, тоест двете величини се сумират, и еталонът изобразява сбора им. Наложил се разклонение по среда — изваждане на едната, пълен отстъп на другата.

Втори пример от същия клас. Поведението на подравняването на съдържанието при списъчна контрола: еталонът установява стойност, при която съдържанието минава под системната лента. Пренасянето ѝ поправи осем интерфейса на мобилната среда и същевременно влоши настолната — където един и същ ред от изходния текст на еталона изобразява различно. Отново се наложи разклонение.

Изводът е по-силен от двата случая поотделно: *една и съща стойност в един и същ изходен текст на еталона поражда различно изобразяване в две среди от едно семейство*. Абстракция, която предполага, че семейството е еднородно, ще сгреша — и ще сгреша безшумно, защото на едната среда резултатът е правилен.

6.3.6. Клас 6 — поведение, изводимо само от реализацията на еталона

Отделен клас, при който разминаването не се дължи на уговорка, а на поведение, което не следва от описанието и се открива само чрез прочитане на реализацията.

Разгледан пример. Заглавната и заключителната част на списъчна контрола се изобразяват или не в зависимост от *съчетанието* на вида разположение и това дали елементите са групирани — защото вътрешното разпределение на еталона се разклонява по типа на разположението и при групиране излиза предсрочно, преди да достигне до тях. Системата беше приложила правилото на единия вид разположение върху всички. Поведението не е документирано никъде и не следва от нито един тест; изведено е от прочитане на разпределението по тип.

6.3.7. Клас 7 — разминавания, които не са дефекти

Два подкласа, разграничени изрично:

1. **Особеност на конкретен платформен инструментариум**, която еталонът също търпи

— възпроизвеждането ѝ би било грешка.

2. Артефакт на измерването (§5.5, §6.1.1).

Класът съществува, защото и трите смущаващи фактора от §5.5 бяха първоначално диагностицирани като дефекти на системата. Изброяването им предпазва следващия изследовател от същата загуба.

6.3.8. Изводи от систематизацията

Три извода следват от систематизацията.

Първо — къде да се насочи вниманието. Не към програмния интерфейс на контролите: съответствието „свойство → native свойство“ се пренася почти без съпротива. Вниманието принадлежи на конвенциите — геометрични, стойностни, средови — които никой инструментариум не документира, защото за него самия те са очевидни.

Второ — разделение на средствата за откриване. Класове 2, 3 и 6 са откриваеми без екран, защото са за сметка на преносимата логика; те принадлежат на модулното тестване. Класове 1, 4 и 5 изискват сравнение на изображения, защото се проявяват само в състав или само на конкретна среда. Никое от двете средства не покрива другото — което е и отговорът на въпроса защо са необходими и двете.

Трето — следствие за реда на работа. Тъй като половината класове са невидими без сравнение на изображения, измервателният апарат трябва да се изгражда *успоредно* със системата, а не след нея. Изграден след това, той открива наведнъж натрупани разминавания, чиито причини вече са преплетени; изграден успоредно, той ги открива по едно, докато причината е още една.

6.4. Цена на реализацията

Измерванията тук са възможни само защото предходните установяват *контролно условие*: двете реализации изобразяват едно и също съдържание върху едни и същи native контроли. Без него разлика в размер или бързодействие не би била отличима от разлика в обхвата на реализираното.

Числата по-долу важат за платформата, чието контролно условие е потвърдено наново срещу release-компилациите — проверка, извършена върху пряко заснемане и независима от наблюдението в §6.1.1.

6.4.1. Размер на артефакта

Таблица 10. Размер на артефакта при release-компиляция от двете страни

Колона	На диск	Симетрично	Конфигурация
еталон (управляван)	78,7 MB	47,3 MB	release, отрязан + AOT
система, по код	9,1 MB	9,1 MB	release, без символи
система, от описанието	22,1 MB	22,1 MB	release, без символи

(а) **Симетрия на обработката.** Чете се колоната „симетрично“, не „на диск“. Native артефактите се разпространяват без символна информация, затова техният размер на диск вече е окончателен, докато управляваният носи 33,3 MB символна информация в самия изпълним файл. Премахването ѝ само от едната страна дава съотношение $8,6\times$ вместо $5,2\times$ — завишение с 65 %. Това е най-лесната възможна грешка в такова сравнение и затова се отбелязва изрично.

Съотношения: $5,2\times$ спрямо изграждането по код и $2,1\times$ спрямо изграждането от описанието. Посоката на прогнозата се потвърждава; величината — не. „Многократно по-обемен“ е пресилено при $2,1\times$ срещу сравнимата по вход колона.

(б) **Разпределение по компоненти.** От 15,0 MB метаданни и управлявани библиотеки:

Компонент	Метаданни	Библиотеки	Общо
среда за изпълнение + базови библиотеки	6,23 MB	2,30 MB	8,53 MB
код на фреймуорка	2,48 MB	3,48 MB	5,96 MB
код на приложението	0,32 MB	0,17 MB	0,49 MB

Средата за изпълнение води, както е предсказано — но кодът на фреймуорка е **40 %** от разпределените байтове, а не пренебрежим остатък. По-острата формулировка на прогнозата („разликата се дължи *специфично* на средата, а не на кода на фреймуорка“) е подкрепена само наполовина.

(в) **Неразпределена част.** Таблицата покрива 15,0 MB. Преобладаващата компонента — 31,5 MB предварително компилиран образ — **не е разпределена**: маркерите в него са структури от данни, а не граници на код, и разделянето ѝ по библиотеки изисква разбор на формата. Мащабиране по пропорциите на метаданните би подсказало приблизително $21,7 / 8,5 / 1,1$ MB — записано като *оценка по заместител*, а не измерване, и затова не влиза в таблицата.

6.4.2. Защо декларативният вариант е по-обемен от кодовия

Двата варианта на системата се различават съществено по размер (9,1 срещу 22,1 MB). Естественото предположение е, че разликата идва от *вградените описания*: `markur`-ът се вгражда в единицата за превод и трябва да лежи някъде в артефакта. Предположението е проверено и се оказва вярно само отчасти.

Общият обем на вградените описания е **820 KB** — около 6 % от разликата. Разпределението по секции на двата изпълними файла показва къде е останалото:

Таблица 11. Състав на двата **native** артефакта (**release**, без символна информация)

Секция	по код	от описанието	разлика
<code>__text</code> (машинен код)	5,96 MB	10,12 MB	+4,16 MB
<code>__const</code> (константни данни)	0,52 MB	1,35 MB	+0,83 MB
<code>__cstring</code> (низове)	53 KB	34 KB	–19 KB
<code>__TEXT</code> общо	7,63 MB	15,04 MB	+7,41 MB

Прирастът на `__const` (+0,83 MB) съвпада с обема на вградените описания — те наистина са там, но са малката част. Прирастът на `__text` е **пет пъти по-голям**: разликата е *машинен код*, а не данни.

Причината следва пряко от липсата на рефлексия (§2.6). Кодовият вариант извиква конструкторите на контролите, които ползва, и свързващият редактор изхвърля всичко останало. Декларативният вариант не знае предварително кой тип ще се появи в описание, затова трябва да носи *регистър на всички възможни типове* — фабрика на обект и преобразувател „текст → стойност“ за всяко свойство — плюс самия разбор, изграждането на обектния граф и разрешаването на изразите за свързване. Всяка от тези регистрации е код, и нито един от тях не подлежи на изхвърляне, защото се достига по данни.

Изводът е обобщим и малко неочакван: при среда без рефлексия цената на декларативния вход не е в описанието, а в *машинерията, която прави описанието изпълнимо*. Управляваната среда плаща същото, но веднъж — нейният механизъм за рефлексия вече е в средата за изпълнение и се използва от всяко приложение.

6.4.3. Време до първи кадър (H2b-1)

Това е единствената от трите хипотези за бързодействие, за която има измерване. Инструментът реализира определението, регистрирано предварително в §5.6: времето от стартиране до *първия заснет кадър, който не е начален екран* — изрично не до появата на

прозорец. Всяка стойност е медиана от 10 студени стартирания на един и същ интерфейс — медиана над повторения, а не единично число, следвайки практиката за статистически издържано измерване на производителност [32].

Таблица 12. Време до първи кадър при студено стартиране (медиана; в скоби 95-и процентил)

Платформа	еталон	по код	от описанието
Android	1,442 s (1,714)	1,016 s (1,252)	1,060 s (1,184)
iOS	1,990 s (2,259)	1,148 s (1,237)	1,210 s (1,217)

Неизмерени платформи: **Mac Catalyst** — capture path is SSH -> screencapture -> scp (~1-3 s per sample); TTFF is ~1 s, so instrument resolution matches the signal; **AppKit** — same as maccatalyst — VM capture round trip dominates the measurement; **Windows** — app builds and runs on the guest; host-side polling measures transport, not startup.

Системата стартира по-бързо и на двете измерими платформи: **1,42 пъти** на Android и **1,73 пъти** на iOS. Двата входни пътя на системата се държат еднакво — декларативният не плаща за себе си при стартиране, което е и очакваното, след като разборът е извършен при компилация (§4.7).

Ограничение, което не бива да се пропуска. Разделителната способност на измерването е интервалът на извадката — около 0,24 s и на двете платформи. Разликите между двата входни пътя на системата (0,044 s на Android, 0,062 s на iOS) са *под* тази граница и затова *не са разрешими*: те се докладват като „еднакви“, а не като измерена малка разлика. Разликата спрямо еталона — 0,43 s и 0,84 s — е няколкократно над границата и е разрешима. Една от десетте извадки на iOS е пропусната и за двата входни пътя (измерени са 9 от 10).

Забележка за устойчивостта на измерването. По-ранен пробег на същия инструмент даде за Android съотношение 3,1 вместо 1,42 — при непроменен код. Разликата идва от състоянието на емулятора, не от системата, и е записана тук, защото показва нещо за самата величина: студеното стартиране на емулирано устройство е чувствително към условия извън измерваната програма. Цитира се последният пълен пробег, а не най-благоприятният.

Три платформи остават неизмерени и причината е записана в самата таблица: при тях заснемането минава през виртуална машина и обиколката отнема от порядъка на самото измервано време. Инструмент, който измерва предимно собственото си закъснение,

не се докладва като измерване — за тях е необходим таймер от страната на госта.

6.4.4. Потребена памет и останалите хипотези за времето

Измерванията по **H2a** (потребена памет), **H2b-2** (разположение) и **H2b-3** (установен режим на изобразяване) *не са проведени* към момента на предаване.

Отбелязва се изрично, че H2b-3 е формулирана *срещу* очакването на автора — тя прогнозира липса на разлика при установен режим. Потвърждаването на H2b-1 не е довод в нейна полза: студеното стартиране и установеният режим измерват различни неща, и точно затова са регистрирани като отделни хипотези.

6.4.5. Съответствие с предварително регистрираните хипотези

Таблица 13. Съответствие между регистрираните хипотези и измереното

№	Прогноза (регистрирана предварително)	Исход
H1	управляваният артефакт е многократно по-обемен; разликата се дължи специфично на средата за изпълнение	частично потвърдена — посоката се потвърждава ($5,2 \times / 2,1 \times$), величината не; кодът на фреймуорка е 40 % от разпределеното
H1'	разликата е голяма на Apple платформите и малка на Windows, където и двете колонии изискват една и съща платформена среда	непроверена — платформата Windows се изгражда в госта и не е измерена от домакина
H2a	по-малка следа на паметта	непроверена
H2b-1	значително по-бързо студено стартиране	непроверена
H2b-2	измеримо предимство при разположението	непроверена
H2b-3	<i>липса</i> на разлика при установен режим	непроверена

Четири от шестте прогнози остават непроверени, и таблицата го казва вместо да го премълчи. Единствената проверена е и единствената, при която прогнозата беше *твърде силна* — което е точно случаят, заради който предварителната регистрация има смисъл: без нея формулировката „многократно по-обемен“ би била пренаписана след факта на „по-обемен“ и никой не би узнал, че първоначалното очакване е било по-крайно.

6.5. Ограничения на изследването

1. **Обхват на проверката за достоверност.** Механичната проверка (§5.5) улавя размножен кадър и неприложена тема, но не улавя кадър, заснет по време на преход, ако размерите му са правилни. Такъв кадър би понижил оценката, тоест грешката е в консервативната посока — но остава неоткривана автоматично.
2. **Освобождаване на памет.** Изолацията на всяко измерване (§5.2) прави невъзможно наблюдението на освобождаване при напускане на интерфейс — измерва се следа, не своевременност.
3. **Разделителна способност при измерване на стартиране** — определението стъпва на честотата на заснемане и не претендира за милисекундна точност.
4. **Обвързаност с една версия на еталона.** Цялото сравнение е спрямо конкретна разпространена версия; резултатът не се пренася автоматично към следваща.
5. **Подбор на интерфейсите.** Наборът е съставен за покритие на контролите и алгоритмите, не като извадка от типични приложения — заключенията се отнасят за покритието, не за средното приложение.

ЗАКЛЮЧЕНИЕ

В хода на дипломното проектиране е изградена работеща система, която описва графичен потребителски интерфейс независимо от платформения инструментариум, и — което е по-същественото — методика, с която това описание може да бъде *проверено*, а не само заявено. По задачите от Увода, в същия ред: направен е преглед на подходите и е показано защо твърдението за еднакво изобразяване остава непроверено (Раздел I); анализирани са алтернативите по пет предварително изброени критерия и изборът е обоснован, включително с преброяване в кодовата база вместо с преценка (Раздел II); проектирана е слоеста архитектура, в която платформено обусловената част е сведена до таблично зададено съответствие (Раздел III); тя е реализирана върху четири native инструментариума и един слой без екран (Раздел IV); изградена е методика за автоматизирано измерване на еквивалентността на изобразяването (Раздел V); и е проведена експериментална проверка върху 172 интерфейса, от която е изведена систематизация на установените разминавания (Раздел VI).

Приноси.

1. **Систематизация на нарушенията на независимостта** — шест класа, всеки изведен от доведени до първопричина случаи, с обща констатация, която противоречи на очакването: абстракцията не протича на границата на програмния интерфейс, а на местата, където инструментариумът кодира неизказана уговорка. Това е приносът, който се обобщава извън конкретния артефакт.
2. **Методика за количествено измерване на еквивалентността** между две независими реализации на едно описание, включително правилата за оценяване, записани предварително, и механичните проверки за достоверност на самото измерване.
3. **Разграничение на средствата за откриване:** половината класове са откриваеми без екран, половината изискват сравнение на изображения — от което следва, че измервателният апарат трябва да се изгражда успоредно със системата, а не след нея.
4. **Реализацията като инструмент** — система върху пет платформени слоя, която прави измерването изобщо възможно, с публичен интерфейс, при който най-честият клас грешки при управление на паметта е грешка при компилация.
5. **Предварително регистрирани хипотези** за цената на реализацията, с изричните

изисквания за валидност — включително хипотеза, формулирана срещу очакването на автора.

Предимства на предложеното решение. Спрямо алтернативите от Раздел II решението печели по трите критерия, които определят изхода на работата. По **K1 (проверимост)** — послойното изграждане, водено от тестове, е единствената от трите стратегии, при която съществува общо описание, изпълнявано и от двете реализации, тоест контролно условие за сравнение. По **K4 (безопасност по конструкция)** — само преместваемият притежаващ манипулатор превръща най-честия начин за създаване на цикъл в грешка при компилация, при измерена ергономична цена, падаща върху малцинство от местата. По **K5 (цена за платформа)** — изборът на език дава пряк достъп до три от петте инструментариума без нито един ред обвързващ код.

Отделно стои предимство, което не е било предвидено като цел. На настолната Apple платформа еталонната реализация предлага *само* пътя през мобилния инструментариум, пренесен на настолната машина — приложението изглежда като мобилно приложение в прозорец. Разработената система носи *и двата* слоя: същия мобилен път, който позволява прякото сравнение в Раздел VI, и втори слой върху native настолния инструментариум, чиито контроли са тези на операционната система. Изборът е на приложението и не изисква промяна в описанието на интерфейса. Това е пряко следствие от архитектурата: щом платформената част е сведена до таблица от действия, две различни таблици за една и съща операционна система не са изключение, а обикновен случай.

Печели се и нещо друго, също непредвидено: побайтово съвпадение с еталона на част от интерфейсите (§6.1.1) — най-силната форма, която резултатът от такова измерване изобщо може да приеме.

Недостатъци и ограничения.

1. **Четири от шестте регистрирани хипотези остават непроверени.** Измерванията за потребена памет и за време за реакция не са проведени; инструментът е проектиран, но не е изпълнен. Единствената проверена хипотеза се оказва *твърде силна* по величина.
2. **Паритетът не е постигнат навсякъде.** На всяка платформа остават интерфейси с незначително разминаване, а на две — и със съществено. Причините са отнесени към класове в §6.3, но не са отстранени.
3. **Един платформен слой е реализиран частично по замисъл** — изобразяват се само

част от контролите, останалите използват огледалото без екран.

4. **Цената на модела на собствеността** е реална: достъпът до задържана контрола изисква проверка за живост, а съвместното притежаване — изрично действие. Това е съзнателна размяна на удобство срещу проверимост, не безплатно предимство.
5. **Инструментът не може да наблюдава освобождаване на памет** при напускане на интерфейс, тъй като всяко измерване се извършва в отделен процес (§5.2) — ограничение на конструкцията, не пропуск в изпълнението.
6. **Резултатът е обвързан с една версия на еталона** и не се пренася автоматично към следваща.
7. **Един критерий за достоверност остаря заради успеха на системата** (§6.1.1) — проверката обявява побайтовото съвпадение за дефект, което е било вярно, докато то е било недостижимо.

Насоки за по-нататъшна работа.

1. **Провеждане на измерванията по H2a и H2b** — пряко следствие от ограничение 1. Инструментът е проектиран; остава изпълнението и отчитането, включително на отрицателния резултат, ако H2b-3 се потвърди.
2. **Стесняване на критерия за достоверност** — следствие от ограничение 7: признакът трябва да стане „междуклонно съвпадение и съвпадение между темите“, иначе броят лъжливи тревоги ще расте с качеството на реализацията.
3. **Довършване на частичния платформен слой** (ограничение 3) и повторно измерване — това е и проверка дали систематизацията от §6.3 предсказва правилно кои класове ще се появят на нов инструментариум.
4. **Платформен слой за Linux** — най-информативното възможно продължение, защото еталонната реализация *няма* такъв. Причината не е техническа невъзможност, а следствие от собствената ѝ архитектура: тя не рисува сама, а преобразува към native инструментариум, и затова изисква на всяка платформа да съществува *един* инструментариум, чиито контроли са очевидният избор — какъвто на Linux няма (GTK и Qt съжителстват, а работната среда не предопределя кой от двата е „системният“). Към това се добавя, че поддръжката на платформа за нея означава и разпространение на средата за изпълнение за нея.

Разработената система няма нито едното ограничение: тя не носи среда за изпълнение, а слой се пише срещу избран инструментариум. Основното предизвикателство се измества другаде — при липса на еталон на тази платформа *отпада оракулът*, а с него и методиката от Раздел V. Проверката там може да бъде само вътрешна: съгласуваност между двата входни пътя и съответствие с изчисленото от преносимия слой разположение, но не и съвпадение с външен образец. Това е и границата на самия метод, която новата платформа би направила видима.

5. **Разширяване на систематизацията с шести платформен слой** от друго семейство — всяка нова среда е *повторение* на експеримента и проверява дали класовете са свойство на задачата или на конкретните четири инструментариума.
6. **Автоматизирано следене на разминаванията във времето** — запазване не само на текущото състояние, а и на заснеманията при всяка кампания, което би направило възможна и кривата от §6.1, и съпоставките ПРЕДИ/СЛЕД, които сега липсват (ограничение по §6.2).

ЛИТЕРАТУРА

- [1] E. T. Barr, M. Harman, P. McMinn, M. Shahbaz, and S. Yoo, “The oracle problem in software testing: A survey,” *IEEE Transactions on Software Engineering*, vol. 41, no. 5, pp. 507–525, 2015.
- [2] W. M. McKeeman, “Differential testing for software,” *Digital Technical Journal*, vol. 10, no. 1, pp. 100–107, 1998.
- [3] L. Chen and A. Avizienis, “N-Version programming: A fault-tolerance approach to reliability of software operation,” in *Digest of Papers, FTCS-8: Eighth International Symposium on Fault-Tolerant Computing*, (Toulouse, France), 1978.
- [4] M. C. Feathers, *Working Effectively with Legacy Code*. Prentice Hall, 2004.
- [5] A. Biørn-Hansen, C. Rieger, T.-M. Grønli, T. A. Majchrzak, and G. Ghinea, “An empirical investigation of performance overhead in cross-platform mobile development frameworks,” *Empirical Software Engineering*, vol. 25, no. 4, pp. 2997–3040, 2020.
- [6] Microsoft, “Xamarin.Forms documentation.” <https://learn.microsoft.com/xamarin/xamarin-forms/>. посетен на 10 август 2026 г.
- [7] Microsoft, “.NET multi-platform app UI (.NET MAUI) documentation.” <https://learn.microsoft.com/dotnet/maui/>. посетен на 4 август 2026 г.
- [8] Microsoft, “Handlers — .NET MAUI.” <https://learn.microsoft.com/dotnet/maui/user-interface/handlers/>. посетен на 4 август 2026 г.
- [9] Google, “Flutter documentation.” <https://docs.flutter.dev/>. посетен на 4 август 2026 г.
- [10] The Qt Company, “Qt documentation.” <https://doc.qt.io/>. посетен на 4 август 2026 г.
- [11] AvaloniaUI, “Avalonia UI documentation.” <https://docs.avaloniaui.net/>. посетен на 4 август 2026 г.
- [12] SixtyFPS GmbH, “Slint documentation.” <https://docs.slint.dev/>. посетен на 10 август 2026 г.
- [13] Meta Platforms, “React Native documentation.” <https://reactnative.dev/docs/getting-started>. посетен на 4 август 2026 г.

- [14] V. Garousi, İ. B. Işık, A. Z. Boyraz, W. Afzal, B. Baydan, B. Yolaçan, A. Çağlar, S. Çaylak, and K. Herkiloğlu, “Comparing automated visual GUI testing tools: an industrial case study,” in *Proceedings of the 8th ACM SIGSOFT International Workshop on Automated Software Testing (A-TEST)*, (Paderborn, Germany), pp. 21–28, 2017.
- [15] R. Jung, J.-H. Jourdan, R. Krebbers, and D. Dreyer, “RustBelt: Securing the foundations of the Rust programming language,” *Proceedings of the ACM on Programming Languages*, vol. 2, no. POPL, pp. 1–34, 2018.
- [16] B. Stroustrup, *The C++ Programming Language*. Addison-Wesley, 4th ed., 2013.
- [17] R. Jones, A. Hosking, and E. Moss, *The Garbage Collection Handbook: The Art of Automatic Memory Management*. Chapman and Hall/CRC, 2011.
- [18] M. Tofte and J.-P. Talpin, “Region-based memory management,” *Information and Computation*, vol. 132, no. 2, pp. 109–176, 1997.
- [19] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994.
- [20] International Organization for Standardization, “ISO/IEC 14882:2024 — programming languages — C++.” ISO, Geneva, Switzerland, 2024.
- [21] M. Fowler, “Presentation model.” <https://martinfowler.com/eaDev/PresentationModel.html>, 2004. публікувана на 19 юлі 2004 г.; посетена на 10 август 2026 г.
- [22] J. Lakos, *Large-Scale C++ Software Design*. Addison-Wesley, 1996.
- [23] K. Beck, *Test-Driven Development: By Example*. Addison-Wesley, 2003.
- [24] K. Serebryany and T. Iskhodzhanov, “ThreadSanitizer: Data race detection in practice,” in *Proceedings of the Workshop on Binary Instrumentation and Applications (WBIA '09)*, pp. 62–71, ACM, 2009.
- [25] E. Stepanov and K. Serebryany, “MemorySanitizer: Fast detector of uninitialized memory use in C++,” in *2015 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*, pp. 46–55, IEEE, 2015.

- [26] K. Serebryany, D. Bruening, A. Potapenko, and D. Vyukov, “AddressSanitizer: A fast address sanity checker,” in *2012 USENIX Annual Technical Conference (USENIX ATC 12)*, pp. 309–318, USENIX Association, 2012.
- [27] E. W. Dijkstra, “Notes on structured programming,” in *Structured Programming* (O.-J. Dahl, E. W. Dijkstra, and C. A. R. Hoare, eds.), pp. 1–82, London: Academic Press, 1972.
- [28] Z. Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli, “Image quality assessment: From error visibility to structural similarity,” *IEEE Transactions on Image Processing*, vol. 13, no. 4, pp. 600–612, 2004.
- [29] G. Sharma, W. Wu, and E. N. Dalal, “The CIEDE2000 color-difference formula: Implementation notes, supplementary test data, and mathematical observations,” *Color Research & Application*, vol. 30, no. 1, pp. 21–30, 2005.
- [30] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. P. Xing, H. Zhang, J. E. Gonzalez, and I. Stoica, “Judging LLM-as-a-Judge with MT-Bench and chatbot arena,” in *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*, vol. 36, 2023.
- [31] N. A. Ernst and M. T. Baldassarre, “Registered reports in software engineering,” *Empirical Software Engineering*, vol. 28, no. 2, p. 55, 2023.
- [32] A. Georges, D. Buytaert, and L. Eeckhout, “Statistically rigorous Java performance evaluation,” in *Proceedings of the 22nd Annual ACM SIGPLAN Conference on Object-Oriented Programming Systems, Languages and Applications (OOPSLA '07)*, pp. 57–76, ACM, 2007.
- [33] J. Dean and L. A. Barroso, “The tail at scale,” *Communications of the ACM*, vol. 56, no. 2, pp. 74–80, 2013.
- [34] Microsoft, “dotnet/maui — .NET MAUI source repository.” <https://github.com/dotnet/maui>. посетен на 4 август 2026 г.
- [35] cppreference.com, “C++ reference.” <https://en.cppreference.com/w/cpp>. посетен на 4 август 2026 г.

- [36] Apple, “UIKit — apple developer documentation.” <https://developer.apple.com/documentation/uikit>. посетен на 4 август 2026 г.
- [37] Google, “Views — android developers.” <https://developer.android.com/reference/android/view/View>. посетен на 4 август 2026 г.
- [38] Microsoft, “WinUI 3 — windows app SDK.” <https://learn.microsoft.com/windows/apps/winui/winui3/>. посетен на 4 август 2026 г.

ПРИЛОЖЕНИЯ

Приложение А. Пълен изходен текст на срез Button

Срезът за бутон в пълен вид — договорът, обработчикът с двете си таблици и платформената реализация за една среда. В Раздел IV са дадени само откъсите, носещи проектните решения (листинги 1 и 2); тук се привежда цялото, за да е проверимо, че между откъса и цялото няма разлика.

Показани са само *относимите* откъси, а не целите файлове: пълните текстове са в хранилището и възпроизвеждането им тук би добавило десетки страници без да добави информация. Всеки откъс се *чете* от истинския файл при компилация (\lstinputlisting с диапазон от редове), тоест не може да се разминее с изходния текст — но и не го дублира.

```
1  class i_button : public i_view, public i_padding, public i_button_stroke
2  {
3  public:
4      // Called by the platform view on touch-down.
5      virtual void send_pressed() = 0;
6      // Called by the platform view on touch-up (inside or outside) and on cancel.
7      virtual void send_released() = 0;
8      // Called by the platform view on a completed tap (touch-up inside).
9      virtual void send_clicked() = 0;
10 };
11 } // namespace maui::core
```

Листинг 8: Договор — i_button: само свойства и събития, без изобразяване

```
1  class button_handler : public view_handler<button_handler, i_button, button_platform>
2  {
3  public:
4      button_handler();
5
6      // Shared mapper tables (cross-platform --- defined in button_handler.cpp). 'mapper'
↪ chains the
7      // text mapper + the image mapper, mirroring ButtonHandler.Mapper chaining
↪ TextButtonMapper +
8      // ImageButtonMapper.
9      static property_mapper<i_button, button_handler>& mapper();
10     static property_mapper<i_text_button, button_handler>& text_mapper();
11     // C# ButtonHandler.ImageButtonMapper --- PropertyMapper<IImage, IButtonHandler> keyed
↪ on the image
12     // source ([Source] → MapImageSource) plus the controls-side ContentLayout remap (
```

```

↪ Button.Mapper.cs's
13     // MapContentLayout). Keyed on i_text_button (Button's virtual view, which carries
↪ image_source()):
14     // the diamond-free narrow seam (Button is not an i_image --- see i_text_button.hpp).
15     static property_mapper<i_text_button, button_handler>& image_mapper();
16     static command_mapper<i_button, button_handler>& command_mapper();
17
18     // Platform recipe (defined per backend: src/platform/<backend>/button_handler.{cpp,mm})
↪ .
19     // create + disconnect need no handler state (static); connect captures 'this' to route
↪ the
20     // native control's events back to the virtual view.
21     static std::unique_ptr<button_platform> create_platform_view();
22     void on_connect_handler(button_platform& platform);
23     static void on_disconnect_handler(button_platform& platform);

```

Листинг 9: Обявяване на обработчика — трите таблици и създаването на native контролата

```

1     property_mapper<i_button, button_handler>& button_handler::mapper()
2     {
3         static property_mapper<i_button, button_handler> table = [] {
4             property_mapper<i_button, button_handler> mapped{
5                 {"padding", &button_handler::map_padding},
6                 {"stroke_color", &button_handler::map_stroke_color},
7                 {"stroke_thickness", &button_handler::map_stroke_thickness},
8                 {"corner_radius", &button_handler::map_corner_radius},
9             };
10            // Reverse-order iteration in keys() means the LAST chained mapper's keys come first
↪ , so listing
11            // image_mapper then text_mapper then view_mapper yields: view (generic IView) keys,
↪ then text
12            // keys, then image keys (C# lists TextButtonMapper, ImageButtonMapper, ViewMapper
↪ --- same set).
13            mapped.set_chained({&image_mapper(), &text_mapper(), &view_mapper()});
14            return mapped;
15        }();
16        return table;
17    }
18
19    // Cross-platform source routing --- image_handler::map_source's twin over the button
↪ platform primitives
20    // (ButtonHandler.MapImageSource → MapImageSourceAsync → ImageSourceLoader.
↪ UpdateImageSourceAsync): the
21    // file fast-path is synchronous; every other source (uri/stream/font) goes through the
↪ handler-owned
22    // loader (async, with the source-identity recheck); a null/empty source cancels + clears.
23    void button_handler::map_image_source(button_handler& handler, i_text_button& view)
24    {

```

```

25     auto* platform = handler.typed_platform_view();
26     if (platform == nullptr)
27     {
28         return;
29     }
30     // Record the current ContentLayout (position + spacing) on the mirror BEFORE the load,
↪ so the
31     // per-backend icon primitive (android apply_button_icon) places the compound drawable
↪ correctly ---
32     // "source" runs before "content_layout" in the mapper table, so the icon primitive can'
↪ t rely on

```

Листинг 10: Преносима реализация — веригата от таблици

```

1     std::unique_ptr<button_platform> button_handler::create_platform_view()
2     {
3         auto platform = std::make_unique<button_platform>();
4         UIButton* const button = [UIButton buttonWithType:UIButtonTypeSystem];
5         set_control_properties_from_proxy(button);
6         platform->native = (__bridge_retained void*)button; // the void* slot owns one reference
7         return platform;
8     }
9
10    void button_handler::on_connect_handler(button_platform& platform)
11    {
12        UIButton* const button = as_button(platform.native);
13        MauiButtonEventProxy* const proxy = [[MauiButtonEventProxy alloc] init];
14        proxy.handler = this;
15        // ButtonEventProxy.Connect --- the four touch events. UIControl holds its targets
↪ weakly, so the
16        // proxy is kept alive for the button's lifetime via an associated object (the AppKit
↪ pattern).
17        [button addTarget:proxy action:@selector(onTouchUpInside:) forControlEvents:
↪ UIControlEventTouchUpInside];
18        [button addTarget:proxy action:@selector(onTouchUpOutside:) forControlEvents:
↪ UIControlEventTouchUpOutside];
19        [button addTarget:proxy action:@selector(onTouchDown:) forControlEvents:
↪ UIControlEventTouchDown];
20        [button addTarget:proxy action:@selector(onTouchCancel:) forControlEvents:
↪ UIControlEventTouchCancel];
21        objc_setAssociatedObject(button, &k_proxy_key, proxy, OBJC_ASSOCIATION_RETAIN_NONATOMIC)
↪ ;
22    }

```

Листинг 11: Платформена реализация (UIKit) — създаване и едно действие

Приложение Б. Публичен програмен интерфейс на системата

Публичните типове по слоеве, съпоставени с типовете на еталона. Съответствието е едно към едно по покритие; имената се преобразуват по правилата на езиковия профил, а всеки заглавен файл носи в коментар пълното име на съответстващия тип.

Списъкът е генериран от първия ред на всеки публичен заглавен файл, който носи съответствието в коментар (tools/gen_tex.py). Обхваща **604** типа.

Таблица 14. Публични типове и съответствието им с типовете на еталона

Слой	Система	Еталон
animations	maui::animations::animation	Microsoft.Maui.Animations. Animation
animations	maui::animations::animation_ manager	Microsoft.Maui.Animations. AnimationManager
animations	maui::animations::easing	Microsoft.Maui.Easing
animations	maui::animations::i_ animation_manager	Microsoft.Maui.Animations. IAnimationManager
animations	maui::animations::i_animator	Microsoft.Maui.Animations. IAnimator
animations	maui::animations::i_ticker	Microsoft.Maui.Animations. ITicker
animations	maui::animations::lerp	Microsoft.Maui.Animations.Lerp
animations	maui::animations::lerping_ animation	Microsoft.Maui.Animations. LerpingAnimation
animations	maui::animations::create_ platform_ticker	Microsoft.Maui.Animations. PlatformTicker (the)
animations	maui::animations::ticker	Microsoft.Maui.Animations.Ticker
controls	maui::controls::absolute_ layout	Microsoft.Maui.Controls.Absolute Layout
controls	maui::controls::activity_ indicator	Microsoft.Maui.Controls.Activity Indicator
controls	maui::controls::animation	Microsoft.Maui.Controls. Animation
controls	maui::controls animation extensions	Microsoft.Maui.Controls. AnimationExtensions
controls	maui::controls::application	Microsoft.Maui. Controls.Application (+ Microsoft.Maui.IApplication)
controls	maui::controls::behavior	Microsoft.Maui.Controls.Behavior
bindings	maui::controls::binding	Microsoft.Maui.Controls.Binding
bindings	maui::controls::binding_base	Microsoft.Maui.Controls.Binding Base

Слой	Система	Еталон
bindings	maui::controls binding diagnostics	Microsoft.Maui.Controls.Xaml. Diagnostics.BindingDiagnostics
bindings	maui::controls::i_multi_ value_converter	Microsoft.Maui.Controls.IMulti ValueConverter
bindings	maui::controls::i_value_ converter	Microsoft.Maui.Controls.IValue Converter
bindings	maui::controls::multi_binding	Microsoft.Maui.Controls.Multi Binding
bindings	maui::controls::relative_ binding_source	Microsoft.Maui.Controls.Relative BindingSource
controls	maui::controls::border	Microsoft.Maui.Controls.Border
controls	maui::controls::box_view	Microsoft.Maui.Controls.BoxView
brushes	maui::controls::brush	Microsoft.Maui.Controls.Brush
brushes	maui::controls::convert_brush	Microsoft.Maui.Controls.Brush TypeConverter (string → Brush.)
brushes	maui::controls::gradient_ brush	Microsoft.Maui.Controls.Gradient Brush (+ GradientStopCollection)
brushes	maui::controls::gradient_stop	Microsoft.Maui.Controls.Gradient Stop
brushes	maui::controls::image_brush	Microsoft.Maui.Controls.Image Brush (internal)
brushes	maui::controls::immutable_ brush	Microsoft.Maui.Controls. ImmutableBrush (internal)
brushes	maui::controls::linear_ gradient_brush	Microsoft.Maui.Controls.Linear GradientBrush
brushes	maui::controls::radial_ gradient_brush	Microsoft.Maui.Controls.Radial GradientBrush
brushes	maui::controls::solid_color_ brush	Microsoft.Maui.Controls.Solid ColorBrush
controls	maui::controls::button	Microsoft.Maui.Controls.Button (M2 subset)
controls	maui::controls::button_ content_layout	Microsoft.Maui.Controls.Button. ButtonContentLayout
cells	maui::controls::cell	Microsoft.Maui.Controls.Cell
cells	maui::controls::entry_cell	Microsoft.Maui.Controls.Entry Cell
cells	maui::controls::image_cell	Microsoft.Maui.Controls.Image Cell
cells	maui::controls::switch_cell	Microsoft.Maui.Controls.Switch Cell
cells	maui::controls::text_cell	Microsoft.Maui.Controls.TextCell
cells	maui::controls::view_cell	Microsoft.Maui.Controls.ViewCell

Слой	Система	Еталон
controls	maui::controls::check_box	Microsoft.Maui.Controls.CheckBox
controls	maui::controls::column_definition	Microsoft.Maui.Controls.ColumnDefinition
controls	maui::controls::command	Microsoft.Maui.Controls.Command
controls	maui::controls::content_page	Microsoft.Maui.Controls.ContentPage
controls	maui::controls::content_view	Microsoft.Maui.Controls.ContentView
controls	maui::controls::data_package_operation	Microsoft.Maui.Controls.DataPackageOperation
controls	maui::controls::date_picker	Microsoft.Maui.Controls.DatePicker
controls	maui::controls::dynamic_resource	Microsoft.Maui.Controls.Internals.DynamicResource
controls	maui::controls::editor	Microsoft.Maui.Controls.Editor
controls	maui::controls::editor_auto_size_option	Microsoft.Maui.Controls.EditorAutoSizeOption
controls	maui::controls::effect	Microsoft.Maui.Controls.Effect
controls	maui::controls::effect_collection	the Element.Effects TrackableCollection<Effect> + its
controls	maui::controls effect registry	Microsoft.Maui.Controls.Internals.Registrar.Effects
controls	maui::controls::element	Microsoft.Maui.Controls.Element / VisualElement (lifecycle subset)
controls	maui::controls::entry	Microsoft.Maui.Controls.Entry
controls	maui::controls::event_trigger	Microsoft.Maui.Controls.EventTrigger
controls	maui::controls::file_image_source	Microsoft.Maui.Controls.FileImageSource
controls	maui::controls::flex_layout	Microsoft.Maui.Controls.FlexLayout
controls	maui::controls::flyout_base	Microsoft.Maui.Controls.FlyoutBase
controls	maui::controls::flyout_layout_behavior	Microsoft.Maui.Controls.FlyoutLayoutBehavior
controls	maui::controls::flyout_page	Microsoft.Maui.Controls.FlyoutPage
controls	maui::controls::font_image_source	Microsoft.Maui.Controls.FontImageSource
controls	maui::controls::formatted_string	Microsoft.Maui.Controls.FormattedString
controls	maui::controls::frame	Microsoft.Maui.Controls.Frame

Слой	Система	Еталон
gestures	maui::controls::buttons_mask	Microsoft.Maui.Controls.ButtonsMask ([Flags])
gestures	maui::controls::drag_gesture_recognizer	Microsoft.Maui.Controls.DragGestureRecognizer
gestures	maui::controls::drop_gesture_recognizer	Microsoft.Maui.Controls.DropGestureRecognizer
gestures	maui::controls::gesture_platform_manager	Microsoft.Maui.Controls.Platform.GestureRecognizer +
gestures	maui::controls::gesture_recognizer	Microsoft.Maui.Controls.GestureRecognizer
gestures	maui::controls::gesture_recognizer_collection	the ObservableCollection<IGestureRecognizer>
gestures	maui::controls::pan_gesture_recognizer	Microsoft.Maui.Controls.PanGestureRecognizer
gestures	maui::controls::pinch_gesture_recognizer	Microsoft.Maui.Controls.PinchGestureRecognizer
gestures	maui::controls::pointer_gesture_recognizer	Microsoft.Maui.Controls.PointerGestureRecognizer
gestures	maui::controls::swipe_gesture_recognizer	Microsoft.Maui.Controls.SwipeGestureRecognizer
gestures	maui::controls::tap_gesture_recognizer	Microsoft.Maui.Controls.TapGestureRecognizer
controls	maui::controls::graphics_view	Microsoft.Maui.Controls.GraphicsView (+ TouchEventArgs)
controls	maui::controls::grid	Microsoft.Maui.Controls.Grid
controls	maui::controls::horizontal_stack_layout	Microsoft.Maui.Controls.HorizontalStackLayout
controls	maui::controls::html_web_view_source	Microsoft.Maui.Controls.HtmlWebViewSource
controls	maui::controls::hybrid_web_view	Microsoft.Maui.Controls.HybridWebView
controls	maui::controls::hybrid_web_view_handler	Microsoft.Maui.Handlers.HybridWebViewHandler
controls	maui::controls::i_command	System.Windows.Input.ICommand
controls	maui::controls::i_effect_control_provider	Microsoft.Maui.Controls.IEffectControlProvider
controls	maui::controls::i_table_view	Microsoft.Maui.Controls.ITableViewController (the handler-facing)
controls	maui::controls::image	Microsoft.Maui.Controls.Image
controls	maui::controls::image_button	Microsoft.Maui.Controls.ImageButton

Слой	Система	Еталон
controls	maui::controls::indicator_shape	Microsoft.Maui.Controls.IndicatorShape
controls	maui::controls::indicator_view	Microsoft.Maui.Controls.IndicatorView
items	maui::controls::boxed_item	the C# 'object' item of the Items layer (ItemsSource elements,
items	maui::controls::carousel_view	Microsoft.Maui.Controls.CarouselView
items	maui::controls::collection_view	Microsoft.Maui.Controls.CollectionView
items	maui::controls::collection_view_handler	Microsoft.Maui.Controls.Handlers.Items.
items	maui::controls::grid_items_layout	Microsoft.Maui.Controls.GridItemsLayout
items	maui::controls::groupable_items_view	Microsoft.Maui.Controls.GroupableItemsView
items	maui::controls::i_items_view	the virtual-view face of Microsoft.Maui.Controls.ItemsView
items	maui::controls::i_item_collection	the C# 'IEnumerable ItemsSource' face (+ IList fast paths)
items	maui::controls::item_sizing_strategy	Microsoft.Maui.Controls.ItemSizingStrategy
items	maui::controls::items_layout	Microsoft.Maui.Controls.ItemsLayout (+ ItemsLayout)
items	maui::controls::items_layout_orientation	Microsoft.Maui.Controls.ItemsLayoutOrientation
items	maui::controls::items_source_factory	Microsoft.Maui.Controls.Handlers.Items.ItemsSourceFactory
items	maui::controls::items_updating_scroll_mode	Microsoft.Maui.Controls.ItemsUpdatingScrollMode
items	maui::controls::items_view	Microsoft.Maui.Controls.ItemsView
items	maui::controls::items_view_scrolled_event_args	Microsoft.Maui.Controls.ItemsViewScrolledEventArgs
items	maui::controls::i_items_view_source	Microsoft.Maui.Controls.Handlers.Items.IItemsViewSource
items	maui::controls::linear_items_layout	Microsoft.Maui.Controls.LinearItemsLayout
items	maui::controls::reorderable_items_view	Microsoft.Maui.Controls.ReorderableItemsView
items	maui::controls::scroll_to_request_event_args	Microsoft.Maui.Controls.ScrollToRequestEventArgs

Слой	Система	Еталон
items	maui::controls::selectable_items_view	Microsoft.Maui.Controls.SelectableItemsView
items	maui::controls::selection_changed_event_args	Microsoft.Maui.Controls.SelectionChangedEventArgs
items	maui::controls::selection_list	Microsoft.Maui.Controls.SelectionList (internal)
items	maui::controls::selection_mode	Microsoft.Maui.Controls.SelectionMode
items	maui::controls::snap_points_alignment	Microsoft.Maui.Controls.SnapPointsAlignment
items	maui::controls::snap_points_type	Microsoft.Maui.Controls.SnapPointsType
items	maui::controls::structured_items_view	Microsoft.Maui.Controls.StructuredItemsView
controls	maui::controls::label	Microsoft.Maui.Controls.Label
controls	maui::controls::layout<Layout Interface>	Microsoft.Maui.Controls.Layout
controls	maui::controls::menu_bar	Microsoft.Maui.Controls.MenuBar
controls	maui::controls::menu_bar_item	Microsoft.Maui.Controls.MenuBarItem
controls	maui::controls::menu_bar_tracker	Microsoft.Maui.Controls.MenuBarTracker
controls	maui::controls::menu_element_list<TItem>	the IList<T> + handler-notification shape shared by
controls	maui::controls::menu_flyout	Microsoft.Maui.Controls.MenuFlyout
controls	maui::controls::menu_flyout_item	Microsoft.Maui.Controls.MenuFlyoutItem
controls	maui::controls::menu_flyout_separator	Microsoft.Maui.Controls.MenuFlyoutSeparator
controls	maui::controls::menu_flyout_sub_item	Microsoft.Maui.Controls.MenuFlyoutSubItem
controls	maui::controls::menu_item	Microsoft.Maui.Controls.MenuItem (over BaseMenuItem : StyleableElement)
controls	maui::controls::menu_item_tracker<TItem>	Microsoft.Maui.Controls.MenuItemTracker<TMenuItem>
controls	maui::controls::merged_style	Microsoft.Maui.Controls.MergedStyle
controls	maui::controls::modal_event_args	Microsoft.Maui.Controls.ModalEventArgs
controls	maui::controls::multi_page<TPage>	Microsoft.Maui.Controls.MultiPage<T>

Слой	Система	Еталон
controls	maui::controls::navigation_ page	Microsoft.Maui.Controls. NavigationPage
controls	maui::controls::observable_ collection<T>	System.Collections.ObjectModel. ObservableCollection<T>
controls	maui::controls::picker	Microsoft.Maui.Controls.Picker
platform_ configuration	maui::controls::platform_ configuration	Microsoft.Maui.Controls.Platform Configuration
controls	maui::controls::progress_bar	Microsoft.Maui.Controls.Progress Bar
controls	maui::controls::radio_button	Microsoft.Maui.Controls.Radio Button
controls	maui::controls::radio_button_ group	Microsoft.Maui.Controls.Radio ButtonGroup
controls	maui::controls::refresh_view	Microsoft.Maui.Controls.Refresh View
controls	maui::controls::resource_ dictionary	Microsoft.Maui.Controls.Resource Dictionary
controls	maui::controls::routing_ effect	Microsoft.Maui.Controls.Routing Effect
controls	maui::controls::row_ definition	Microsoft.Maui.Controls.Row Definition
controls	maui::controls::scroll_to_ position	Microsoft.Maui.Controls.ScrollTo Position
controls	maui::controls::scroll_view	Microsoft.Maui.Controls.Scroll View
controls	maui::controls::search_bar	Microsoft.Maui.Controls.Search Bar
controls	maui::controls::setter	Microsoft.Maui.Controls.Setter
shapes	maui::controls::shapes:: composite_transform	Microsoft.Maui.Controls.Shapes. CompositeTransform
shapes	maui::controls::shapes:: ellipse	Microsoft.Maui.Controls.Shapes. Ellipse
shapes	maui::controls::shapes:: ellipse_geometry	Microsoft.Maui.Controls.Shapes. EllipseGeometry
shapes	maui::controls::shapes:: fill_rule	Microsoft.Maui.Controls.Shapes. FillRule
shapes	maui::controls::shapes:: geometry	Microsoft.Maui.Controls.Shapes. Geometry (+ IGeometry.cs)
shapes	maui::controls::shapes:: geometry_group	Microsoft.Maui.Controls.Shapes. GeometryGroup
shapes	maui::controls::shapes:: geometry_helper	Microsoft.Maui.Controls.Shapes. GeometryHelper

Слой	Система	Еталон
shapes	maui::controls::shapes::line	Microsoft.Maui.Controls.Shapes.Line
shapes	maui::controls::shapes::line_geometry	Microsoft.Maui.Controls.Shapes.LineGeometry
shapes	maui::controls::shapes::matrix	Microsoft.Maui.Controls.Shapes.Matrix
shapes	maui::controls::shapes::matrix_transform	Microsoft.Maui.Controls.Shapes.MatrixTransform
shapes	maui::controls::shapes::path	Microsoft.Maui.Controls.Shapes.Path
shapes	maui::controls::shapes::path_figure	Microsoft.Maui.Controls.Shapes.PathFigure
shapes	maui::controls::shapes::path_geometry	Microsoft.Maui.Controls.Shapes.PathGeometry
shapes	maui::controls::shapes - the path markup / point list string parsers	
shapes	maui::controls::shapes::path_segment (+ the seven concrete segments)	
shapes	maui::controls::shapes::polygon	Microsoft.Maui.Controls.Shapes.Polygon
shapes	maui::controls::shapes::polyline	Microsoft.Maui.Controls.Shapes.Polyline
shapes	maui::controls::shapes::rectangle	Microsoft.Maui.Controls.Shapes.Rectangle
shapes	maui::controls::shapes::rectangle_geometry	Microsoft.Maui.Controls.Shapes.RectangleGeometry
shapes	maui::controls::shapes::rotate_transform	Microsoft.Maui.Controls.Shapes.RotateTransform
shapes	maui::controls::shapes::round_rectangle	Microsoft.Maui.Controls.Shapes.RoundRectangle
shapes	maui::controls::shapes::round_rectangle_geometry	
shapes	maui::controls::shapes::scale_transform	Microsoft.Maui.Controls.Shapes.ScaleTransform
shapes	maui::controls::shapes::shape	Microsoft.Maui.Controls.Shapes.Shape
shapes	maui::controls::shapes::skew_transform	Microsoft.Maui.Controls.Shapes.SkewTransform
shapes	maui::controls::shapes::sweep_direction	Microsoft.Maui.Controls.SweepDirection

Слой	Система	Еталон
shapes	maui::controls::shapes::transform	Microsoft.Maui.Controls.Shapes.Transform
shapes	maui::controls::shapes::transform_group	Microsoft.Maui.Controls.Shapes.TransformGroup
shapes	maui::controls::shapes::translate_transform	Microsoft.Maui.Controls.Shapes.TranslateTransform
shell	maui::controls::base_shell_item	Microsoft.Maui.Controls.BaseShellItem
shell	maui::controls::flyout_behavior	Microsoft.Maui.FlyoutBehavior
shell	maui::controls::flyout_display_options	Microsoft.Maui.Controls.FlyoutDisplayOptions
shell	maui::controls::flyout_header_behavior	Microsoft.Maui.Controls.FlyoutHeaderBehavior
shell	maui::controls::i_query_attributable	Microsoft.Maui.Controls.IQueryAttributable
shell	maui::controls::list_proxy	Microsoft.Maui.Controls.ListProxy (IListProxy / IReadOnlyList<object>)
shell	maui::controls::request_definition	Microsoft.Maui.Controls.RequestDefinition (internal)
shell	maui::controls::route_request_builder	Microsoft.Maui.Controls.RouteRequestBuilder (internal)
shell	maui::controls::routing	Microsoft.Maui.Controls.Routing (+ RouteFactory)
shell	maui::controls::search_box_visibility	Microsoft.Maui.Controls.SearchBoxVisibility
shell	maui::controls::search_handler	Microsoft.Maui.Controls.SearchHandler (+ ISearchHandlerController)
shell	maui::controls::shell	Microsoft.Maui.Controls.Shell (model/ownership/navigation surface)
shell	maui::controls::shell_appearance	Microsoft.Maui.Controls.ShellAppearance
shell	maui::controls::shell_content	Microsoft.Maui.Controls.ShellContent
shell	maui::controls::shell_group_item	Microsoft.Maui.Controls.ShellGroupItem
shell	maui::controls::shell_item (+ flyout_item, tab_bar)	Microsoft.Maui.Controls.ShellItem /
shell	maui::controls::shell_navigated_event_args	Microsoft.Maui.Controls.ShellNavigatedEventArgs

Слой	Система	Еталон
shell	maui::controls::shell_navigating_deferral	Microsoft.Maui.Controls.Shell NavigatingDeferral
shell	maui::controls::shell_navigating_event_args	Microsoft.Maui.Controls.Shell NavigatingEventArgs
shell	maui::controls::shell_navigation_manager	Microsoft.Maui.Controls.Shell NavigationManager
shell	maui::controls::shell_navigation_parameters	Microsoft.Maui.Controls.Shell NavigationParameters
shell	maui::controls::shell_navigation_request	Microsoft.Maui.Controls.Shell NavigationRequest
shell	maui::controls::shell_navigation_source	Microsoft.Maui.Controls.Shell NavigationSource
shell	maui::controls::shell_navigation_state	Microsoft.Maui.Controls.Shell NavigationState
shell	maui::controls::shell_route_parameters	Microsoft.Maui.Controls.Shell RouteParameters
shell	maui::controls::shell_section (+ tab)	Microsoft.Maui.Controls.ShellSection / Tab
shell	maui::controls::shell_uri	the System.Uri subset Shell routing relies on
shell	maui::controls::shell_uri_handler	Microsoft.Maui.Controls.ShellUriHandler (internal)
controls	maui::core::shell_handler	Microsoft.Maui.Controls.Handlers.ShellHandler /
controls	maui::controls::slider	Microsoft.Maui.Controls.Slider
controls	maui::controls::span	Microsoft.Maui.Controls.Span
controls	maui::controls::stack_layout	Microsoft.Maui.Controls.StackLayout
controls	maui::controls::stack_layout_manager	Microsoft.Maui.Controls.StackLayoutManager
controls	maui::controls::stack_orientation	Microsoft.Maui.Controls.StackOrientation
controls	maui::controls::state_trigger_base	Microsoft.Maui.Controls.StateTriggerBase
controls	maui::controls::stepper	Microsoft.Maui.Controls.Stepper
controls	maui::controls::stream_image_source	Microsoft.Maui.Controls.StreamImageSource
controls	maui::controls::style	Microsoft.Maui.Controls.Style
controls	maui::controls::swipe_item	Microsoft.Maui.Controls.SwipeItem
controls	maui::controls::swipe_item_view	Microsoft.Maui.Controls.SwipeItemView

Слой	Система	Еталон
controls	maui::controls::swipe_items	Microsoft.Maui.Controls.Swipe Items
controls	maui::controls::swipe_view	Microsoft.Maui.Controls.Swipe View
controls	maui::controls::tabbed_page	Microsoft.Maui.Controls.Tabbed Page
controls	maui::controls::table_intent	Microsoft.Maui.Controls.Table Intent
controls	maui::controls::table_model	Microsoft.Maui.Controls. Internals.TableModel (ITableModel)
controls	maui::controls::table_root	Microsoft.Maui.Controls.Table Root
controls	maui::controls::table_section	Microsoft.Maui.Controls.Table Section
controls	maui::controls::table_ section_base	Microsoft.Maui.Controls.Table SectionBase
controls	maui::controls::table_view	Microsoft.Maui.Controls.Table View
controls	maui::controls::table_view_ handler	Microsoft.Maui.Controls.Handlers. Compatibility.TableViewRenderer
templates	maui::controls::content_ presenter	Microsoft.Maui.Controls.Content Presenter
templates	maui::controls::control_ template	Microsoft.Maui.Controls.Control Template
templates	maui::controls::data_template	Microsoft.Maui.Controls.Data Template (+ IDataTemplateController)
templates	maui::controls::data_ template_selector	Microsoft.Maui.Controls.Data TemplateSelector
templates	maui::controls::element_ template	Microsoft.Maui.Controls.Element Template
templates	maui::controls::i_control_ templated	Microsoft.Maui.Controls.IControl Templated (internal)
templates	maui::controls::template_ binding	Microsoft.Maui.Controls.Template Binding
templates	maui::controls::template_ utilities	Microsoft.Maui.Controls.Template Utilities (internal)
templates	maui::controls::templated_ page	Microsoft.Maui.Controls. TemplatedPage
templates	maui::controls::templated_ view	Microsoft.Maui.Controls. TemplatedView
controls	maui::controls::time_picker	Microsoft.Maui.Controls.Time Picker

Слой	Система	Еталон
controls	maui::controls::title_bar	Microsoft.Maui.Controls.TitleBar (the basics)
controls	maui::controls::toggle_switch	Microsoft.Maui.Controls.Switch
controls	maui::controls::tool_tip_properties	Microsoft.Maui.Controls.ToolTip Properties
controls	maui::controls::toolbar	Microsoft.Maui.Controls.Toolbar (+ the NavigationPageToolbar aggregate)
controls	maui::controls::toolbar_item	Microsoft.Maui.Controls.Toolbar Item
controls	maui::controls::toolbar_item_order	Microsoft.Maui.Controls.Toolbar ItemOrder
controls	maui::controls::toolbar_tracker	Microsoft.Maui.Controls.Toolbar Tracker
controls	maui::controls::uri_image_source	Microsoft.Maui.Controls.UriImage Source
controls	maui::controls::url_web_view_source	Microsoft.Maui.Controls.UrlWebViewSource
controls	maui::controls::vertical_stack_layout	Microsoft.Maui.Controls.Vertical StackLayout
controls	maui::controls::view<View Interface>	Microsoft.Maui.Controls.View / Visual Element (minimal M2)
controls	maui::controls view extensions	Microsoft.Maui.Controls.View Extensions (the animation surface)
controls	maui::controls::visual_state_manager	Microsoft.Maui.Controls.Visual StateManager
controls	maui::controls::web_navigation_event_args	Microsoft.Maui.Controls.Web NavigationEventArgs
controls	maui::controls::web_view	Microsoft.Maui.Controls.WebView
controls	maui::controls::web_view_source	Microsoft.Maui.Controls.WebView Source
controls	maui::controls::window	Microsoft.Maui.Controls.Window (+ Microsoft.Maui.IWindow lifecycle)
controls	maui::controls::window_overlay	Microsoft.Maui.WindowOverlay
core	maui::core::activity_indicator_handler	Microsoft.Maui.Handlers.Activity IndicatorHandler
core	maui::core::app_theme	Microsoft.Maui.ApplicationModel. AppTheme
core	maui::core::aspect	Microsoft.Maui.Aspect
core	maui::core::bindable_object	Microsoft.Maui.Controls.Bindable Object

Слой	Система	Еталон
core	maui::core::bindable_ property<T>	Microsoft.Maui.Controls.Bindable Property
core	maui::core::bind	Microsoft.Maui.Controls.Binding / BindingExpression (the typed-accessor port)
core	maui::core::binding_ expression	Microsoft.Maui.Controls.Binding Expression (internal)
core	maui::core::binding_mode	Microsoft.Maui.Controls.Binding Mode
core	maui::core::border_handler	Microsoft.Maui.Handlers.Border Handler
core	maui::core boxed-value helpers	Microsoft.Maui.Controls.Binding ExpressionHelper (TryConvert) +)
core	maui::core::button_handler	Microsoft.Maui.Handlers.Button Handler
core	maui::core::cancellation_ token	System.Threading.CancellationToken / CancellationTokenSource
core	maui::core::check_box_handler	Microsoft.Maui.Handlers.CheckBox Handler
core	maui::core::clear_button_ visibility	Microsoft.Maui.ClearButton Visibility
core	maui::core::command_mapper	Microsoft.Maui.CommandMapper / ICommandMapper
core	maui::core::content_page_ handler	Microsoft.Maui.Handlers.Content ViewHandler
core	maui::core::crc64_hash_string	Microsoft.Maui.Crc64 / Crc64Hash Algorithm
core	maui::core::date_picker_ handler	Microsoft.Maui.Handlers.Date PickerHandler
core	maui::core::date_time / maui:: core::time_span	System.DateTime / System.TimeSpan
core	maui::core::dimension	Microsoft.Maui.Primitives. Dimension
core	maui::core::i_dispatcher	Microsoft.Maui.Dispatching. IDispatcher
core	maui::core::editor_handler	Microsoft.Maui.Handlers.Editor Handler
core	maui::core::entry_handler	Microsoft.Maui.Handlers.Entry Handler
core	maui::core::evaluate_java_ script_request	Microsoft.Maui.EvaluateJava ScriptAsyncRequest

Слой	Система	Еталон
core	maui::core::event<Args...>	the C# 'event EventHandler<TArgs>' / multicast-delegate pattern
core	maui::core::file_image_source_service	Microsoft.Maui.FileImageSource Service
core	maui::core::flow_direction	Microsoft.Maui.FlowDirection
core	maui::core::flyout_behavior	Microsoft.Maui.FlyoutBehavior
core	maui::core::flyout_page_handler	Microsoft.Maui.Handlers.Flyout ViewHandler
core	maui::core::focus_request	Microsoft.Maui.FocusRequest (: RetrievePlatformValueRequest<bool>)
core	maui::core::font	Microsoft.Maui.Font (+ FontWeight, FontSlant)
core	maui::core::font_attributes	Microsoft.Maui.Controls.Font Attributes
core	maui::core::font_file	Microsoft.Maui.FontFile
core	maui::core::font_image_source_service	Microsoft.Maui.FontImageSource Service
core	maui::core::font_manager	Microsoft.Maui.Font Manager (FontManager.iOS.cs / FontManager.Standard.cs)
core	maui::core::font_registrar	Microsoft.Maui.FontRegistrar
core	maui::core::gcd_dispatcher	Microsoft.Maui.Dispatching. Dispatcher (Dispatcher.iOS.cs)
core	maui::core::gesture_status	Microsoft.Maui.GestureStatus
core	maui::core::graphics_view_handler	Microsoft.Maui.Handlers.Graphics ViewHandler
core	maui::core::grid_length	Microsoft.Maui.GridLength
core	maui::core::grid_unit_type	Microsoft.Maui.GridUnitType
core	maui::core::hybrid_web_view_protocol	Microsoft.Maui.Handlers.Hybrid WebViewHelper.ProcessRawMessage
core	maui::core::i_absolute_layout	Microsoft.Maui.IAbsoluteLayout
core	maui::core::i_activity_indicator	Microsoft.Maui.IActivity Indicator
core	maui::core::i_adorner	Microsoft.Maui.IAdorner
core	maui::core::i_application	Microsoft.Maui.IApplication (: IElement)
core	maui::core::i_border_stroke	Microsoft.Maui.IBorderStroke
core	maui::core::i_border_view	Microsoft.Maui.IBorderView
core	maui::core::i_button	Microsoft.Maui.IButton
core	maui::core::i_button_stroke	Microsoft.Maui.IButtonStroke
core	maui::core::i_check_box	Microsoft.Maui.ICheckBox

Слой	Система	Еталон
core	maui::core::i_container	Microsoft.Maui.IContainer (IContainer : IList<IView>)
core	maui::core::i_content_view	Microsoft.Maui.IContentView
core	maui::core::i_context_flyout_ element	Microsoft.Maui.IContextFlyout Element
core	maui::core::i_cross_platform_ layout	Microsoft.Maui.ICrossPlatform Layout
core	maui::core::i_date_picker	Microsoft.Maui.IDatePicker
core	maui::core::i_editor	Microsoft.Maui.IEditor
core	maui::core::i_element	Microsoft.Maui.IElement
core	maui::core::i_element_handler	Microsoft.Maui.IElementHandler
core	maui::core::i_entry	Microsoft.Maui.IEntry
core	maui::core::i_flex_layout	Microsoft.Maui.IFlexLayout
core	maui::core::i_flyout	Microsoft.Maui.IFlyout
core	maui::core::i_flyout_view	Microsoft.Maui.IFlyoutView
core	maui::core::i_font_image_ source	Microsoft.Maui.IFontImageSource
core	maui::core::i_font_manager	Microsoft.Maui.IFontManager (+ IFontManager.iOS DefaultFont / GetFont)
core	maui::core::i_font_registrar	Microsoft.Maui.IFontRegistrar
core	maui::core::i_graphics_view	Microsoft.Maui.IGraphicsView
core	maui::core::i_grid_column_ definition	Microsoft.Maui.IGridColumn Definition
core	maui::core::i_grid_layout	Microsoft.Maui.IGridLayout
core	maui::core::i_grid_row_ definition	Microsoft.Maui.IGridRow Definition
core	maui::core::i_hybrid_web_view	Microsoft.Maui.IHybridWebView
core	maui::core::i_image	Microsoft.Maui.IImage (: IView, IImageSourcePart)
core	maui::core::i_image_button	Microsoft.Maui.IImageButton
core	maui::core::i_image_source	Microsoft.Maui.IImageSource
core	maui::core::i_image_source_ service	Microsoft.Maui.IImageSource Service
core	maui::core::i_indexable	the C# indexer property ('this[...]') + DefaultMemberAttribute, as a
core	maui::core::i_indicator_view	Microsoft.Maui.IIndicatorView
core	maui::core::i_initialization_ aware_web_view	Microsoft.Maui.IInitialization AwareWebView
core	maui::core::i_ios_entry_ specifics	(port seam) Microsoft.Maui.Controls. Entry's iOS remap surface
core	maui::core::i_ios_page_ specifics	Microsoft.Maui.Platform.IiOSPage Specifics

Слой	Система	Еталон
core	maui::core::i_ios_slider_specifics	(port seam) Microsoft.Maui.Controls.Slider's iOS remap surface
core	maui::core::i_item_delegate<T>	Microsoft.Maui.IItemDelegate<T>
core	maui::core::i_label	Microsoft.Maui.ILabel
core	maui::core::i_layout	Microsoft.Maui.ILayout
core	maui::core::i_layout_handler	Microsoft.Maui.ILayoutHandler
core	maui::core::i_maui_context	Microsoft.Maui.IMauiContext
core	maui::core::i_menu_bar	Microsoft.Maui.IMenuBar
core	maui::core::i_menu_bar_element	Microsoft.Maui.IMenuBarElement
core	maui::core::i_menu_bar_item	Microsoft.Maui.IMenuBarItem
core	maui::core::i_menu_element	Microsoft.Maui.IMenuElement
core	maui::core::i_menu_flyout	Microsoft.Maui.IMenuFlyout
core	maui::core::i_menu_flyout_item	Microsoft.Maui.IMenuFlyoutItem
core	maui::core::i_menu_flyout_separator	Microsoft.Maui.IMenuFlyoutSeparator
core	maui::core::i_menu_flyout_sub_item	Microsoft.Maui.IMenuFlyoutSubItem
core	maui::core::i_padding	Microsoft.Maui.IPadding
core	maui::core::i_picker	Microsoft.Maui.IPicker
core	maui::core::i_progress	Microsoft.Maui.IProgress
core	maui::core::i_radio_button	Microsoft.Maui.IRadioButton
core	maui::core::i_range	Microsoft.Maui.IRange
core	maui::core::i_refresh_view	Microsoft.Maui.IRefreshView
core	maui::core::i_safe_area_view	Microsoft.Maui.ISafeAreaView
core	maui::core::i_scroll_view	Microsoft.Maui.IScrollView
core	maui::core::i_search_bar	Microsoft.Maui.ISearchBar
core	maui::core::i_shadow	Microsoft.Maui.IShadow
core	maui::core::i_shape_view	Microsoft.Maui.IShapeView
core	maui::core::i_slider	Microsoft.Maui.ISlider
core	maui::core::i_stack_layout	Microsoft.Maui.IStackLayout
core	maui::core::i_stack_navigation	Microsoft.Maui.IStackNavigation
core	maui::core::i_stepper	Microsoft.Maui.IStepper
core	maui::core::i_stream_image_source	Microsoft.Maui.IStreamImageSource
core	maui::core::i_stroke	Microsoft.Maui.IStroke
core	maui::core::i_swipe_item	Microsoft.Maui.ISwipeItem
core	maui::core::i_swipe_item_menu_item	Microsoft.Maui.ISwipeItemMenuItem

Слой	Система	Еталон
core	maui::core::i_swipe_item_view	Microsoft.Maui.ISwipeItemView
core	maui::core::i_swipe_items	Microsoft.Maui.ISwipeItems
core	maui::core::i_swipe_view	Microsoft.Maui.ISwipeView
core	maui::core::i_switch	Microsoft.Maui.ISwitch
core	maui::core::i_tabbed_view	Microsoft.Maui.ITabbedView
core	maui::core::i_text	Microsoft.Maui.IText
core	maui::core::i_text_alignment	Microsoft.Maui.ITextAlignment
core	maui::core::i_text_button	Microsoft.Maui.ITextButton
core	maui::core::i_text_input	Microsoft.Maui.ITextInput
core	maui::core::i_text_style	Microsoft.Maui.ITextStyle
core	maui::core::i_time_picker	Microsoft.Maui.ITimePicker
core	maui::core::i_title_bar	Microsoft.Maui.ITitleBar (the basics)
core	maui::core::i_title_bar_element	the TitleBar slot of Microsoft.Maui.IWindow
core	maui::core::i_tool_tip_element	Microsoft.Maui.IToolTipElement
core	maui::core::i_toolbar	Microsoft.Maui.IToolbar
core	maui::core::i_toolbar_element	Microsoft.Maui.IToolbarElement
core	maui::core::i_toolbar_item	the toolbar-item face of Microsoft.Maui.Controls.ToolbarItem
core	maui::core::i_transform	Microsoft.Maui.ITransform
core	maui::core::i_uri_image_source	Microsoft.Maui.IUriImageSource
core	maui::core::i_view	Microsoft.Maui.IView
core	maui::core::i_view_handler	Microsoft.Maui.IViewHandler
core	maui::core::i_web_request_intercepting_web_view	Microsoft.Maui.IWebRequestInterceptingWebView
core	maui::core::i_web_view	Microsoft.Maui.IWebView
core	maui::core::i_web_view_delegate	Microsoft.Maui.IWebViewDelegate
core	maui::core::i_web_view_source	Microsoft.Maui.IWebViewSource
core	maui::core::i_window	Microsoft.Maui.IWindow (:ITitledElement : IElement)
core	maui::core::i_window_overlay	Microsoft.Maui.IWindowOverlay
core	maui::core::i_window_overlay_element	Microsoft.Maui.IWindowOverlayElement
core	maui::core::image_button_handler	Microsoft.Maui.Handlers.ImageButtonHandler
core	maui::core::image_handler	Microsoft.Maui.Handlers.ImageHandler (aspect + sources)

Слой	Система	Еталон
core	maui::core::image_source_loader	Microsoft.Maui.Platform.ImageSourcePartLoader +
core	maui::core::image_source_paint	Microsoft.Maui.ImageSourcePaint
core	maui::core::image_source_result	Microsoft.Maui.IImageSourceServiceResult<T> / ImageSourceServiceResult
core	maui::core::image_source_service_provider	Microsoft.Maui.IImageSourceServiceProvider +
core	maui::core::image_source_service_registry	Microsoft.Maui.Hosting.ImageSourceServiceProvider +
core	maui::core::indicator_view_handler	Microsoft.Maui.Handlers.IndicatorViewHandler
core	maui::core::invoke_java_script_request	Microsoft.Maui.HybridWebViewInvokeJavaScriptRequest
core	maui::core::keyboard	Microsoft.Maui.Keyboard
core	maui::core::keyboard_accelerator	Microsoft.Maui.IKeyboardAccelerator (+ Controls.KeyboardAccelerator)
core	maui::core::keyboard_flags	Microsoft.Maui.KeyboardFlags
core	maui::core::label_handler	Microsoft.Maui.Handlers.LabelHandler
core	maui::core::label_run	the per-span result of Span.ToNSAttributedString
core	maui::core::layout_alignment	Microsoft.Maui.Primitives.LayoutAlignment
core	maui::core::layout_handler	Microsoft.Maui.Handlers.LayoutHandler
core	maui::core::order_by_z_index / get_layout_handler_index	Microsoft.Maui.Handlers.LayoutExtensions
core	maui::core::line_break_mode	Microsoft.Maui.LineBreakMode
core	maui::core::navigation_page_handler	Microsoft.Maui.Handlers.NavigationViewHandler
core	maui::core::navigation_request	Microsoft.Maui.NavigationRequest
core	maui::core::observable_collection<T>	System.Collections.ObjectModel.ObservableCollection<T>
core	maui::core::open_swipe_item	Microsoft.Maui.OpenSwipeItem
core	maui::core::path_aspect	Microsoft.Maui.PathAspect
core	maui::core::picker_handler	Microsoft.Maui.Handlers.PickerHandler
core	maui::core::progress_bar_handler	Microsoft.Maui.Handlers.ProgressBarHandler

Слой	Система	Еталон
core	maui::core::property_mapper	Microsoft.Maui.PropertyMapper / IPropertyMapper
core	maui::core::property_path	Microsoft.Maui.Controls.Binding Expression.ParsePath +
core	maui::core::radio_button_handler	Microsoft.Maui.Handlers.Radio ButtonHandler
core	maui::core::refresh_view_handler	Microsoft.Maui.Handlers.Refresh ViewHandler
core	maui::core::return_type	Microsoft.Maui.ReturnType
core	maui::core::safe_area_edges	Microsoft.Maui.SafeAreaEdges
core	maui::core::safe_area_regions	Microsoft.Maui.SafeAreaRegions
core	maui::core::scroll_bar_visibility	Microsoft.Maui.ScrollBar Visibility
core	maui::core::scroll_orientation	Microsoft.Maui.ScrollOrientation
core	maui::core::scroll_to_request	Microsoft.Maui.ScrollToRequest
core	maui::core::scroll_view_handler	Microsoft.Maui.Handlers.Scroll ViewHandler
core	maui::core::search_bar_handler	Microsoft.Maui.Handlers.Search BarHandler
core	maui::core::semantics	Microsoft.Maui.Semantics (+ Microsoft.Maui.SemanticHeadingLevel)
core	maui::core::setter_specificity	Microsoft.Maui.Controls.Setter Specificity (internal)
core	maui::core::setter_specificity_list<T>	Microsoft.Maui.Controls.Setter SpecificityList<T> (internal)
core	maui::core::shadow	Microsoft.Maui.Controls.Shadow (the concrete IShadow)
core	maui::core::shape_drawable	Microsoft.Maui.Graphics.Shape Drawable
core	maui::core::shape_view_handler	Microsoft.Maui.Handlers.Shape ViewHandler
core	maui::core::slider_handler	Microsoft.Maui.Handlers.Slider Handler
core	maui::core::stepper_handler	Microsoft.Maui.Handlers.Stepper Handler
core	maui::core::stream_image_source_service	Microsoft.Maui.StreamImageSource Service
core	maui::core::swipe_behavior_on_invoked	Microsoft.Maui.SwipeBehaviorOn Invoked
core	maui::core::swipe_direction	Microsoft.Maui.SwipeDirection ([Flags])

Слой	Система	Еталон
core	maui::core::swipe_item_menu_item_handler	Microsoft.Maui.Handlers.SwipeItemMenuItemHandler
core	maui::core::swipe_mode	Microsoft.Maui.SwipeMode
core	maui::core::swipe_transition_mode	Microsoft.Maui.SwipeTransitionMode
core	maui::core::swipe_view_handler	Microsoft.Maui.Handlers.SwipeViewHandler
core	maui::core::switch_handler	Microsoft.Maui.Handlers.SwitchHandler
core	maui::core::tabbed_page_handler	Microsoft.Maui.Handlers.TabbedViewHandler
core	maui::core::text_alignment	Microsoft.Maui.TextAlignment
core	maui::core::text_decorations	Microsoft.Maui.TextDecorations
core	maui::core::thickness	Microsoft.Maui.Thickness
core	maui::core::time_picker_handler	Microsoft.Maui.Handlers.TimePickerHandler
core	maui::core::uri_image_disk_cache	Microsoft.Maui.UriImageSourceService.DownloadAndCacheImageAsync
core	maui::core::uri_image_source_service	Microsoft.Maui.UriImageSourceService
core	maui::core::view_chrome_ops	the per-platform native-view chrome extensions behind
core	maui::core::view_command_mapper	Microsoft.Maui.Handlers.ViewHandler.ViewCommandMapper (the generic)
core	maui::core::view_focus_ops	the per-platform native first-responder extensions behind
core	maui::core::view_handler<Derived, Virtual, Platform>	Microsoft.Maui.Handlers.ElementHandler +
core	maui::core::view_mapper	Microsoft.Maui.Handlers.ViewHandler.ViewMapper (the generic-IView)
core	maui::core::view_platform_base	the platform-view face of Microsoft.Maui.Handlers.ViewHandler's
core	maui::core::visibility	Microsoft.Maui.Visibility
core	maui::core::web_navigation_event	Microsoft.Maui.WebNavigationEvent
core	maui::core::web_navigation_result	Microsoft.Maui.WebNavigationResult
core	maui::core::web_view_handler	Microsoft.Maui.Handlers.WebViewHandler

Слой	Система	Еталон
core	maui::core::escape_js_string	Microsoft.Maui.Handlers.WebViewHelper.EscapeJsString
core	maui::core::window_handler	Microsoft.Maui.Handlers.WindowHandler
devflow	maui::devflow::agent	(no C# 1:1) — the port's analog of Microsoft's experimental .NET MAUI DevFlow
essentials	maui::devices::sensors::accelerometer	Microsoft.Maui.Devices.Sensors.Accelerometer (static facade)
essentials	maui::application_model::app_actions	Microsoft.Maui.ApplicationModel.AppActions (static facade)
essentials	maui::application_model::app_info	Microsoft.Maui.ApplicationModel.AppInfo (static facade)
essentials	maui::devices::sensors::barometer	Microsoft.Maui.Devices.Sensors.Barometer (static facade)
essentials	maui::devices::battery	Microsoft.Maui.Devices.Battery (static facade)
essentials	maui::application_model::browser	Microsoft.Maui.ApplicationModel.Browser (static facade)
essentials	maui::application_model::clipboard	Microsoft.Maui.ApplicationModel.DataTransfer.Clipboard (static facade)
essentials	maui::devices::sensors::compass	Microsoft.Maui.Devices.Sensors.Compass (static facade)
essentials	maui::networking::connectivity	Microsoft.Maui.Networking.Connectivity (static facade)
essentials	maui::application_model::communication::contact	Microsoft.Maui.ApplicationModel.Communication.Contact
essentials	maui::application_model::communication::contacts	Microsoft.Maui.ApplicationModel.Communication.Contacts
essentials	maui::devices::device_display	Microsoft.Maui.Devices.DeviceDisplay (static facade)
essentials	maui::devices::device_info	Microsoft.Maui.Devices.DeviceInfo (static facade)
essentials	maui::application_model::communication::email	Microsoft.Maui.ApplicationModel.Communication.Email
essentials	maui::application_model::feature_not_supported	Microsoft.Maui.ApplicationModel.FeatureNotSupportedException
essentials	maui::storage::file_picker	Microsoft.Maui.Storage.FilePicker (static facade)
essentials	maui::storage::file_result	Microsoft.Maui.Storage.FileResult / FileBase

Слой	Система	Еталон
essentials	maui::storage::file_system	Microsoft.Maui.Storage.File System (static facade)
essentials	maui::devices::flashlight	Microsoft.Maui.Devices.Flashlight (static facade)
essentials	maui::devices::sensors::geocoding	Microsoft.Maui.Devices.Sensors.Geocoding (static facade)
essentials	maui::devices::sensors::geolocation	Microsoft.Maui.Devices.Sensors.Geolocation (static facade)
essentials	maui::devices::sensors::gyroscope	Microsoft.Maui.Devices.Sensors.Gyroscope (static facade)
essentials	maui::devices::haptic_feedback	Microsoft.Maui.Devices.Haptic Feedback (static facade)
essentials	maui::application_model::launcher	Microsoft.Maui.ApplicationModel.Launcher (static facade)
essentials	maui::devices::sensors::location	Microsoft.Maui.Devices.Sensors.Location (Types/)
essentials	maui::devices::sensors::magnetometer	Microsoft.Maui.Devices.Sensors.Magnetometer (static facade)
essentials	maui::application_model::main_thread	Microsoft.Maui.ApplicationModel.MainThread (static facade)
essentials	maui::media::media_picker	Microsoft.Maui.Media.MediaPicker (static facade)
essentials	maui::devices::sensors::orientation_sensor	Microsoft.Maui.Devices.Sensors.OrientationSensor (static facade)
essentials	maui::application_model::permissions	Microsoft.Maui.ApplicationModel.Permissions (static class + nested)
essentials	maui::application_model::communication::phone_dialer	Microsoft.Maui.ApplicationModel.Communication.PhoneDialer
essentials	maui::devices::sensors::placemark	Microsoft.Maui.Devices.Sensors.Placemark (Types/)
essentials	maui::storage::preferences	Microsoft.Maui.Storage.Preferences (static facade)
essentials	maui::media::screenshot	Microsoft.Maui.Media.Screenshot (static facade)
essentials	maui::storage::secure_accessible	Security.SecAccessible
essentials	maui::storage::secure_storage	Microsoft.Maui.Storage.Secure Storage (static facade)
essentials	maui::accessibility::semantic_screen_reader	Microsoft.Maui.Accessibility.SemanticScreenReader (static facade)
essentials	maui::devices::sensors sensor-reading value types	Microsoft.Maui.Devices.Sensors.*

Слой	Система	Еталон
essentials	maui::application_model:: data_transfer::share	
essentials	maui::application_model:: communication::sms	Microsoft.Maui.ApplicationModel. Communication.Sms (static)
essentials	maui::media::text_to_speech	Microsoft.Maui.Media.TextTo Speech (static facade)
essentials	maui::media::unit_converters	Microsoft.Maui.Media.Unit Converters
essentials	maui::application_model:: version_tracking	Microsoft.Maui.ApplicationModel. VersionTracking (static)
essentials	maui::devices::vibration	Microsoft.Maui.Devices.Vibration (static facade)
essentials	maui::authentication::web_ authenticator	Microsoft.Maui.Authentication. WebAuthenticator (static facade)
graphics	maui::graphics::abstract_ canvas	Microsoft.Maui.Graphics.Abstract Canvas<TState>
graphics	maui::graphics::abstract_ pattern	Microsoft.Maui.Graphics.Abstract Pattern
graphics	maui::graphics::blend_mode	Microsoft.Maui.Graphics.Blend Mode
graphics	maui::graphics::canvas_ defaults	Microsoft.Maui.Graphics.Canvas Defaults
graphics	maui::graphics::canvas_state	Microsoft.Maui.Graphics.Canvas State
graphics	maui::graphics::color	Microsoft.Maui.Graphics.Color
graphics	maui::graphics::colors	Microsoft.Maui.Graphics.Colors
graphics	maui::graphics::corner_radius	Microsoft.Maui.CornerRadius
graphics	maui::graphics::font	Microsoft.Maui.Graphics.Font (+ IFont, FontStyleType, FontWeights)
graphics	maui::graphics::gradient_ paint	Microsoft.Maui.Graphics.Gradient Paint
graphics	maui::graphics::gradient_stop	Microsoft.Maui.Graphics.Paint GradientStop
graphics	maui::graphics::headless_ image	(test/headless stand-in for Microsoft.Maui. Graphics.PlatformImage)
graphics	maui::graphics::horizontal_ alignment	Microsoft.Maui.Graphics. HorizontalAlignment
graphics	maui::graphics::i_canvas	Microsoft.Maui.Graphics.ICanvas
graphics	maui::graphics::i_drawable	Microsoft.Maui.Graphics. IDrawable
graphics	maui::graphics::i_graphics_ image	Microsoft.Maui.Graphics.IImage

Слой	Система	Еталон
graphics	maui::graphics::i_pattern	Microsoft.Maui.Graphics.IPattern
graphics	maui::graphics::i_picture	Microsoft.Maui.Graphics.IPicture
graphics	maui::graphics::i_shape	Microsoft.Maui.Graphics.IShape
graphics	maui::graphics::image_paint	Microsoft.Maui.Graphics.Image Paint
graphics	maui::graphics::line_cap	Microsoft.Maui.Graphics.LineCap
graphics	maui::graphics::line_join	Microsoft.Maui.Graphics.LineJoin
graphics	maui::graphics::linear_ gradient_paint	Microsoft.Maui.Graphics.Linear GradientPaint
graphics	maui::graphics::matrix3x2	System.Numerics.Matrix3x2
graphics	maui::graphics::paint	Microsoft.Maui.Graphics.Paint
graphics	maui::graphics::paint_pattern	Microsoft.Maui.Graphics.Paint Pattern
graphics	maui::graphics::path_builder	Microsoft.Maui.Graphics.Path Builder
graphics	maui::graphics::path_f	Microsoft.Maui.Graphics.PathF (+ PathOperation.cs, ArcFlattener.cs)
graphics	maui::graphics::path_ operation	Microsoft.Maui.Graphics.Path Operation
graphics	maui::graphics::pattern_paint	Microsoft.Maui.Graphics.Pattern Paint
graphics	maui::graphics::picture_ pattern	Microsoft.Maui.Graphics.Picture Pattern
graphics	maui::graphics::point	Microsoft.Maui.Graphics.Point
graphics	maui::graphics::point_f	Microsoft.Maui.Graphics.PointF
graphics	maui::graphics::quaternion	System.Numerics.Quaternion
graphics	maui::graphics::radial_ gradient_paint	Microsoft.Maui.Graphics.Radial GradientPaint
graphics	maui::graphics::rect	Microsoft.Maui.Graphics.Rect
graphics	maui::graphics::rect_f	Microsoft.Maui.Graphics.RectF
shapes	maui::graphics::shapes:: ellipse	Microsoft.Maui.Controls.Shapes. Ellipse (as an IShape clip)
shapes	maui::graphics::shapes:: rectangle	Microsoft.Maui.Controls.Shapes. Rectangle (as an IShape clip)
shapes	maui::graphics::shapes:: round_rectangle	Microsoft.Maui.Controls.Shapes. RoundRectangle (IShape clip)
graphics	maui::graphics::size	Microsoft.Maui.Graphics.Size
graphics	maui::graphics::size_f	Microsoft.Maui.Graphics.SizeF
graphics	maui::graphics::solid_paint	Microsoft.Maui.Graphics.Solid Paint
graphics	maui::graphics::system_ background_paint	(no direct C# type — models UIColor. SystemBackground /

Слой	Система	Еталон
text	maui::graphics::text:: attributed_text	Microsoft.Maui.Graphics.Text. AttributedText
text	maui::graphics::text:: attributed_text_run	Microsoft.Maui.Graphics.Text. AttributedTextRun
text	maui::graphics::text::i_ attributed_text	Microsoft.Maui.Graphics.Text. IAttributedText
text	maui::graphics::text::text_ attributes	Microsoft.Maui.Graphics.Text. TextAttributes
graphics	maui::graphics::text_flow	Microsoft.Maui.Graphics.TextFlow
graphics	maui::graphics::vector2	System.Numerics.Vector2
graphics	maui::graphics::vector3	System.Numerics.Vector3
graphics	maui::graphics::vector4	System.Numerics.Vector4
graphics	maui::graphics::vertical_ alignment	Microsoft.Maui.Graphics.Vertical Alignment
graphics	maui::graphics::winding_mode	Microsoft.Maui.Graphics.Winding Mode
hosting	maui::hosting::i_lifecycle_ builder	Microsoft.Maui.LifecycleEvents. ILifecycleBuilder
hosting	maui::hosting::i_lifecycle_ event_service	Microsoft.Maui.LifecycleEvents. ILifecycleEventService
hosting	maui::hosting::i_maui_ handlers_collection	Microsoft.Maui.Hosting.IMaui HandlersCollection
hosting	maui::hosting::ios_lifecycle_ builder	Microsoft.Maui.LifecycleEvents. IiOSLifecycleBuilder
hosting	maui::hosting::lifecycle_ event_service	Microsoft.Maui.LifecycleEvents. LifecycleEventService
hosting	maui::hosting::maui_app	Microsoft.Maui.Hosting.MauiApp
hosting	maui::hosting::maui_app_ builder	Microsoft.Maui.Hosting.MauiApp Builder
hosting	maui::hosting::maui_context	Microsoft.Maui.MauiContext
hosting	maui::hosting::add_maui_ controls_handlers	Microsoft.Maui.Controls.Hosting. AppHostBuilderExtensions
hosting	maui::hosting::maui_handlers_ collection	Microsoft.Maui.Hosting.Internal. MauiHandlersCollection
layouts	maui::layouts::absolute_ layout_flags	Microsoft.Maui.Layouts.Absolute LayoutFlags
layouts	maui::layouts::absolute_ layout_manager	Microsoft.Maui.Layouts.Absolute LayoutManager
layouts	maui::layouts::flex_basis	Microsoft.Maui.Layouts.FlexBasis
layouts	maui::layouts flex enums	Microsoft.Maui.Layouts.{Flex Direction, FlexJustify, FlexAlign Content,

Слой	Система	Еталон
layouts	maui::layouts::flex_layout_manager	Microsoft.Maui.Layouts.FlexLayoutManager
layouts	maui::layouts::grid_layout_manager	Microsoft.Maui.Layouts.GridLayoutManager
layouts	maui::layouts::horizontal_stack_layout_manager	Microsoft.Maui.Layouts.HorizontalStackLayoutManager
layouts	maui::layouts::i_layout_manager	Microsoft.Maui.Layouts.ILayoutManager
layouts	maui::layouts::layout_manager	Microsoft.Maui.Layouts.LayoutManager
layouts	maui::layouts::stack_layout_manager	Microsoft.Maui.Layouts.StackLayoutManager
layouts	maui::layouts::vertical_stack_layout_manager	Microsoft.Maui.Layouts.VerticalStackLayoutManager
ui	maui::ui::app	the consumer-facing application base (hides maui::core::i_window from create_window()).
xaml	maui::xaml::hydration_context	Microsoft.Maui.Controls.Xaml.HydrationContext
xaml	maui::xaml::i_markup_extension	Microsoft.Maui.Controls.Xaml.IMarkupExtension
xaml	maui::xaml - the v1 markup extensions (M7 wave 2)	src/Controls/src/Xaml/MarkupExtensions/*:
xaml	maui::xaml::name_scope	Microsoft.Maui.Controls.Internals.NameScope (+ INamespace)
xaml	maui::xaml::xaml_binding_applier	the ApplyPropertiesVisitor.TrySetBinding seam
xaml	maui::xaml string -> value converters (M7)	the C# TypeConverters the XAML loader applies.
xaml	maui::xaml::xaml_loader	Microsoft.Maui.Controls.Xaml.XamlLoader (XamlLoader.cs)
xaml	maui::xaml - the XAML node tree	Microsoft.Maui.Controls.Xaml(XamlNode.cs)
xaml	maui::xaml::xaml_parse_exception	Microsoft.Maui.Controls.Xaml.XamlParseException
xaml	maui::xaml::xaml_parser	Microsoft.Maui.Controls.Xaml.XamlParser (the parse half)
xaml	maui::xaml::register_runtime_bindings	ApplyPropertiesVisitor.TrySetBinding
xaml	maui::xaml::device_platform	Microsoft.Maui.Devices.DevicePlatform

Слой	Система	Еталон
xaml	maui::xaml - DataTemplate body inflation (W4)	ApplyPropertiesVisitor.SetTemplate +
xaml	maui::xaml - the loader's visitor pipeline (M7 wave 2)	src/Controls/src/Xaml/*Visitor.cs:

Приложение В. Пълна таблица на съвпадението по интерфейси

Топлинната карта (Табл. 15) дава измереното структурно сходство за всеки интерфейс на всяка платформа. Цветът на клетката следва стойността: зелено при точно съвпадение (1,000), жълто при 0,75 и червено при 0,50 и по-ниско, с плавен преход между тях. Прочит по ред показва как един интерфейс се държи на четирите платформи; прочит по колона — къде една платформа се отклонява системно.

Таблица 15. Съвпадение по интерфейси и платформи. Клетката носи структурното сходство за ДВАТА входни пътя — по код /от описанието

Интерфейс	iOS	Mac Catalyst	Android	Windows
Absolute Layout	0.991 / 1.000	0.995 / 0.997	0.990 / 1.000	0.995 / 0.999
Activity Indicator	0.952 / 0.952	0.998 / 0.996	0.998 / 0.998	0.990 / 0.983
Adaptive Collection	0.999 / 1.000	0.997 / 0.997	0.998 / 1.000	0.999 / 1.000
Alerts	1.000 / 1.000	0.998 / 0.997	1.000 / 1.000	1.000 / 1.000
Alignment	0.996 / 0.996	0.997 / 0.996	0.985 / 0.985	1.000 / 1.000
Animation	0.934 / 0.934	0.998 / 0.997	1.000 / 1.000	1.000 / 1.000
App Theme Binding	1.000 / 1.000	0.998 / 0.997	1.000 / 1.000	1.000 / 1.000
Application Control	1.000 / 1.000	0.998 / 0.997	1.000 / 1.000	1.000 / 1.000
Auto Size Shapes	1.000 / 1.000	0.998 / 0.992	1.000 / 1.000	0.995 / 0.995
Basic Grouping	1.000 / 1.000	0.998 / 0.997	1.000 / 1.000	1.000 / 1.000
Basic Swipe	1.000 / 1.000	0.998 / 0.997	1.000 / 1.000	0.998 / 0.998
Behaviors	1.000 / 1.000	0.998 / 0.997	1.000 / 1.000	1.000 / 1.000
Border	0.999 / 0.999	0.978 / 0.977	0.959 / 0.959	1.000 / 1.000
Border Clip	0.971 / 0.975	0.989 / 0.990	0.967 / 0.972	0.986 / 0.990
Playground				
Border Layout	1.000 / 1.000	0.997 / 0.996	0.994 / 0.994	0.995 / 0.995
Border Playground	0.999 / 0.999	0.994 / 0.996	0.968 / 0.968	1.000 / 1.000
Border Resize	0.989 / 0.984	0.994 / 0.992	0.975 / 0.956	0.997 / 0.998
Content				
Border Stroke	0.995 / 0.995	0.984 / 0.982	0.930 / 0.930	1.000 / 1.000
Borderless	1.000 / 1.000	0.998 / 0.997	1.000 / 1.000	1.000 / 1.000
Box View	1.000 / 1.000	0.998 / 0.997	1.000 / 1.000	1.000 / 1.000
Button	0.982 / 0.982	0.990 / 0.989	0.996 / 0.996	1.000 / 1.000
Carousel Page	0.921 / 0.921	0.998 / 0.997	0.985 / 0.985	1.000 / 1.000
Chat Example	1.000 / 1.000	0.998 / 0.997	1.000 / 1.000	1.000 / 1.000

Интерфейс	iOS	Mac Catalyst	Android	Windows
Check Box	0.999 / 1.000	0.997 / 0.997	0.999 / 1.000	0.999 / 1.000
Chrome	0.997 / 0.997	0.998 / 0.997	1.000 / 1.000	1.000 / 1.000
Clip	1.000 / 1.000	0.885 / 0.884	0.993 / 0.993	0.999 / 0.999
Clip Corner Radius	1.000 / 1.000	0.998 / 0.997	0.999 / 0.999	1.000 / 1.000
Clip Gallery	1.000 / 1.000	0.998 / 0.997	1.000 / 1.000	1.000 / 1.000
Clip Views	0.998 / 0.999	0.998 / 0.997	0.977 / 0.977	0.998 / 0.998
Clipping	0.994 / 0.994	0.996 / 0.995	0.988 / 0.988	1.000 / 1.000
CollectionView	1.000 / 1.000	0.998 / 0.997	1.000 / 1.000	0.997 / 0.997
Composition Gallery	1.000 / 1.000	0.998 / 0.997	1.000 / 1.000	1.000 / 1.000
Containers	0.990 / 0.991	0.988 / 0.988	0.988 / 0.988	0.988 / 0.988
Content View	1.000 / 1.000	0.998 / 0.997	1.000 / 1.000	1.000 / 1.000
Context Flyout	1.000 / 1.000	0.918 / 0.916	0.728 / 0.728	0.475 / 0.475
Controls Stack	0.995 / 0.996	0.996 / 0.995	0.997 / 0.997	0.998 / 0.998
Custom Layout	0.994 / 1.000	0.998 / 0.997	1.000 / 1.000	1.000 / 1.000
Custom Size Swipe	0.971 / 1.000	0.998 / 0.997	1.000 / 1.000	1.000 / 1.000
Custom Swipe Item View	1.000 / 1.000	0.998 / 0.997	1.000 / 1.000	1.000 / 1.000
Cv Visual States	1.000 / 1.000	0.998 / 0.997	0.981 / 0.981	0.999 / 0.999
Data Template Selector	0.999 / 0.999	0.998 / 0.997	1.000 / 1.000	1.000 / 1.000
Date Picker	0.988 / 0.994	0.998 / 0.997	0.970 / 0.970	0.997 / 1.000
Device	0.999 / 1.000	0.992 / 0.997	0.988 / 1.000	0.991 / 1.000
Dispatcher	1.000 / 1.000	0.998 / 0.997	1.000 / 1.000	1.000 / 1.000
Drag Drop	1.000 / 1.000	0.998 / 0.997	1.000 / 1.000	0.810 / 0.810
Editor	0.998 / 0.998	0.998 / 0.997	0.998 / 0.998	1.000 / 1.000
Effects	1.000 / 1.000	0.998 / 0.997	1.000 / 1.000	1.000 / 1.000
Ellipse Gallery	1.000 / 1.000	0.998 / 0.997	0.988 / 0.988	0.998 / 0.998
Empty View	1.000 / 1.000	0.998 / 0.997	1.000 / 1.000	1.000 / 1.000
Empty View Load Simulate	0.996 / 0.996	0.998 / 0.997	0.995 / 0.995	0.993 / 0.993
Empty View Null	1.000 / 1.000	0.998 / 0.997	0.996 / 0.996	0.994 / 0.994
Empty View Rtl	0.997 / 1.000	0.996 / 0.997	0.997 / 1.000	0.995 / 0.997
Empty View Selector	1.000 / 1.000	0.998 / 0.997	1.000 / 1.000	1.000 / 1.000
Empty View Swap	1.000 / 1.000	0.998 / 0.997	0.999 / 0.999	0.997 / 0.997
Empty View Template	1.000 / 1.000	0.998 / 0.997	1.000 / 1.000	0.997 / 0.997
Empty View View	1.000 / 1.000	0.998 / 0.997	1.000 / 1.000	0.997 / 0.997
Entry	1.000 / 1.000	0.998 / 0.997	0.987 / 0.987	1.000 / 1.000
Filter Collection	0.995 / 1.000	0.996 / 0.997	0.992 / 0.999	0.993 / 0.997
Filter Selection	1.000 / 1.000	0.998 / 0.997	1.000 / 1.000	1.000 / 1.000
Flex Layout	1.000 / 1.000	0.998 / 0.997	1.000 / 1.000	1.000 / 1.000
Focus	1.000 / 1.000	0.998 / 0.997	1.000 / 1.000	1.000 / 1.000
Fonts	1.000 / 1.000	0.998 / 0.997	1.000 / 1.000	1.000 / 1.000
Footer Only String	1.000 / 1.000	0.998 / 0.997	1.000 / 1.000	1.000 / 1.000
Formatted Text	0.998 / 1.000	0.997 / 0.997	0.997 / 0.997	0.998 / 1.000

продължение

Интерфейс	iOS	Mac Catalyst	Android	Windows
Gestures	0.996/0.996	0.998/0.997	1.000/1.000	1.000/1.000
Gradient	1.000/1.000	0.998/0.997	1.000/1.000	1.000/1.000
Grid	1.000/1.000	0.998/0.997	0.999/0.999	1.000/1.000
Grid Grouping	1.000/1.000	0.998/0.997	1.000/1.000	0.999/0.999
Grouping No	1.000/1.000	0.998/0.997	1.000/1.000	1.000/1.000
Templates				
Grouping Plus	1.000/1.000	0.998/0.997	1.000/1.000	1.000/1.000
Selection				
Header Footer	1.000/1.000	0.998/0.997	1.000/1.000	1.000/1.000
Header Footer Grid	0.991/0.991	0.992/0.990	1.000/1.000	0.982/0.993
Header Footer Grid	0.976/0.976	0.979/0.978	1.000/1.000	0.996/0.996
Horizontal				
Header Footer	1.000/1.000	0.998/0.997	1.000/1.000	0.989/0.989
Template				
Header Footer View	1.000/1.000	0.998/0.997	1.000/1.000	0.996/0.996
Hit Testing	1.000/1.000	0.998/0.996	0.995/0.995	0.986/0.986
Horizontal Stack	1.000/1.000	0.998/0.997	1.000/1.000	1.000/1.000
Hybrid Web View	0.988/0.988	0.998/0.997	0.803/0.803	0.999/0.999
Image	0.524/0.524	0.998/0.997	0.478/0.999	0.998/0.998
Image Button	0.989/0.989	0.995/0.994	0.985/0.985	1.000/1.000
Indicator	0.984/0.986	0.989/0.989	0.984/0.985	0.990/0.991
Input Controls	0.996/0.996	0.997/0.995	0.999/0.999	1.000/1.000
Input Transparent	1.000/1.000	0.998/0.997	1.000/1.000	0.999/0.999
Invalidate Brush	1.000/1.000	0.998/0.997	1.000/1.000	1.000/1.000
Invalidate Shadow	0.999/0.999	0.997/0.996	0.983/0.983	1.000/1.000
Host				
Ios Blur Effect	0.996/0.996	0.998/0.997	0.971/0.971	1.000/1.000
Ios Date Picker	1.000/1.000	0.998/0.997	1.000/1.000	1.000/1.000
Ios Entry	0.985/1.000	0.998/0.997	1.000/1.000	1.000/1.000
Ios First Responder	1.000/1.000	0.998/0.997	1.000/1.000	1.000/1.000
Ios Pan Gesture	0.996/0.996	0.998/0.997	1.000/1.000	1.000/1.000
Ios Picker	1.000/1.000	0.998/0.997	1.000/1.000	1.000/1.000
Ios Safe Area	1.000/1.000	0.998/0.997	1.000/1.000	1.000/1.000
Ios Scroll View	1.000/1.000	0.998/0.997	1.000/1.000	1.000/1.000
Ios Search Bar	0.999/0.999	0.998/0.997	1.000/1.000	1.000/1.000
Ios Slider Update On	1.000/1.000	0.998/0.997	1.000/1.000	1.000/1.000
Tap				
Ios Swipe Transition	0.995/0.995	0.998/0.997	1.000/1.000	1.000/1.000
Ios Time Picker	0.999/0.999	0.998/0.997	1.000/1.000	1.000/1.000
Items	1.000/1.000	0.998/0.997	1.000/1.000	1.000/1.000
Items Updating Scroll	1.000/1.000	0.998/0.997	1.000/1.000	1.000/1.000
Mode				
Label	0.997/0.997	0.998/0.997	0.986/0.986	1.000/1.000
Layout Is Enabled	1.000/1.000	0.998/0.997	0.976/0.976	1.000/1.000
Line Gallery	1.000/1.000	0.998/0.997	1.000/1.000	1.000/1.000

продължение

Интерфейс	iOS	Mac Catalyst	Android	Windows
Line Join Gallery	1.000 / 1.000	0.998 / 0.997	1.000 / 1.000	1.000 / 1.000
Measure First	1.000 / 1.000	0.998 / 0.997	1.000 / 1.000	1.000 / 1.000
Strategy				
Menu Bar	1.000 / 1.000	0.998 / 0.997	1.000 / 1.000	1.000 / 1.000
Modal	1.000 / 1.000	0.998 / 0.997	1.000 / 1.000	0.999 / 0.999
Multiple Bound	1.000 / 1.000	0.998 / 0.997	1.000 / 1.000	0.997 / 0.997
Selection				
Navigation Gallery	1.000 / 1.000	0.998 / 0.997	1.000 / 1.000	1.000 / 1.000
Nested Collection	1.000 / 1.000	0.998 / 0.997	1.000 / 1.000	1.000 / 1.000
Pan Gesture Events	0.996 / 0.992	0.997 / 0.997	0.997 / 1.000	0.998 / 1.000
Path Aspect Gallery	1.000 / 1.000	0.998 / 0.997	1.000 / 1.000	1.000 / 1.000
Path Gallery	1.000 / 1.000	0.909 / 0.908	1.000 / 1.000	1.000 / 1.000
Path Transform	0.988 / 1.000	0.994 / 0.997	0.986 / 1.000	0.994 / 1.000
String				
Picker	1.000 / 1.000	0.998 / 0.997	0.987 / 0.987	1.000 / 1.000
Pickers	0.996 / 1.000	0.996 / 0.997	0.994 / 1.000	0.998 / 1.000
Pointer Gesture	0.995 / 0.995	0.998 / 0.997	1.000 / 1.000	1.000 / 1.000
Polygon Gallery	1.000 / 1.000	0.998 / 0.997	0.999 / 0.999	0.998 / 0.998
Polyline Gallery	0.985 / 1.000	0.985 / 0.997	0.983 / 1.000	0.982 / 0.998
Preselected Item	1.000 / 1.000	0.998 / 0.997	1.000 / 1.000	1.000 / 1.000
Preselected Items	1.000 / 1.000	0.998 / 0.997	1.000 / 1.000	0.996 / 0.996
Progress Bar	1.000 / 1.000	0.998 / 0.997	0.998 / 0.998	1.000 / 1.000
Radio Button Border	0.942 / 0.942	0.941 / 0.940	0.997 / 0.997	1.000 / 1.000
Radio Button Content	0.976 / 0.976	0.990 / 0.989	0.991 / 0.991	1.000 / 1.000
Radio Button Group	0.992 / 0.992	0.995 / 0.993	0.999 / 0.999	1.000 / 1.000
Radio Button Group	0.992 / 0.992	0.995 / 0.993	0.999 / 0.999	1.000 / 1.000
Binding				
Radio Button Group	0.939 / 0.939	0.972 / 0.971	0.997 / 0.997	1.000 / 1.000
Gallery				
Radio Content	0.970 / 0.970	0.984 / 0.983	0.998 / 0.998	1.000 / 1.000
Properties				
Radio Template From	0.996 / 1.000	0.995 / 0.997	0.989 / 0.992	0.996 / 0.996
Style				
Rectangle Gallery	1.000 / 1.000	0.998 / 0.997	0.981 / 0.981	0.998 / 0.998
Refresh View	1.000 / 1.000	0.998 / 0.997	1.000 / 1.000	0.995 / 0.995
Relative Layout	1.000 / 1.000	0.998 / 0.997	1.000 / 1.000	0.999 / 0.999
Scattered Radio	0.993 / 0.993	0.996 / 0.994	0.992 / 0.992	1.000 / 1.000
Button				
Scroll Mode Test	0.996 / 1.000	0.996 / 0.997	0.996 / 1.000	0.997 / 1.000
Scroll To Group	1.000 / 1.000	0.998 / 0.997	1.000 / 1.000	1.000 / 1.000
Scroll View	0.999 / 1.000	0.997 / 0.997	1.000 / 1.000	1.000 / 1.000
Search Bar	0.998 / 0.998	0.998 / 0.997	0.993 / 0.993	1.000 / 1.000
Selection Command	1.000 / 1.000	0.998 / 0.997	1.000 / 1.000	1.000 / 1.000
Param				

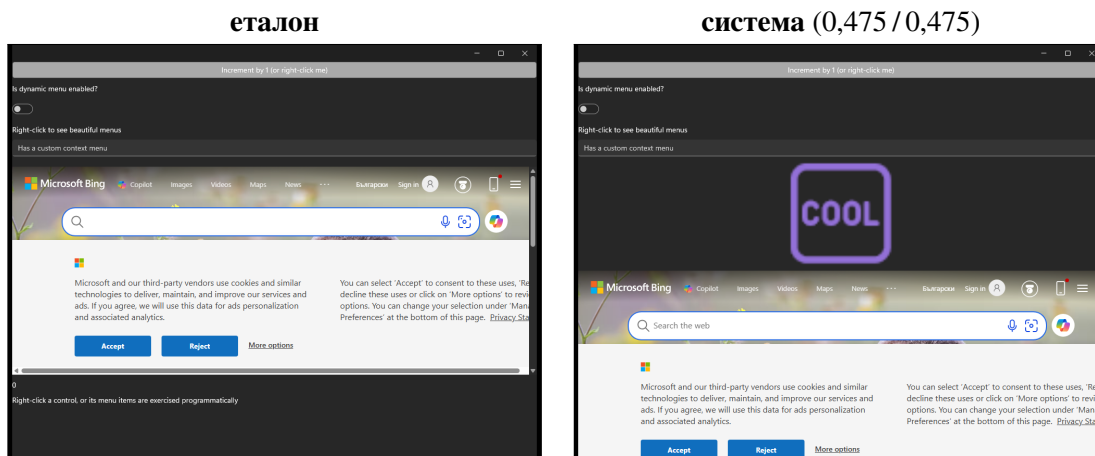
продължение

Интерфейс	iOS	Mac Catalyst	Android	Windows
Selection	1.000 / 1.000	0.998 / 0.997	0.987 / 0.987	1.000 / 1.000
Synchronization				
Semantics	1.000 / 1.000	0.998 / 0.997	0.999 / 0.999	1.000 / 1.000
Shadow Playground	1.000 / 1.000	0.998 / 0.997	0.999 / 0.999	1.000 / 1.000
Shape App Theme	1.000 / 1.000	0.958 / 0.958	1.000 / 1.000	1.000 / 1.000
Shapes	1.000 / 1.000	0.998 / 0.997	1.000 / 1.000	1.000 / 1.000
Single Bound	1.000 / 1.000	0.998 / 0.997	1.000 / 1.000	1.000 / 1.000
Selection				
Slider	1.000 / 1.000	0.998 / 0.997	0.993 / 0.993	1.000 / 1.000
Some Empty Groups	0.990 / 1.000	0.994 / 0.997	0.994 / 1.000	0.994 / 1.000
Stack Layout	1.000 / 1.000	0.998 / 0.997	1.000 / 1.000	1.000 / 1.000
Staggered Layout	1.000 / 1.000	0.998 / 0.997	1.000 / 1.000	0.998 / 0.998
Stepper	1.000 / 1.000	0.998 / 0.997	0.999 / 0.999	0.990 / 0.990
Styles	1.000 / 1.000	0.998 / 0.997	1.000 / 1.000	1.000 / 1.000
Swipe Gesture	0.996 / 0.996	0.998 / 0.997	1.000 / 1.000	1.000 / 1.000
Swipe Item Position	0.991 / 0.991	0.998 / 0.997	0.997 / 0.997	1.000 / 1.000
Swipe Item Size	1.000 / 1.000	0.641 / 0.640	1.000 / 1.000	1.000 / 1.000
Swipe Refresh	0.967 / 0.967	0.990 / 0.989	1.000 / 1.000	0.993 / 0.993
Swipe Threshold	1.000 / 1.000	0.998 / 0.997	1.000 / 1.000	1.000 / 1.000
Swipe View Margin	1.000 / 1.000	0.998 / 0.997	1.000 / 1.000	1.000 / 1.000
Swipe View Shadow	0.999 / 1.000	0.997 / 0.997	0.987 / 0.987	1.000 / 1.000
Switch	0.999 / 0.999	0.997 / 0.996	0.998 / 0.998	1.000 / 1.000
Switch Grouping	1.000 / 1.000	0.998 / 0.997	0.999 / 0.999	1.000 / 1.000
Tabbed Flyout	1.000 / 1.000	0.998 / 0.997	1.000 / 1.000	1.000 / 1.000
Templated View	1.000 / 1.000	0.998 / 0.997	0.991 / 0.991	1.000 / 1.000
Time Picker	0.986 / 0.988	0.998 / 0.997	0.970 / 0.970	0.998 / 1.000
Title Bar	1.000 / 1.000	0.998 / 0.997	1.000 / 1.000	1.000 / 1.000
Toolbar	1.000 / 1.000	0.998 / 0.997	1.000 / 1.000	1.000 / 1.000
Transform	0.997 / 1.000	0.997 / 0.997	0.995 / 0.999	0.998 / 1.000
Playground				
Transformations	1.000 / 1.000	0.998 / 0.997	0.999 / 0.999	0.997 / 0.997
Triggers	1.000 / 1.000	0.998 / 0.997	1.000 / 1.000	1.000 / 1.000
Update Path Data	1.000 / 1.000	0.998 / 0.997	1.000 / 1.000	1.000 / 1.000
Varied Size Selector	0.998 / 0.998	0.985 / 0.984	0.999 / 0.999	1.000 / 1.000
Vertical Stack	1.000 / 1.000	0.998 / 0.997	1.000 / 1.000	1.000 / 1.000
Visual States	1.000 / 1.000	0.998 / 0.997	0.994 / 0.994	1.000 / 1.000
Web View	0.991 / 0.985	0.994 / 0.989	1.000 / 1.000	0.996 / 1.000
Z Index	1.000 / 1.000	0.998 / 0.997	1.000 / 1.000	0.999 / 0.999

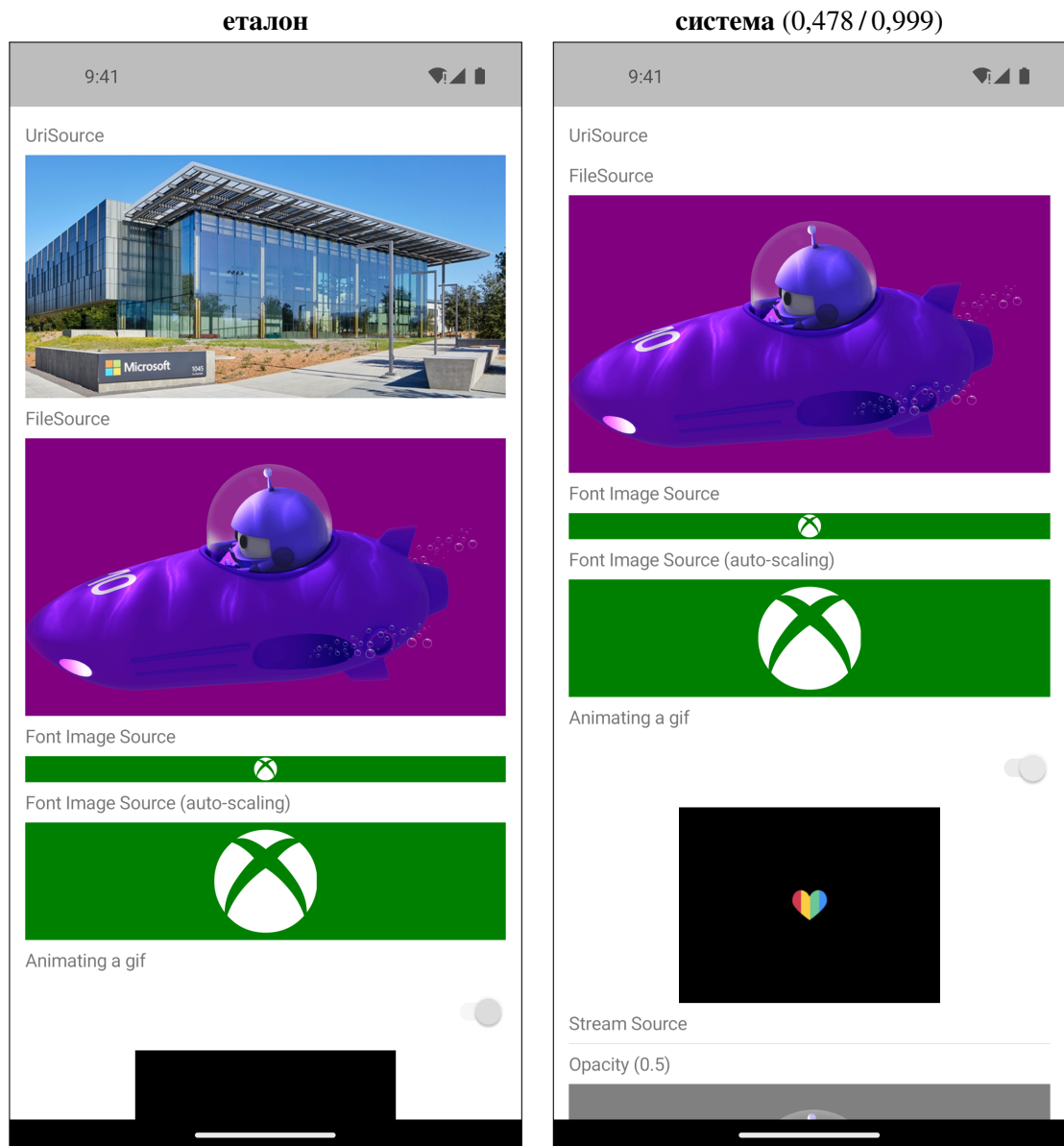
Най-слабо съвпадащите интерфейси — съпоставки

Всеки интерфейс от (Табл. 8) е показан тук един до друг с еталона, в темата, която дърпа оценката надолу — показването на светлия кадър за разминаване, което съществува само в тъмната тема, не илюстрира нищо. Измерените стойности стоят в

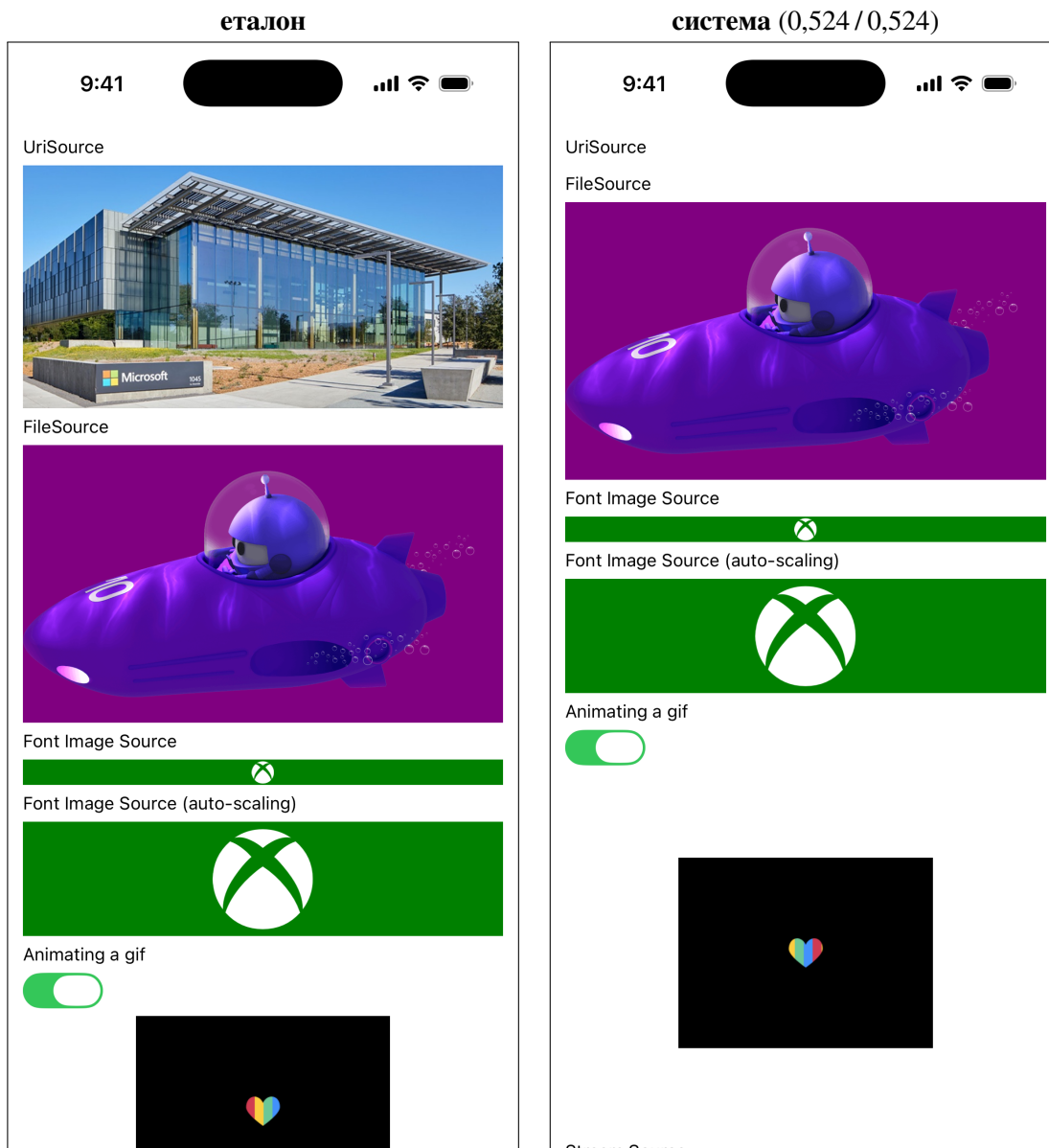
заглавието на дясната колона и в легендата, така че твърдението в текста може да бъде сверено с изображението, вместо да се приема на доверие.



Фигура 16. context_flyout — Windows, тъмна тема. Структурно сходство 0,475 по код и 0,475 от описанието. Живо външно уеб съдържание.



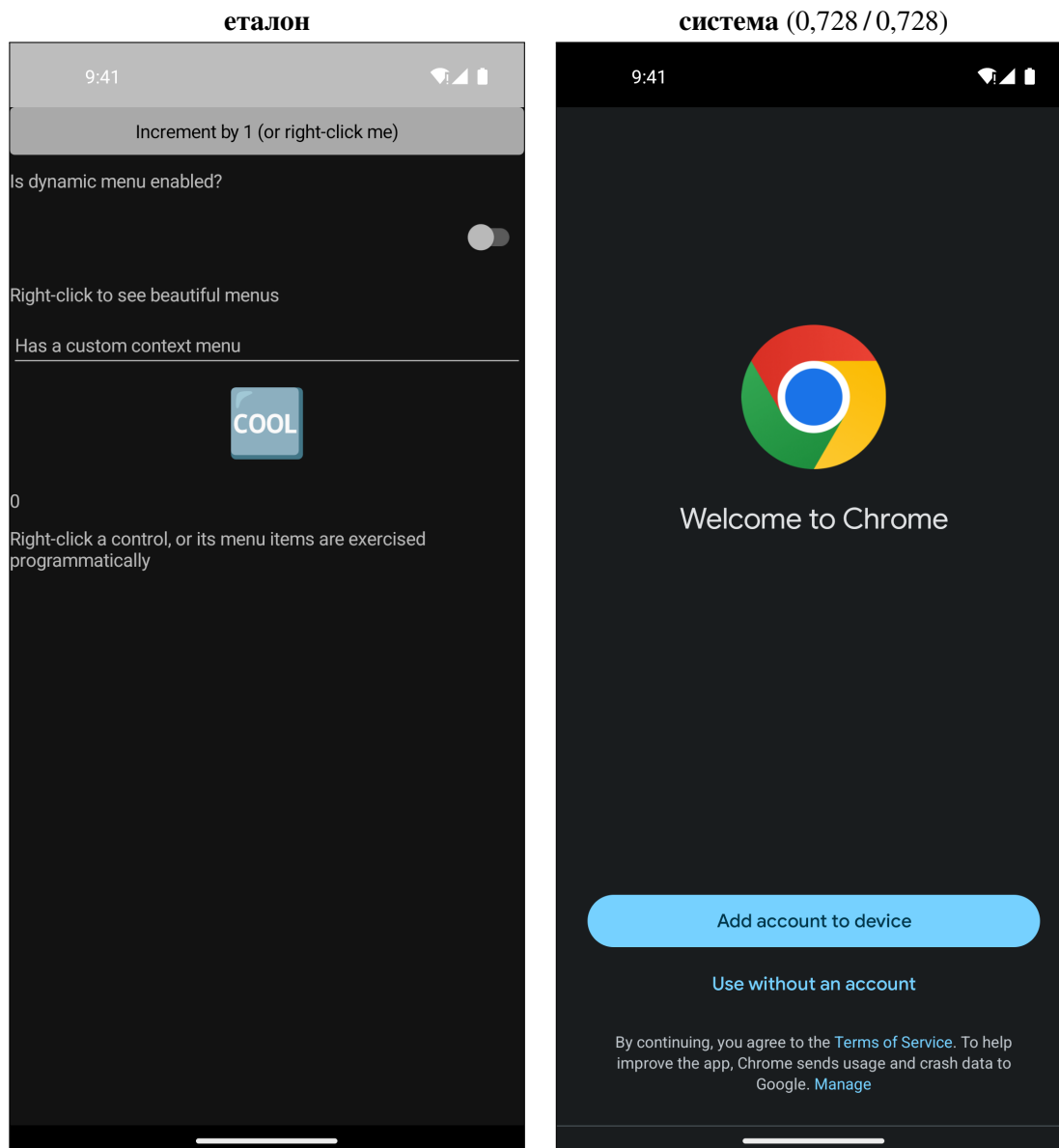
Фигура 17. image — Android, светла тема. Структурно сходство 0,478 по код и 0,999 от описанието. Незареден отдалечен образ.



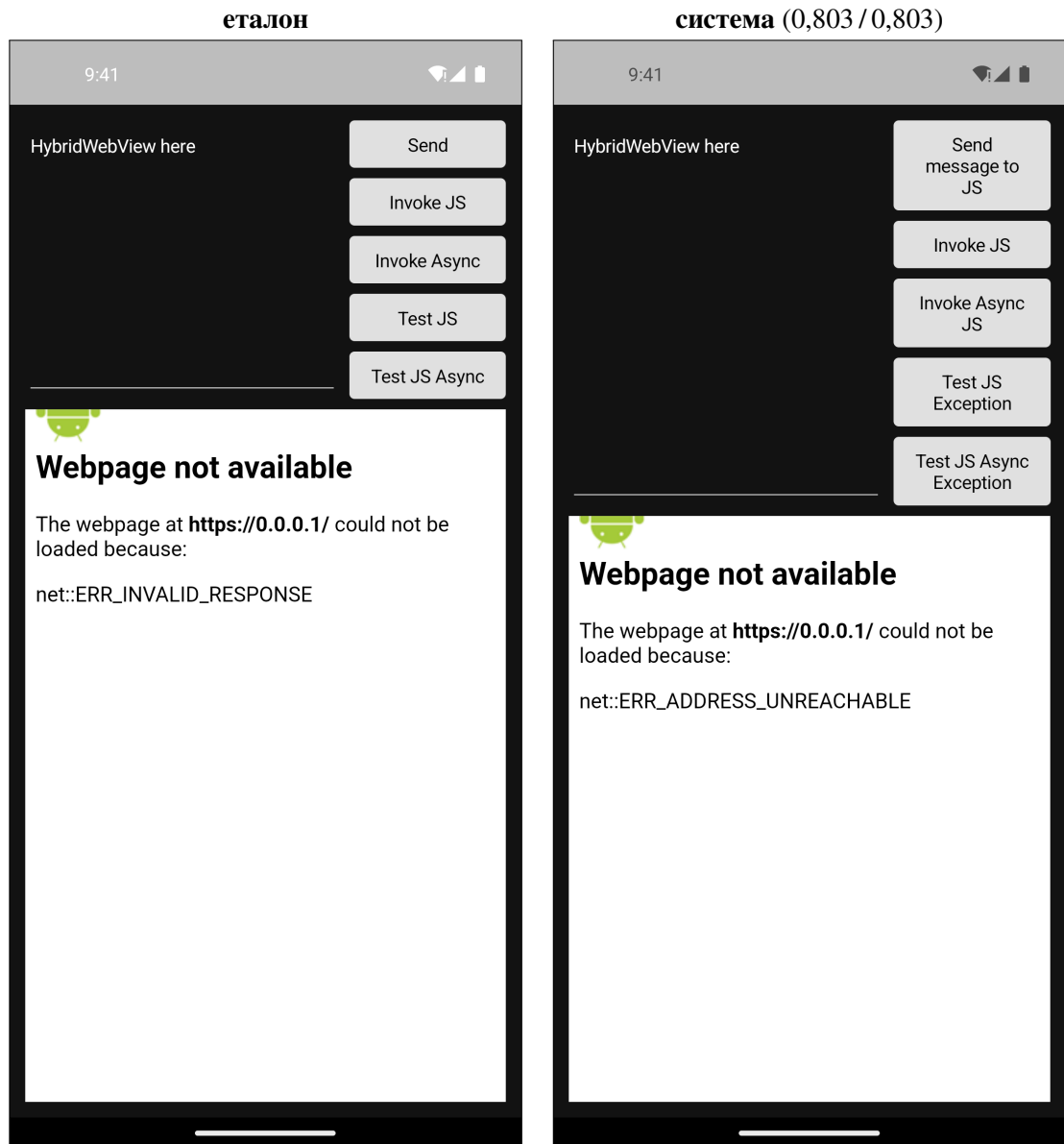
Фигура 18. image — iOS, светла тема. Структурно сходство 0,524 по код и 0,524 от описанието. Незареден отдалечен образ.



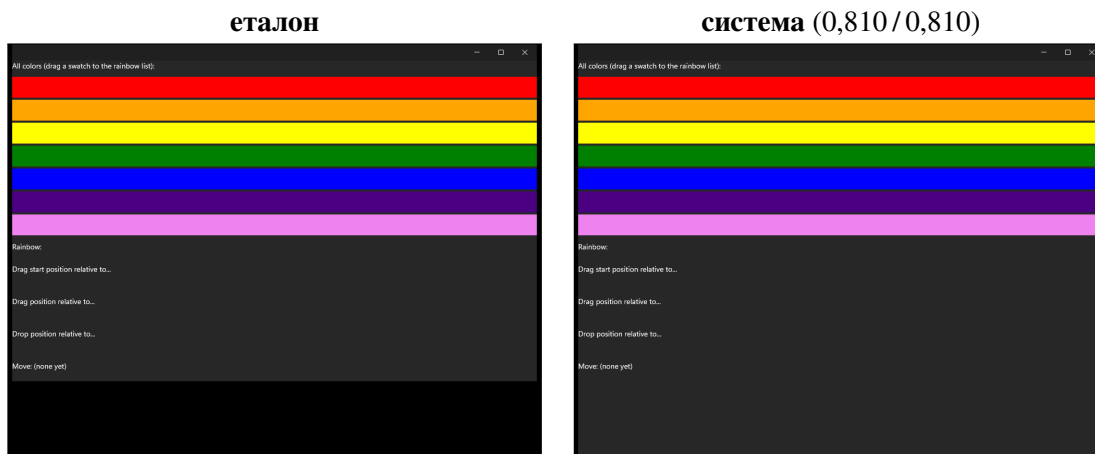
Фигура 19. swipe_item_size — Mac Catalyst, тъмна тема. Структурно сходство 0,641 по код и 0,640 от описанието. Равномерно отместване 32 px (изрязване от рамката).



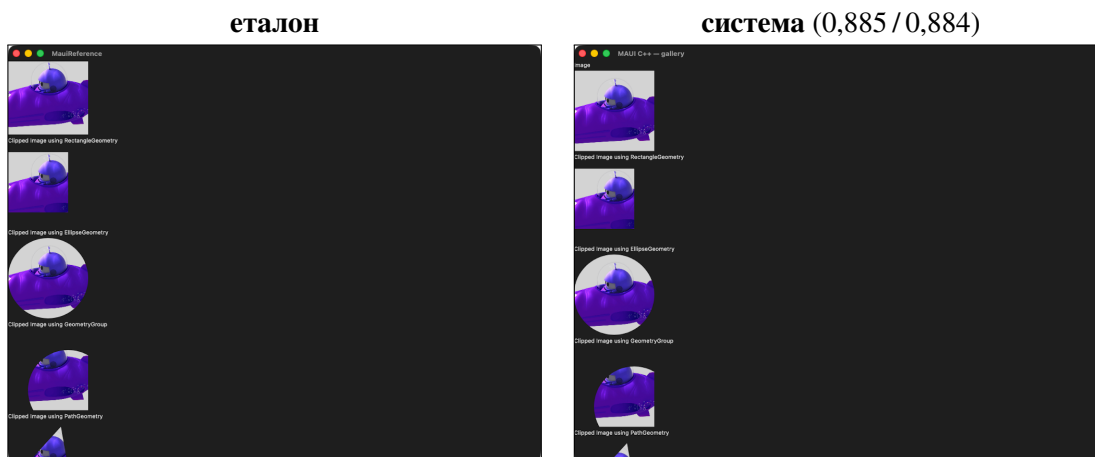
Фигура 20. context_flyout — Android, тъмна тема. Структурно сходство 0,728 по код и 0,728 от описанието. Живо външно уеб съдържание.



Фигура 21. hybrid_web_view — Android, тъмна тема. Структурно сходство 0,803 по код и 0,803 от описанието. Страница за грешка на брауъра.



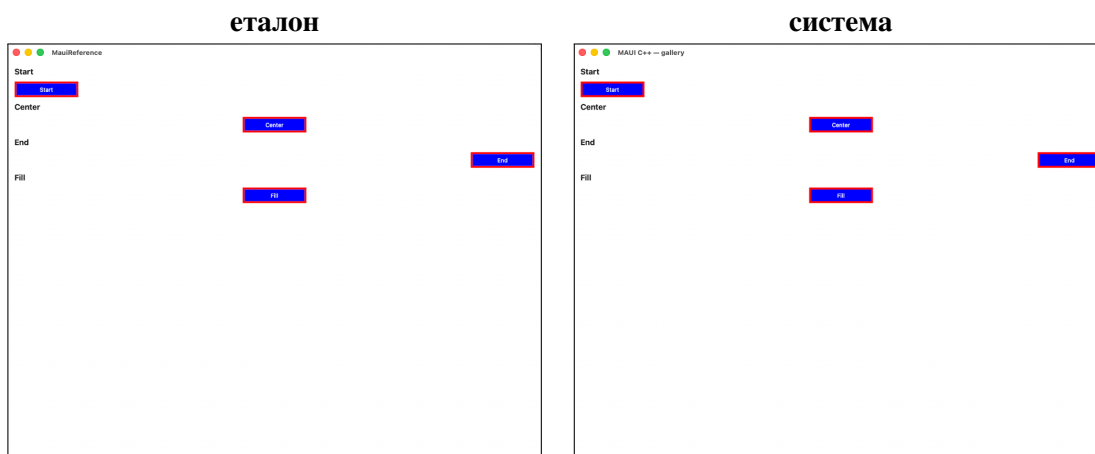
Фигура 22. drag_drop — Windows, тъмна тема. Структурно сходство 0,810 по код и 0,810 от описанието. Непокрита област: черна в еталона, боядисана в системата.



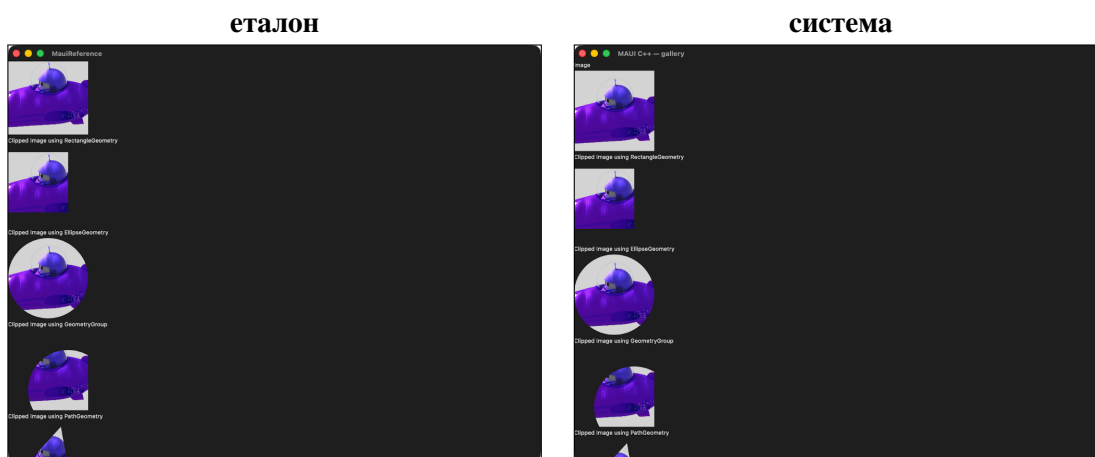
Фигура 23. clip — Mac Catalyst, тъмна тема. Структурно сходство 0,885 по код и 0,884 от описанието. Заглавието на прозореца, не съдържанието.

Приложение Г. Екранни форми

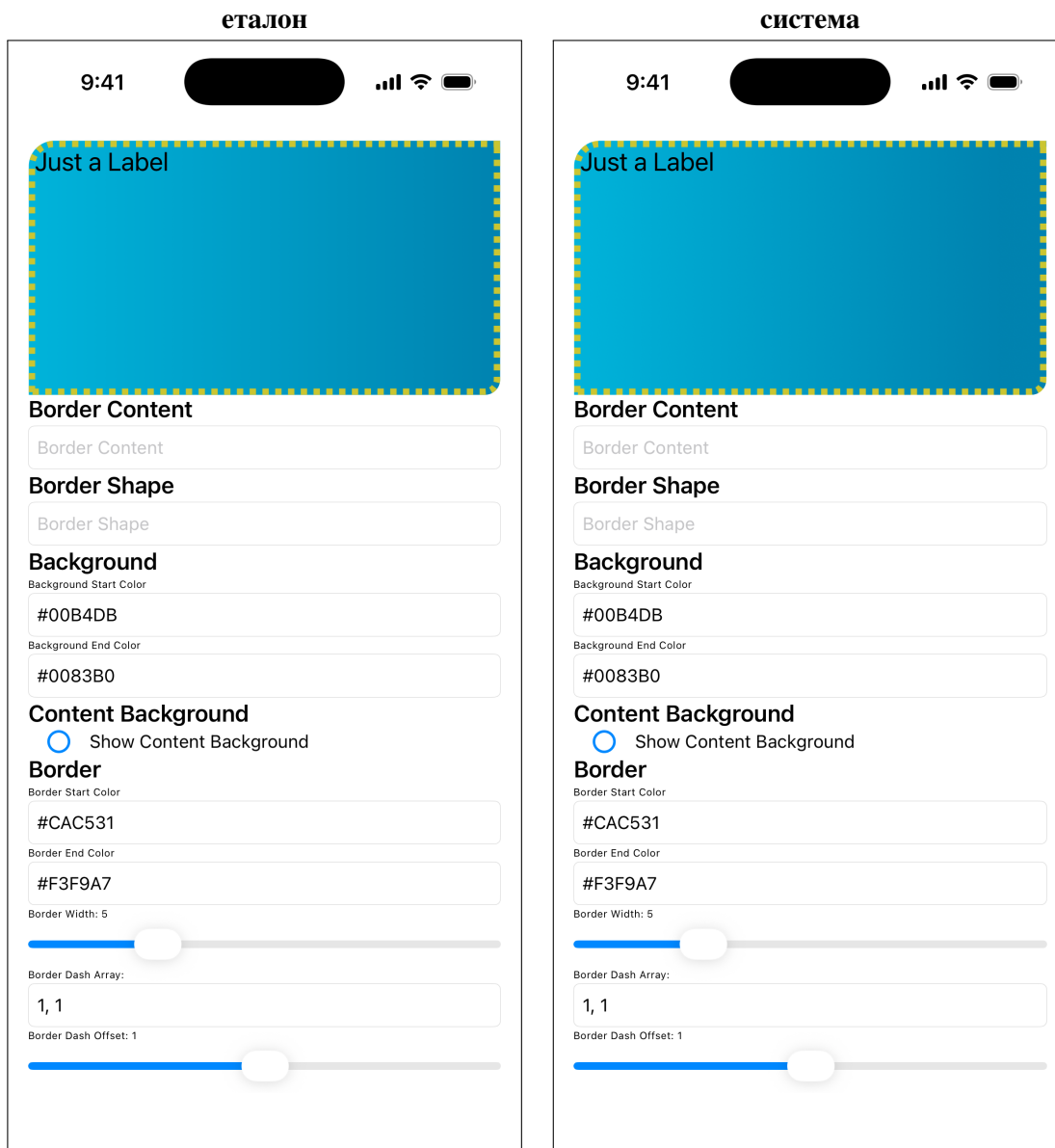
Съпоставки за интерфейси, които не са включени в Раздел VI. Всички са символни връзки към заснеманията от борда, тоест същите файлове, които са били оценени.



Фигура 24. Подравняване — проверява разположението без собствено изобразяване на контролите.



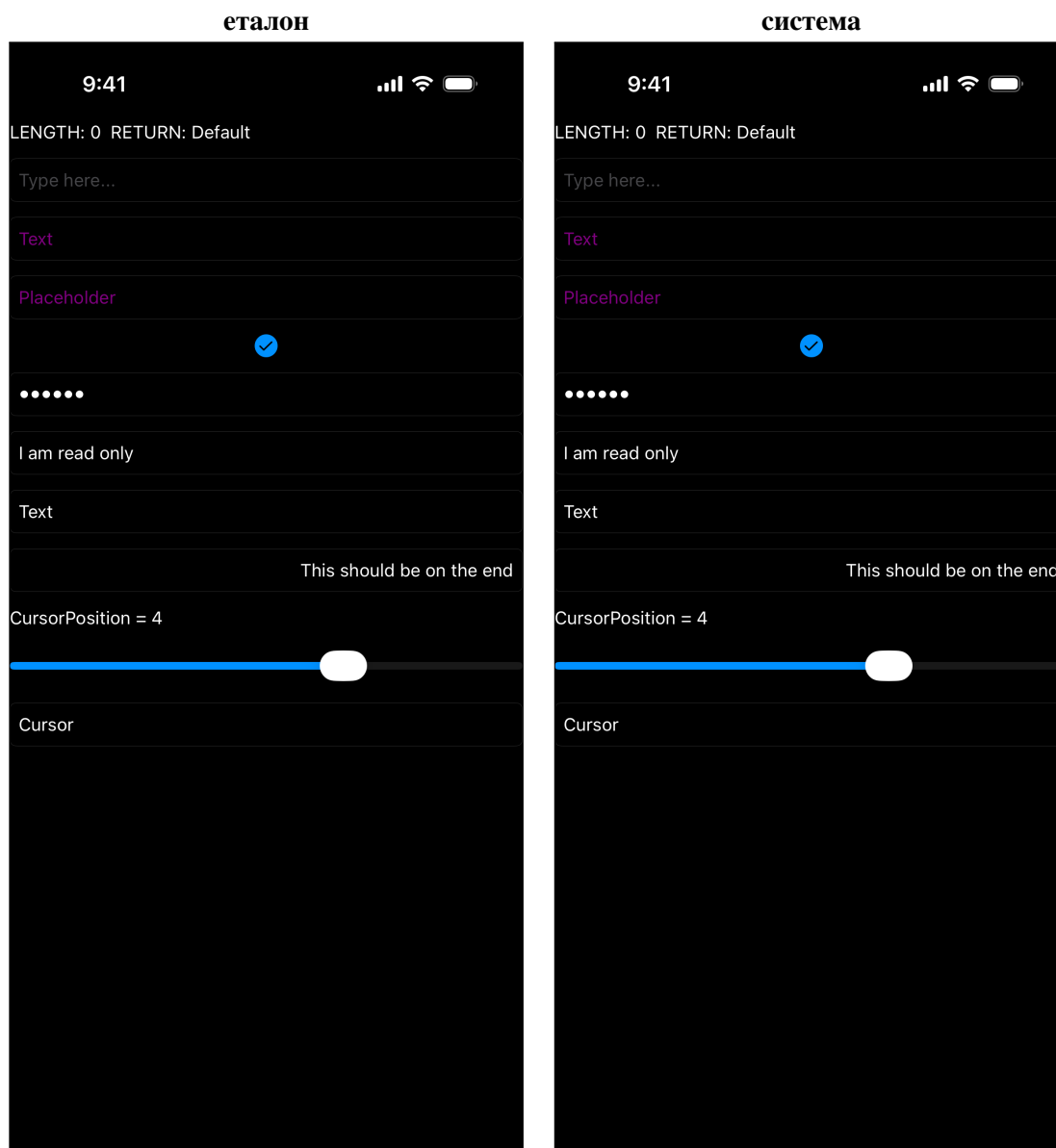
Фигура 25. Отсичане, тъмна тема — класът, при който половинпикселното свиване на контура се проявява (§6.3).



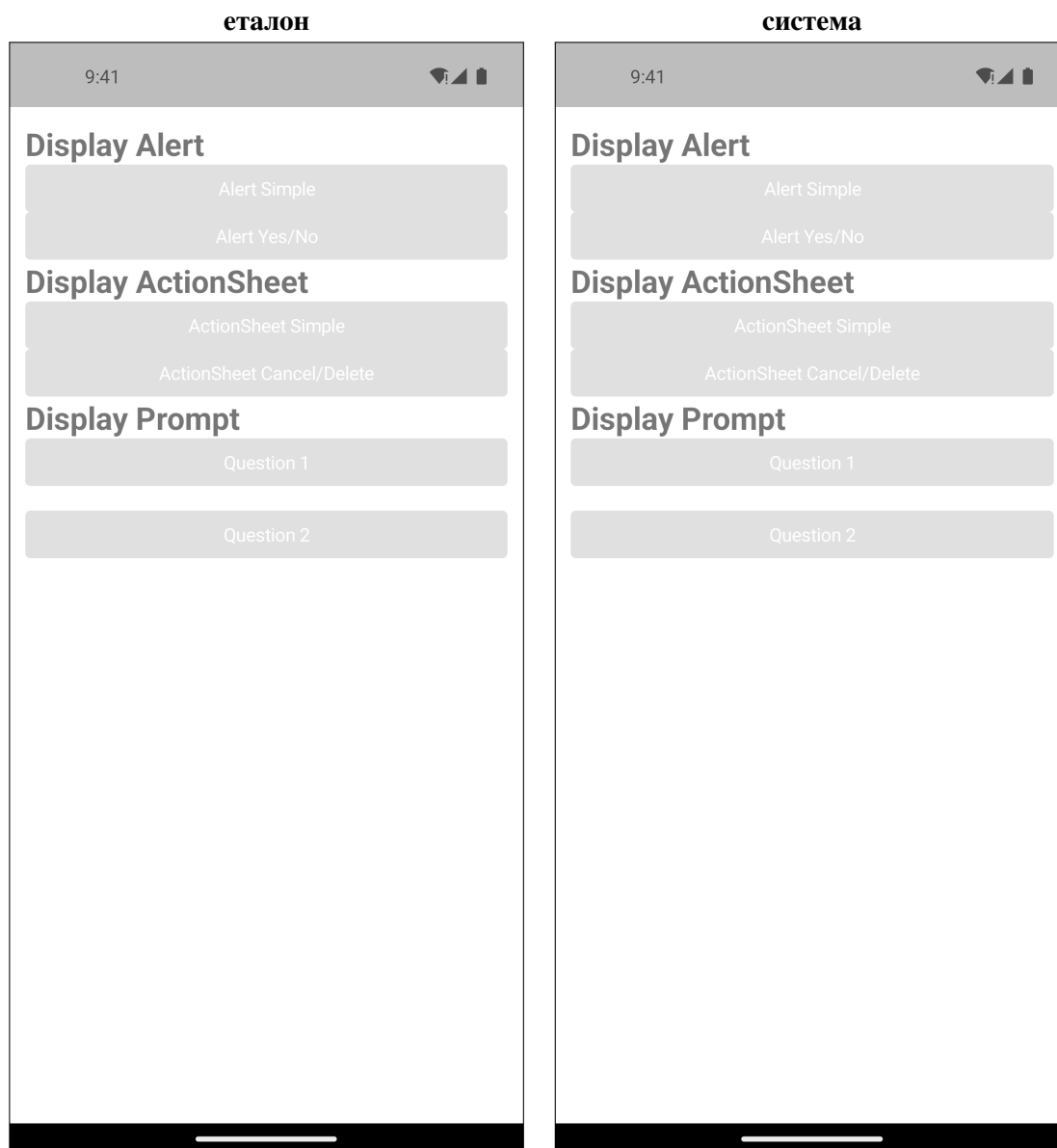
Фигура 26. Рамки и радиуси — интерфейсьт, върху който е измерена промяната в размера на превключвателя (§2.3).



Фигура 27. Решетка — распределяне на пропорционални измерения.



Фигура 28. Поле за въвеждане, тъмна тема — интерфейсът, върху който се прояви размерът на превключвателя (§2.3).



Фигура 29. Съобщения — native диалози на трета среда.

За анимираните интерфейси (14 на брой) съпоставката би показала първи кадър; тук такива не са включени.